

T1

B

简单的概率统计

时间限制: 1000 ms 内存限制: 65536 kb
总通过人数: 130 总提交人数: 180

题目描述

斐波那契的定义如下：

- $F_0 = F_1 = 1, F_n = F_{n-1} + F_{n-2}$

但是这个式子太平凡了，我们改写成如下形式：

- $F'_0 = 1, F'_n = F'_i + F'_j$, 其中 i, j 在 $[0, n-1]$ 中随机等概率选择，可以相同。

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d",&n);
    printf("%d",n+1);
    return 0;
}
```

C

分形图2

时间限制: 1000 ms 内存限制: 65536 kb
总通过人数: 2 总提交人数: 6

题目背景

分形通常被定义为“一个粗糙或零碎的几何形状，可以分成数个部分，每个部分都是（至少近似是）整体缩小后的形状”，即具有自相似的形状。

某种分形定义如下：

```
#include <bits/stdc++.h>

using namespace std;
```

```

const int N = 3000;

char Map[5][5]; int n;
bool Ans[N][N], fractal[3][3];

int _base[] = {0, 0, 1, 3, 9, 27, 81, 243, 729};
bool draw(int dep, int x, int y) {
    if (dep == 1) return Ans[x][y] = true; int base = _base[dep];
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
            if (fractal[i][j])
                draw(dep - 1, x + i * base, y + j * base);
    return true;
}

int main() {
    int T; scanf("%d", &T); while (T--) {
        memset(Map, 0, sizeof Map);
        scanf("%d", &n); getchar(); fgets(Map[0], 5, stdin); fgets(Map[1], 5,
stdin); fgets(Map[2], 5, stdin);
        for (int i = 0; i < 3; ++i)
            for (int j = 0; j < 3; ++j)
                fractal[i][j] = Map[i][j] == 'o';
        memset(Ans, 0, sizeof Ans); draw(n, 0, 0);
        int totalLine = n == 1 ? 1 : 3 * _base[n];
        for (int i = 0; i < totalLine; ++i, puts(""))
            for (int j = 0; j < totalLine; ++j)
                printf("%c", Ans[i][j] ? 'o' : ' ');
        puts("");
    }
    return 0;
}

```

D

D 星际穿越

时间限制：1000ms 内存限制：65536kb

通过率： / (%) 正确率： / (%)

题目描述

为了应对资源枯竭的问题，2300 年的人们决定使用跃迁引擎技术整体迁移到另一个宜居星球上，该星球距离地球的距离是 n 。

以地球的科技水平，可以制造出 m 个不同型号的跃迁引擎，每一次引擎启动，都可以前进恰好 a_i 个单位长度， $1 \leq i \leq m$ 。在每次跃迁之后，可以选择更换引擎，或者继续使用当前引擎。请你计算一下，最少要跃迁几次，才能到达目标星球？

注意，跃迁的方向一开始是定死的，也就是中间不能拐弯，也不能越过目标星球再向后走。数据保证一定可以到达目标星球，不会存在只能走过头的情况。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)

```

```

#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 2020
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,j;
int a[N],f[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    n=read();    m=read();
    for (i=1;i<=m;i++)    a[i]=read();
    memset(f,INF,sizeof(f));
    f[0] = 0;
    for (i=1;i<=m;i++)
        for (j=a[i];j<=n;j++)
            f[j] = Min(f[j], f[j-a[i]]+1);
    cout<<f[n]<<endl;
    return 0;
}

```

E

E 拆卸玩具

时间限制：1000ms 内存限制：65536kb

通过率：25/37 (67.57%) 正确率：25/65 (38.46%)

题目描述

现有一个玩具，它由n个块和m个杆构成，两个块之间可能有一个杆连接。初始时，各个块之间通过杆直接或间接连接，使得玩具是一个整体。每个块有一个编号（从1到n），每个杆有一个价值。

现在你需要通过拆除杆来让这些块互不相连。你一共要进行n-1次拆卸。

每次拆卸你会先选择一个块，把所有与它相连的杆全部拆卸，从而拆卸下这个块。你需要保证在拆卸后，所有未被你选择的块仍被杆连接成一个整体。

每次拆卸，你可以获得你本次拆卸的所有杆的价值的最大值的奖励分。

那么你最多一共能获得多少的奖励分呢？

```

#include<bits/stdc++.h>
#define MAXN 100005
#define MAXM 1000005
#define INF 1e9
using namespace std;
struct Edge

```

```

{
    int u,v,w;
} edge[MAXM];
int f[MAXN],n,m,ans,cnt;
int cmp

```

F

F 普通的 Floyd

时间限制：5000ms 内存限制：131072kb

通过率：181/235 (77.02%) 正确率：181/599 (30.22%)

题目描述

这是一个模板题

给定一张 n 个点的图，求任意两点间的最短路

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>

```

```

#include<vector>
#include<iostream>
#include<cmath>
#include<set>
#include<map>
#include<algorithm>
#include<functional>
#include<queue>
#define MAX 1205
#define INF 50000
using namespace std;
int n,m,q;
int main(){
    int x,y,s;
    scanf("%d %d %d",&n,&m,&q);
    int map[n][n];
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(i==j) map[i][j]=0;
            else map[i][j]=INF;
        }
    }
    for(int j=0;j<m;j++){
        scanf("%d %d %d",&x,&y,&s);
        map[x-1][y-1]=s;
        map[y-1][x-1]=s;
    }

    for(int k=0;k<n;k++){
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                map[i][j]=min(map[i][j],map[i][k]+map[k][j]);
            }
        }
    }
    for(int i=0;i<q;i++){
        scanf("%d %d",&x,&y);
        if(map[x-1][y-1]>=INF){
            printf("-1\n");
        }
        else{
            printf("%d\n",map[x-1][y-1]);
        }
    }
}

```

G Gcd Chain

时间限制: 1000ms 内存限制: 65536kb

通过率: 5/9 (55.56%) 正确率: 5/17 (29.41%)

题目描述

最大公约数运算满足结合律, 即对所有整数 a 、 b 和 c , 满足

$$\gcd(a, b, c) = \gcd(a, \gcd(b, c)) = \gcd(\gcd(a, b), c)$$

采用辗转相除法 $\gcd(a, b) = \gcd(b, a \% b)$, 写直接的递归程序求解 a 和 b 的最大公约数。

输入多个整数, 计算他们的最大公约数和结合顺序。不同的结合顺序, 递归调用的次数可能不同。输出递归调用次数最少的结合方式。若有多种结合方式的递归调用的次数相同, 按左结合优先的顺序输出, 比如, 假设输入4个数 a 、 b 、 c 和 d , 若 $\gcd(a, \gcd(b, \gcd(c, d)))$ 和 $\gcd(\gcd(a, b), \gcd(c, d))$ 这两种结合方式执行递归调用的次数最少, 则输出 $((12)(34))$, 表示这四个数用 $\gcd$ 计算时的结合顺序。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL unsigned long long
#define N 510
#define INF 0x3f3f3f3f
using namespace std;

int n,i,j,k,g,cnt;
int f[N][N];
LL x;
LL a[N],b[N][N];

IL LL Gcd(LL a, LL b){
    cnt++;
    if (!b) return a;
    return Gcd(b,a%b);
}

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-') f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL int Cal(LL x, LL y){
    cnt = 0;
    Gcd(x,y);
    return cnt;
}
```

```

IL void Print(int l,int r){
    reg int i=0;
    if (l == r){
        printf("%d",l);
        return;
    }
    putchar('(');
    for (i=r-1;i>=1;i--){
        if (f[l][r] == f[l][i] + f[i+1][r] + cal(b[l][i], b[i+1][r])){
            Print(l,i);
            Print(i+1,r);
            break;
        }
    }
    putchar(')');
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();
    for (i=1;i<=n;i++) scanf("%llu",&a[i]);

    memset(f,INF,sizeof(f));
    for (i=1;i<=n;i++) f[i][i] = 0;
    for (i=1;i<=n;i++) b[i][i] = a[i];

    for (k=1;k<n;k++){
        for (i=1;i<n-k+1;i++){
            j = i+k;
            for (g = i;g < j;g++){
                f[i][j] = Min(f[i][j], f[i][g] + f[g+1][j] + cal(b[i][g], b[g+1][j]));
                b[i][j] = Gcd(b[i][g], b[g+1][j]);
            }
        }
    }
    cout<<b[1][n]<<endl;
    Print(1,n);
    putchar(10);
    cout<<f[1][n]<<endl;
    return 0;
}

```

H

H 简单的二分

时间限制: 1000ms 内存限制: 65536kb

通过率: 153/218 (70.18%) 正确率: 153/446 (34.30%)

题目描述

大鱼吃小鱼是一个非常考验决策力的游戏。

游戏的规则很简单:

- 海洋内除了自己还有 n 条鱼, 第 i 条鱼的大小为 w_i
- 任意时刻只能吃掉大小小于等于自己的鱼
- 当吃了第 i 条鱼后, 鱼的大小会从 x 变成 $x + a_i$

Alex 想知道, 他初始的鱼需要有多大, 才能吃完这 n 条鱼, 吃的顺序可以任意。

```
#include<stdio.h>
#include<math.h>
#include<stdlib.h>
int ans=0,n,w[1000000],a[1000000];
struct yu{
    int w,a;
}fish[1000000];
int cmp(const void *a,const void *b){
    struct yu *c=(struct yu *)a;
    struct yu *d=(struct yu *)b;
    return c->w-d->w;
}
int main(){
    int i,j,k;
    scanf("%d",&n);
    for(i=0;i<n;i++){
        scanf("%d%d",&fish[i].w,&fish[i].a);
    }
    int chi=0;
    qsort(fish,n,sizeof(fish[0]),cmp);
    for(i=0;i<n;i++){
        if(ans<(fish[i].w-chi)){
            ans=fish[i].w-chi;
        }
        chi+=fish[i].a;
    }
    printf("%d",ans);
    return 0;
}
```


I 普通的计算几何

时间限制: 1000ms 内存限制: 65536kb

通过率: 0/43 (0.00%) 正确率: 0/138 (0.00%)

题目描述

游戏棒，又称撒棒或挑棍，是一种民间流传已广的游戏方式，一般为两人游戏，也可多人游戏。

他的规则如下：

把一大把细棒从一定高度一起抛下，使得细棒之间互相碰撞后互相叠在一起，玩家的任务是逐根将细棒向上抬起移出场地。

然而这并不是总能做到，如果最后出现了矛盾那么之前的努力也会有一种白费了的感觉。

那么问题来了，现在 Alex 扔下了一把细棒，但他不想玩一个不完整的游戏，所以判断游戏能否进行完毕的任务就交给你了。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define EPS 1e-5
#define N 2020
#define INF 0x3f3f3f3f
using namespace std;

inline double Min(double a,double b){return (a<b)?a:b;}
inline double Max(double a,double b){return (a>b)?a:b;}
inline double Fabs(double x){return (x<=-EPS)?-x:(x>EPS)?x:0;}
inline bool Equ(double x,double y){return (Fabs(x-y)<EPS);}
inline int Sgn(double x){
    if (Fabs(x)==0) return 0;
    if (x>0) return 1; else return -1;
}

struct Vector{
    double x,y;
    void Clean(void){x=y=0;}
    void write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
    Vector operator +(Vector p){return (Vector){x+p.x,y+p.y};}
    Vector operator -(Vector p){return (Vector){x-p.x,y-p.y};}
    double operator *(Vector p){return x*p.y-y*p.x;}
    double operator &(Vector p){return x*p.x+y*p.y;}
};
typedef Vector V;

struct Point{
    double x,y;
    void Clean(void){x=y=0.0;}
```

```

void write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
bool operator ==(Point a){return (Equ(x-a.x,0) && Equ(y-a.y,0));}
Point operator +(Point p){return (Point){x+p.x,y+p.y};}
Point operator /(double l){return (Point){x/l,y/l};}
Vector operator -(Point p){return (Vector){x-p.x,y-p.y};}
};

typedef Point P;

inline void SW(P &a,P &b){P t=a;    a=b;    b=t;}
inline double Dis2(P a,P b){return pow(a.x-b.x,2)+pow(a.y-b.y,2);}
inline double Dis(P a,P b){return pow(pow(a.x-b.x,2)+pow(a.y-b.y,2),0.5);}

inline bool Point_On_Seg(P x,P y,P z){
    double s;
    s=(x-y)*(x-z);
    if (Fabs(s)>EPS)    return 0;
    if (Min(y.x,z.x)-EPS>x.x || Max(y.x,z.x)+EPS<x.x)    return 0;
    if (Min(y.y,z.y)-EPS>x.y || Max(y.y,z.y)+EPS<x.y)    return 0;
    return 1;
}

inline bool Seg_Cross(P a,P b,P c,P d){
    if (Point_On_Seg(a,c,d) || Point_On_Seg(b,c,d) || Point_On_Seg(c,a,b) ||
Point_On_Seg(d,a,b))    return 1;
    if (Sgn((b-a)*(c-a)) * Sgn((b-a)*(d-a)) >= 0)    return 0;
    if (Sgn((d-c)*(a-c)) * Sgn((d-c)*(b-c)) >= 0)    return 0;
    return 1;
}

inline int Seg_Cross_P(P a,P b,P c,P d,P &e){
    V A=b-a,B=d-c;    e=a;
    if (!Seg_Cross(a,b,c,d))    return 0;
    if (Fabs(A*B) == 0){
        if (Point_On_Seg(a,c,d))    e=a;
        if (Point_On_Seg(b,c,d))    e=b;
        if (Point_On_Seg(c,a,b))    e=c;
        if (Point_On_Seg(d,a,b))    e=d;
        return 1;
    }
    double l=((c-a)*B)/(A*B);
    e.x+=(b.x-a.x)*l;    e.y+=(b.y-a.y)*l;
    return 1;
}

int T,i,n;
int d[N];
P a[N],b[N],c[N];

queue <int> Q;
vector <int> l[N];

IL void Ready(){
    reg int i=0,j=0;
    double u = 0, v = 0;
    memset(d,0,sizeof(d));
    for (i=1;i<=n;i++)    l[i].clear();

    for (i=1;i<=n;i++)

```

```

        for (j=i+1;j<=n;j++){
            if (!Seg_Cross(a[i],b[i],a[j],b[j])) continue;
            P p;
            Seg_Cross_P(a[i],b[i],a[j],b[j], p);

            if (a[i].x != b[i].x)    u = c[i].x + (p.x - a[i].x) * (c[i].y -
c[i].x) / (b[i].x - a[i].x);
            else u = c[i].x + (p.y - a[i].y) * (c[i].y - c[i].x) / (b[i].y -
a[i].y);

            if (b[j].x != a[j].x)    v = c[j].x + (p.x - a[j].x) * (c[j].y -
c[j].x) / (b[j].x - a[j].x);
            else v = c[j].x + (p.y - a[j].y) * (c[j].y - c[j].x) / (b[j].y -
a[j].y);

            if (u < v) {
                d[i]++; l[j].push_back(i);
            } else {
                d[j]++; l[i].push_back(j);
            }
        }
    }

IL void work(){
    reg int i=0,cnt=0;
    for (i=1;i<=n;i++)
        if (!d[i]) Q.push(i);
    while (!Q.empty()){
        int x = Q.front(); Q.pop();
        cnt++;
        for (auto i : l[x]){
            d[i]--;
            if (!d[i]) Q.push(i);
        }
    }
    if (cnt < n)    puts("No");
    else puts("Yes");
}

int main(){
#ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
#endif
    scanf("%d",&T);
    while (T--){
        scanf("%d",&n);
        for (i=1;i<=n;i++)
            scanf("%lf%lf%lf%lf%lf%lf", &a[i].x, &a[i].y, &c[i].x, &b[i].x,
&b[i].y, &c[i].y);
        Ready();
        work();
    }
    return 0;
}

```

J 新年舞会

时间限制: 1000ms 内存限制: 65536kb

通过率: 71/126 (56.35%) 正确率: 71/361 (19.67%)

题目描述

新的一年要来了，软院计划举办一场新年舞会邀请同学们参加。已知有 n 位男生和 m 位女生参加，经过调查，男女生之间两两有一个熟悉程度 a_{ij} 。现在学院想知道，假如一个男生和一个女生之间的熟悉程度不少于 m 才会组成舞伴，那么最多可以结合出多少对舞伴呢？注意，在晚会上一个人至多只能和另一个人组成舞伴。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,j,lsum=1,cnt,flow,a1,Found;
int a[310][310];
int gap[N],gaps[N],head[N];

struct Flow{
    int t,next,f1,re;
}e[N*8];

inline void Add(int s,int t,int f1){
    e[lsum].t=t;    e[lsum].f1=f1;    e[lsum].next=head[s];    e[lsum].re=lsum+1;
    head[s]=lsum++;
    e[lsum].t=s;    e[lsum].f1=0;    e[lsum].next=head[t];    e[lsum].re=lsum-1;
    head[t]=lsum++;
}

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

inline void Sap(int x){
    int i=0,mingap=cnt-1,F=a1;
```

```

if (x==cnt)    {Found=1;    flow+=a1;    return;}
for (i=head[x];i;i=e[i].next)
    if (e[i].fl){
        if (gap[e[i].t]+1==gap[x]){
            a1=Min(a1,e[i].fl);    Sap(e[i].t);
            if (gap[1]>=cnt)    return;
            if (Found)    break;
            a1=F;
        }
        mingap=Min(mingap,gap[e[i].t]);
    }
if (!Found){
    gaps[gap[x]]--;    if (!gaps[gap[x]])    gap[1]=cnt;
    gaps[gap[x]=mingap+1]++;
}    else e[i].fl-=a1,e[e[i].re].fl+=a1;
}

int main(){
    n=read();
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++)
            a[i][j]=read();
    m=read();
    cnt = 2*n+2;
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++)
            if (a[i][j] >= m)    Add(1+i,1+n+j,1);
    for (i=1;i<=n;i++){
        Add(1,1+i,1);
        Add(1+n+i,cnt,1);
    }
    gaps[0] = cnt;
    while (gap[1] < cnt){
        Found = 0;    a1 = INF;    Sap(1);
    }
    cout<<flow<<endl;
    return 0;
}

```

T2

B

B 八进制乘法

时间限制: 1500ms 内存限制: 65536kb

通过率: 47/135 (34.81%) 正确率: 47/340 (13.82%)

题目描述

给出两个超长的八进制正整数（保证不含前导0），请输出它们的乘积。

输入格式

输入共两行，每行有一个八进制正整数。

输出格式

输出一行，表示乘积结果（也是八进制）。不允许有前导0，中间也不允许有多余的空格。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 400010
#define MOD 998244353
#define INF 0x3f3f3f3f
using namespace std;

int n,i,j,len;
char c[N],s[N];
LL a[N],b[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(LL &a,LL &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-') f=-1; c=getchar();}
    while (c>=48&&c<=57) p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}
```

```

}

inline LL Mi(LL x,LL y){
    LL p=x,t=1,Res=1;
    for (;t<=y;(t&y)?Res=(Res*p)%MOD:0,p=(p*p)%MOD,t<<=1);
    return Res;
}

IL void Rader(LL y[]){
    int i=0,j=0,k=0;
    for (i=1,j=len>>1;i<len-1;i++){
        if (i<j) swap(y[i],y[j]);
        k=len>>1;
        while (j>=k) j-=k,k>>=1;
        if (j<k) j+=k;
    }
}

IL void NTT(LL a[],int f){
    int i=0,j=0,k=0;
    LL wn=0,w=0,u=0,t=0;
    Rader(a);
    for (k=2;k<=len;k<<=1){
        wn=Mi(3,(MOD-1)/k);
        if (f== -1) wn=Mi(wn,MOD-2);
        for (i=0;i<len;i+=k){
            w=1;
            for (j=i;j<i+k/2;j++){
                u=a[j]; t=(w*a[j+k/2])%MOD;
                a[j]=(u+t)%MOD; a[j+k/2]=(u-t+MOD)%MOD;
                w=(w*wn)%MOD;
            }
        }
    }
    if (f== -1)
        for (i=0;i<len;i++) a[i]=(a[i]*Mi(len,MOD-2))%MOD;
}

int main(){
    scanf("%s",s);
    scanf("%s",c);

    n=strlen(c);
    for (i=n-1;i>=0;i--) a[n-1-i]=c[i]-48;

    n=strlen(s);
    for (i=n-1;i>=0;i--) b[n-1-i]=s[i]-48;
    n=Max(strlen(c),strlen(s))*2;
    for (len=1;len<=n;len<<=1);
    NTT(a,len); NTT(b,len);
    for (i=0;i<=len;i++) a[i]*=b[i];
    NTT(a,-1);
    for (i=0;i<=len || a[i];i++) a[i+1]+=a[i]/8,a[i]%8;
    for (i=len+10;i>=0;i--){
        if (a[i]){
            n=i; break;
        }
    }
    for (i=n;i>=0;i--) putchar(a[i]+48);
}

```

```
    putchar(10);

    return 0;
}
```

C

C 简单的计算几何

时间限制：1000ms 内存限制：65536kb

通过率：151/231 (65.37%) 正确率：151/498 (30.32%)

题目描述

这是一个简单的计算几何

在一个**正方形**中，给定角1，角2，求角3

```
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <stdlib.h>
#define ll long long
#define pi 3.1415926535
int main()
{
    int a, b;
    scanf("%d %d", &a, &b);
    double c = atan((1 - tan(b*pi/180)) / (1 - 1/tan(a*pi/180)))*180/pi + b;
    printf("%.6f\n", c);
    return 0;
}
```

D

D 生生不息

时间限制：1000ms 内存限制：65536kb

通过率：167/238 (70.17%) 正确率：167/486 (34.36%)

题目描述

XD树的树枝茂密，相互交错，一根树枝可以分散出许多小树枝。

树枝越粗壮，树枝的等级越高，树枝的序列也就越长。如果树枝A的序列 S_A 是 树枝B的序列 S_B 的子串，那么 B 是 A 的祖先。

现在给你树枝 A 的序列 S_A 和其它 m 个树枝的序列 $S_1, S_2, \dots, S_m$ ，请你判断每个树枝与树枝 A 的关系。

```
#include<iostream>
#include<cstring>
#include<cstdio>
#include<cmath>
```



```

#include<algorithm>

using namespace std;
char a[1000010];
char l[1000100];
int main()
{
    int n,i;
    scanf("%s",a);
    int la=strlen(a),ll;
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        scanf("%s",l);
        ll=strlen(l);
        if(ll>la)
        {
            if(strstr(l,a)!=NULL)
            {
                printf("my ancestor\n");
                continue;
            }
            else
            {
                printf("higher level\n");
                continue;
            }
        }
        if(ll==la)
        {
            if(strcmp(a,l)==0)
            {
                printf("just same\n");
                continue;
            }
            else
            {
                printf("same level\n");
                continue;
            }
        }
        if(ll<la)
        {
            if(strstr(a,l)!=NULL)
            {
                printf("my son\n");
                continue;
            }
            else
            {
                printf("lower level\n");
                continue;
            }
        }
    }
    return 0;
}

```

E

E 兑换饮料

时间限制：1000ms 内存限制：65536kb

通过率：81/202 (40.10%) 正确率：81/574 (14.11%)

题目描述

你手中有三种不同饮料的瓶盖，消耗这一定数量三种瓶盖可以兑换一瓶新的饮料（饮料种类自选，详见样例解释）。请问你最多能换多少次饮料。**不可以借瓶盖。**

输入

```
#include <stdio.h>

int main()
{
    int m, n, k, a, b, c;
    scanf("%d%d%d%d%d", &m, &n, &k, &a, &b, &c);
    long long ans = 0;
    while(m >= a && n >= b && k >= c){
        ans++;
        m -= a; n -= b; k -= c;
        double cm = 1.0*m/a, cn = 1.0*n/b, ck = 1.0*k/c;
        if(cm <= cn && cm <= ck) m++;
        else if(cn <= cm && cn <= ck) n++;
        else k++;
    }
    printf("%lld\n", ans);
    return 0;
}
```

F

F 买棋子凑战力

时间限制：1000ms 内存限制：65536kb

通过率：1/6 (16.67%) 正确率：1/17 (5.88%)

题目描述

有这样一种游戏，你可以通过购买棋子，来为自己提高战力。每种棋子有自己的价格和战力，且数量都是无限的。整个游戏分为两大阶段：准备阶段与出战阶段。

在准备阶段中，你的任务是购买一定数量的棋子。棋子具有星级，你购买的棋子均为一星。当你拥有三个**同种**一星的棋子时，他们会**自动合成**一个同种二星棋子。当你拥有三个**同种**二星的棋子时，他们会**自动合成**一个同种三星棋子。棋子星级最高为三级。不同星级的棋子具有不同的战力。在这个阶段中，你可以购买**任意数量**的棋子（只要有足够的钱）。

在出战阶段中，你可以选择不超过你阵容容量的棋子来出战。你的阵容的初始容量为0。如果你当前阵容容量为*i*，那么你需要花费*i* + 1元来将阵容容量提升到*i* + 1。也就是说，如果最后你想选择5个棋子出战，那么你升级容量的花费就是1+2+3+4+5=15元。把所有出战棋子的战力加和，就能得到你的战力。注意，你是不可能同时让三个同种类一星或者二星棋子出战的，因为他们会**自动合成**高星棋子。

现在你有一定数量的金币，那么你能凑出的最高战力是多少呢？

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
```

```

#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,j,k,x,y,a,b,c,v;
int f[110][5010];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __MarvoLo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    m=read();    n=read();
    for (i=1;i<=n;i++){
        v=read();    a=read();    b=read();    c=read();
        for (j=100;j;j--){
            for (k=m;k>=v;k--){
                f[j][k] = max(f[j][k],f[j-1][k-v] + a);
            }
        }
        for (j=100;j;j--){
            for (k=m;k>=v;k--){
                f[j][k] = max(f[j][k],f[j-1][k-v] + a);
            }
        }
        for (j=100;j;j--){
            for (k=m;k>=3*v;k--){
                f[j][k] = max(f[j][k],f[j-1][k-3*v] + b);
            }
        }
        for (j=100;j;j--){
            for (k=m;k>=3*v;k--){
                f[j][k] = max(f[j][k],f[j-1][k-3*v] + b);
            }
        }
        for (j=1;j<=100;j++){
            for (k=9*v;k<=m;k++){
                f[j][k] = max(f[j][k],f[j-1][k-9*v] + c);
            }
        }
        int ans = 0;
        for (i=1;i<=100;i++){
            for (j=0;j<=m;j++){
                x = i*(i+1)/2;
                if (x + j > m) break;
                ans = max(ans, f[i][j]);
            }
        }
        cout<<ans<<endl;
        return 0;
    }
}

```

G

G 制糖厂

时间限制: 1000ms 内存限制: 65536kb

通过率: 5/50 (10.00%) 正确率: 5/136 (3.68%)

题目描述

算法助教们想给 20 级的同学们发礼物, 他们决定发 n 颗糖。

助教们来到了制糖厂, 他们发现制糖厂 x 秒能制造一颗糖。

但是 x 秒太长了, 于是他们给机器施了魔法, 魔法分为两种:

A 类型魔法: 消耗 a_i 点魔力, 将 x 变为 c_i

B 类型魔法: 消耗 b_i 点魔力, 直接变出 d_i 颗糖

助教们的魔力值为 m , 且同种类型的魔法最多使用一个 (可以不使用), 求消耗时间的最小值。

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<ll, ll> pll;
const int N = 2e5 + 5;

// (cost,value)
pll a[N], b[N];

int main() {
    // read data
    ll n, x, m, ans = LONG_LONG_MAX;
    int cntA, cntB;
    scanf("%lld%d%d", &n, &cntA, &cntB);
    scanf("%lld%lld", &x, &m);
    for (int i = 0; i < cntA; ++i) scanf("%lld", &a[i].first);
    for (int i = 0; i < cntA; ++i) scanf("%lld", &a[i].second);
    for (int i = cntB - 1; i >= 0; --i) scanf("%lld", &b[i].first);
    for (int i = cntB - 1; i >= 0; --i) scanf("%lld", &b[i].second);

    // preprocess
    a[cntA] = make_pair(0, x);
    b[cntB] = make_pair(0, 0);
    for (int i = 0; i < cntB; ++i) b[i].first = -b[i].first;

    // traverse A and use B greedily
    for (int i = 0; i <= cntA; ++i) {
        if (a[i].first > m) {
            ans = min(ans, n * x);
        } else {
            ll m_left = m - a[i].first;
            int pos = lower_bound(b, b + cntB, make_pair(-m_left, 0LL)) - b;
```

```

        ans = min(ans, a[i].second * (n - b[pos].second));
    }
}

// print answer
printf("%lld", ans);
return 0;
}

```

H

H 普通的字符串

时间限制: 1000ms 内存限制: 65536kb

通过率: 2/20 (10.00%) 正确率: 2/47 (4.26%)

题目描述

Alex 有一个神奇的打字机，他只支持如下三种操作：

- 操作1: 把一个 **x** 变成 **aX**
- 操作2: 把一个 **x** 变成 **aXbX**
- 操作3: 删除任意一个 **x**

打字机启动时，屏幕上有且仅有一个 **x**

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 1000010
#define INF 0x3f3f3f3f
using namespace std;

int T,i,len,x,y,flag;
char c[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

```

```

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    scanf("%d",&T);
    while (T--){
        scanf("%s", &c);
        len = strlen(c);
        x = y = flag = 0;
        for (i=0; i<len; i++){
            if (c[i] == 'a')    x++;
            else y++;
            if (c[i] == 'b')
                if (x > y)    flag = 1;
                else flag = 0;
            if (x < y){
                flag = -1;
                break;
            }
        }
        if (flag == 0) puts("Happy Alex");
        else if (flag == 1) puts("Sad Alex");
        else puts("Dead Alex");
    }
    return 0;
}

```

I 模n方程求根

时间限制：1000ms 内存限制：65536kb

通过率：7/79 (8.86%) 正确率：7/172 (4.07%)

题目描述

给定正整数 a, b 和 n ，求解方程 $(a * x) \% n = b$

说明：* 表示乘法运算，% 表示求模运算，求解 x 。

输入格式

输入共一行，包含三个正整数 a, b, n （`int` 范围内）。

输出格式

若方程有解，从小到大输出所有的解，每个解占一行（保证小于10000行）。

若方程无解，输出 `NO`。

```
#include<bits/stdc++.h>
```

```

#define bug printf("!!!!\n");
using namespace std;
typedef unsigned long long LL;
typedef pair<LL, LL> PII;
inline LL read() {
    LL x = 0, f1 = 1; char c = getchar();
    while (c < '0' || c > '9') { if(c == '-') f1 = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * f1;
}
const int N = 1010, M = 100010;
LL inf = 1e18 + 7;
int a, b, n;
int exgcd(int a, int b, int &x, int &y) // 扩展欧几里得算法, 求x, y, 使得ax + by = gcd(a, b)
{
    if (!b) {
        x = 1; y = 0;
        return a;
    }
    int d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
}
int main() {
    a = read(), b = read(), n = read();
    int x = 0, y = 0;
    int d = exgcd(a, n, x, y);
    if(b % d == 0) {
        int x0 = (x * (b / d)) % n;
        int k = n / d;
        while (x0 > 0) x0 -= k;
        while(x0 <= 0) x0 += k;
        for (; x0 < n; x0 += k)
            printf("%d\n", x0);
    }
    else puts("NO");
    return 0;
}

```

J

J 接雨水 - 二维

时间限制: 1000ms 内存限制: 65536kb

通过率: 1/22 (4.55%) 正确率: 1/59 (1.69%)

题目描述

给定一个二维矩阵，表示立在一个平面上的柱子的高度。

柱子形成的空间凹区域可以留存雨水，求总共可以留存的雨水量。

```

#include <bits/stdc++.h>
using namespace std;

```

```

typedef pair<int, int> pii;

int trapRainWater(vector<vector<int>>& heightMap) {
    if (heightMap.size() <= 2 || heightMap[0].size() <= 2) {
        return 0;
    }
    int m = heightMap.size();
    int n = heightMap[0].size();
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    vector<vector<bool>> visit(m, vector<bool>(n, false));
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            if (i == 0 || i == m - 1 || j == 0 || j == n - 1) {
                pq.push({heightMap[i][j], i * n + j});
                visit[i][j] = true;
            }
        }
    }

    int res = 0;
    int dirs[] = {-1, 0, 1, 0, -1};
    while (!pq.empty()) {
        pii curr = pq.top();
        pq.pop();
        for (int k = 0; k < 4; ++k) {
            int nx = curr.second / n + dirs[k];
            int ny = curr.second % n + dirs[k + 1];
            if (nx >= 0 && nx < m && ny >= 0 && ny < n && !visit[nx][ny]) {
                if (heightMap[nx][ny] < curr.first) {
                    res += curr.first - heightMap[nx][ny];
                }
                visit[nx][ny] = true;
                pq.push({max(heightMap[nx][ny], curr.first), nx * n + ny});
            }
        }
    }

    return res;
}

vector<vector<int>> h;

int main() {
    int n, m, x;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; ++i) {
        vector<int> tmp;
        for (int j = 0; j < m; ++j) {
            scanf("%d", &x);
            tmp.emplace_back(x);
        }
        h.emplace_back(tmp);
    }
    printf("%d", trapRainWater(h));
    return 0;
}

```


C1

A

A 计数排序

时间限制: 1000ms 内存限制: 65536kb

通过率: / (%) 正确率: / (%)

题目描述

插入排序的一个伪代码如下:

```
insert_sort(A):
    for j = 1 to len(A):
        key = A[j]
        # insert A[j] to sorted sequence A[0...j-1]
        i = j - 1
        while i >= 0 and A[i] > key:
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
```

给定待排序的数组, 请输出排序过程中, 第 6 行和第 8 行的执行次数

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 1010
#define INF 0x3f3f3f3f
using namespace std;

int n,i,j,key,x,y;
int a[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
```

```

    freopen("zht.out","w",stdout);
    #endif
    while (~scanf("%d",&a[n++]));
    n--;
    for (j=1;j<n;j++){
        key = a[j];
        i=j-1;
        while ((++x && i>=0) && a[i]>key){
            a[i+1]=a[i];    i--;    y++;
        }
        a[i+1] = key;
    }
    cout<<x<<" "<<y<<endl;
    return 0;
}

```

B

B 秦九韶

时间限制：1000ms 内存限制：65536kb

通过率：246/250 (98.40%) 正确率：246/1286 (19.13%)

题目描述

秦九韶（约公元1202年 - 1261年），字道古，南宋末年人，出生于鲁郡（今山东曲阜一带人）。宋绍定四年（公元1231），秦九韶考中进士，先后担任县尉、通判、参议官、州守等职。先后在湖北、安徽、江苏、浙江等地做官。南宋理宗景定元年（公元1260年）出任梅州太守，翌年卒于梅州。

秦九韶算法是一种将一元 n 次多项式的求值问题转化为 n 个一次式的算法。其大大简化了计算过程，即使在现代，利用计算机解决多项式的求值问题时，秦九韶算法依然是最优的算法。

现在，给出一个 n 次多项式，请求出其在 $x = x_0$ 处的值。答案对998244353取模。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 1000010
#define INF 0x3f3f3f3f
#define MOD 998244353
using namespace std;

LL n,i,x,Ans;
LL a[N];

inline LL Mi(LL x,LL y){
    LL p=x,t=1,Res=1;
    for (;t<=y;(t&y)?Res=(Res*p)%MOD:0,p=(p*p)%MOD,t<=&1);
    return Res;
}

```

```

}

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL LL read(){
    LL p=0,f=1; char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    n=read();
    for (i=1;i<=n+1;i++)    a[i]=read();
    x=read();
    Ans = a[n+1];
    for (i=1;i<=n;i++)    Ans=(Ans+a[i]*Mi(x,n-i+1)%MOD)%MOD;
    cout<<Ans<<endl;
    return 0;
}

```

C

C 逆序对

时间限制: 1000ms 内存限制: 65536kb

通过率: 232/247 (93.93%) 正确率: 232/1087 (21.34%)

题目描述

给定一个整数序列，求逆序对的个数。

输入

第一行一个整数，表示序列长度 n

接下来一行， n 个整数，保证在 `int` 范围内

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

```

```

int n,i;
int a[N],b[N];
LL Ans;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL void Merge(int low,int high){
    reg int i=0,mid=0,q=low,w=0,e=low;
    if (low == high) return;
    mid=(low+high)>>1;
    Merge(low, mid);    Merge(mid+1,high);
    w=mid+1;
    while (q<=mid && w<=high){
        if (a[q]<=a[w])
            b[e++]=a[q++];
        else {
            Ans += mid-q+1;
            b[e++]=a[w++];
        }
    }
    while (q<=mid) b[e++]=a[q++];
    while (w<=high) b[e++]=a[w++];
    for (i=low;i<=high;i++) a[i]=b[i];
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();
    for (i=1;i<=n;i++) a[i]=read();
    Merge(1,n);
    cout<<Ans<<endl;
    return 0;
}

```

D Ashane算日期 (easy version)

时间限制: 1000ms 内存限制: 65536kb

通过率: 227/239 (94.98%) 正确率: 227/689 (32.95%)

题目描述

Ashane 喜欢用 8 位数字表示一个日期，其中前 4 位代表年，接下来 2 位代表月，最后 2 位代表日。

现在，他想知道指定两个日期之间（包含这两个日期本身），有多少个日期是幸运日期。

一个日期是幸运的当且仅当表示这个日期的 8 位数字是回文的。

```
#include<bits/stdc++.h>
using namespace std;
int days[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

bool check(int date)
{
    int year = date / 10000;
    int month = date % 10000 / 100;
    int day = date % 100;
    if(month == 0 || month > 12) return 0;
    if(day == 0 || month != 2 && day > days[month]) return 0;
    if(month == 2)
    {
        int leap = year % 400 == 0 || year % 100 && year % 4 == 0;
        if(day > 28 + leap) return 0;
    }
    return 1;
}

int main()
{
    int date1, date2;
    cin >> date1 >> date2;
    if(date1 > date2) swap(date1, date2);
    int ans = 0;
    for(int i = 1000; i < 10000; i++)
    {
        int date = i, x = i;
        for(int j = 0; j < 4; j++) date = date * 10 + x % 10, x /= 10;

        if(date >= date1 && date <= date2 && check(date)) ans++;
    }
    cout << ans << endl;
}
```

E

E 繁衍

时间限制: 1000ms 内存限制: 65536kb

通过率: 224/231 (96.97%) 正确率: 224/539 (41.56%)

题目描述

众所周知, 兔子是一种繁殖能力极强的生物。据说在很久很久以前的澳大利亚是没有兔子的, 直到有人带了一对兔子过去, 兔子的踪迹便在数年之内遍布了整个澳大利亚大陆。

现在请你模拟一下当前的情况。假设第1年有2只刚刚出生的幼年兔子, 它们会在出生 a 年后变得成熟。每两只成熟的兔子每年会产一只幼崽 (如果成年兔子的总数是奇数, 则最后一只兔子不能配对, 无法产仔)。每只兔子的成熟期只有 b 年, 在那之后它就会衰老, 就不能再产下幼崽。我们截取的时间比较短, 所以不考虑兔子死亡的情况。现在, 请你算出, 在第 x 年的时候, 一共有多少只兔子。

注意: 假如有2只成熟的兔子, 你可以认为他们一定可以配成一对并产下幼崽。这个题目中的模型比较理想化, 你不需要考虑关于兔子性别的任何问题。

```
// Author: Marvolo
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int a,b,c,T,i,j;
LL tot;
LL s[150],t[150];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    scanf("%d%d%d",&a,&b,&T);
    s[1]=t[1]=2;
    for (i=2;i<=T;i++){
        tot = 0;
        for (j=i-a-b+1;j<=i-a;j++){
            if (j>0)    tot+=s[j];
            s[i] += tot/2;
            for (j=1;j<=i;j++)    t[i]+=s[j];
        }
        cout<<t[T]<<endl;
        return 0;
    }
```

F 黑子的训练

时间限制: 1000ms 内存限制: 65536kb

通过率: 221/235 (94.04%) 正确率: 221/552 (40.04%)

题目描述

BUAA 校级篮球联赛正如火如荼地进行着, 教练在训练时对每个队员又提出了新的标准。

教练要求队员黑子在一次投篮训练中**得到 n 分**, 黑子可以选择每次投两分球 2 分或三分球 3 分, 但由于两分球比较容易投进, 教练要求黑子投射的两分球数量**不超过 max_two 个**。请输出黑子有多少种不同的投篮顺序, 使得**恰好**完成了教练要求的得分 (假设投的球都能命中)。

输入

一行, 两个以空格分隔的正整数 n 、 max_two ($2 \leq n \leq 70$, $2 \leq max_two \leq 8$), 含义如题目描述所述。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 110
#define INF 0x3f3f3f3f
using namespace std;

int n,k;
LL v[N][N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL LL Dfs(int x,int y){
    if (x<0) return 0;
    if (x == 0) return 1;
    if (v[x][y])    return v[x][y];
    LL reg sum = Dfs(x-3,y);
    if (y)    sum += Dfs(x-2,y-1);
    v[x][y] = sum;
    return sum;
}
```



```

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();    k=read();
    cout<<Dfs(n,k)<<endl;
    return 0;
}

```

G

G Matrix53的博弈

时间限制：500ms 内存限制：65536kb

通过率：124/152 (81.58%) 正确率：124/878 (14.12%)

题目描述

Matrix53 和 Alex 都很喜欢下围棋。一天，在和 Alex 下围棋时，Matrix53 突然想到，如果能知道自己当前的胜率该多好啊，这样就可以知道应该激进还是稳健行棋了。

Matrix53 把棋局抽象成了一棵树，树的每个结点代表一个局面，**树根代表初始局面**。Matrix53 或者 Alex 行棋，相当于选择了棋局所在结点的某个子结点。现在给定这棵树，以及棋局位于各个叶子结点时 Matrix53 的胜率，求棋局位于树根时 Matrix53 的胜率。

Matrix53 和 Alex 都会做出最优选择，且 Matrix53 先行棋。

```

// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;

const int INF = 10001;

vector<int> g[1000005];
int score[1000005];

int minNode(int, int);

int maxNode(int now, int limit) {
    if (score[now] != -1) return score[now];
    int ans = -INF;
    for (auto son : g[now]) {
        ans = max(ans, minNode(son, ans));
        if (ans >= limit) break;
    }
    score[now] = ans;
    return ans;
}

int minNode(int now, int limit) {
    if (score[now] != -1) return score[now];
    int ans = INF;
    for (auto son : g[now]) {
        ans = min(ans, maxNode(son, ans));
        if (ans <= limit) break;
    }
    score[now] = ans;
    return ans;
}

```

```

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 1; i < n; ++i) {
        int a, b;
        scanf("%d%d", &a, &b);
        g[a].emplace_back(b);
    }
    memset(score, 0xff, sizeof(score));
    for (int i = 1; i <= n; ++i) {
        if (g[i].empty()) scanf("%d", &score[i]);
    }
    printf("%d", maxNode(1, INF));
    return 0;
}

```

H

H 黑暗之魂

时间限制: 1000ms 内存限制: 65536kb

通过率: 241/241 (100.00%) 正确率: 241/407 (59.21%)

题目描述

有传说说，在特定的年份，黑暗将会降临世界。

```

Yes, indeed.
It is called Lothric,
where the transitory lands of Lords of Cinder converge.
In venturing north,
the Pilgrims discover the truth, the old words:
"The fire fades,
and the Lords go without thrones."
When the link of Fire is threatened,
the bell tolls,
unearthing the old Lords of Cinder from their graves.
Aldrich, saint of the deep;
Farron's Undead Legion, the Abyss Watchers;
and the reclusive lord of the Profaned Capital, Yhorm the Giant.
Only, in truth,
the Lords will abandon their thrones.
And the Unkindled will rise.

```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```

int main() {
    int T; scanf("%d", &T); while (T--) puts("No");
}

```

I

I 对抗

时间限制：1000ms 内存限制：65536kb

通过率：132/143 (92.31%) 正确率：132/322 (40.99%)

题目描述

注意，本题使用评测插件进行评测

为了让大家对算法更深的理解，助教们决定以分组对抗的形式举办一场比赛。一共有 n 名同学。每个同学都有一个编号——从1到 n 的不同的整数。

比赛分多轮，每轮全员参加比赛。同学们被分成两个组，每组可能有不同数量的同学，但每个组必须至少有一名同学。位于不同组的同学之间存在对抗关系。

为了让大家在比赛中被充分调动，助教们希望每对同学都有过对抗关系。但是由于每轮比赛的安排耗费大量精力，助教们希望轮数越少越好。

请你帮助助教们安排分组方案。

```
#include<stdio.h>
int n,ans;
int team[103][1003],cnt[1003];
void div(int l,int r,int round)
{
    if(l==r)
    {
        if(ans<round)
            ans=round;
        return;
    }
    int mid =(l+r)/2;
    for(int i=l;i<=mid;i++)
        team[round][cnt[round]++]=i;
    div(l,mid,round+1);
    div(mid+1,r,round+1);
}
int main()
{
    scanf("%d",&n);
    div(1,n,0);
    printf("%d\n",ans);
    for(int i=0;i<ans;i++)
    {
        printf("%d ",cnt[i]);
        for(int j=0;j<cnt[i];j++)
            printf("%d ",team[i][j]);
        printf("\n");
    }
}
```

J

J 查找

时间限制：1000ms 内存限制：65536kb

通过率：166/206 (80.58%) 正确率：166/1302 (12.75%)

题目描述

查找输入数据在data数组中的第一次出现的位置。

输入

第一行，一个正整数n ($n \leq 1000000$) ；

第二行，n个int型整数x（可能有重复的数），每一个元素xi记为数组data的数组元素；

第三行，多个int型整数（可能有重复的数，但总数不超过1000000个），每一个数记为t。输入的数之间用空白符号（空格、换行、或制表符）分割。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#include<unordered_map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int n,i,x;
unordered_map <int,int> M;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();
    for (i=1;i<=n;i++){
        x=read();
        if (!M.count(x))    M[x]=i;
    }
    while (~scanf("%d",&x)){
        if (M.count(x)) printf("%d\n",M[x]);
        else puts("NO");
    }
```

```
}  
    return 0;  
}
```

C2

A

A 这一刻我已等待许久

时间限制：1000ms 内存限制：65536kb

通过率：/ (%) 正确率：/ (%)

题目描述

众所周知，所有电子设备在霍格沃兹都会失去作用。因此 Alex 只要打开这封信，他就不用出题了。

他在信上了解到，他需要购买一件巫师袍，一只猫头鹰和一根魔杖。

Alex 走进奥利凡德魔杖店，被告知“魔杖选择巫师”

人生而不同，魔杖会根据一个人的勇气，忠诚，智慧，野心的高低来选择合适的巫师

奥利凡德很轻松的就找到了 Alex 的四项属性值 \$a, b, c, d\$。

将魔杖标号，每一根魔杖对主人的要求也可以概括为这四项属性 \$A_i, B_i, C_i, D_i\$

Alex 希望自己能够得到与自己契合度最高的魔杖，也就是使得 \$|a-A_i|+|b-B_i|+|c-C_i|+|d-D_i|\$ 最小。他应该买第几根魔杖呢？

对了，标号越大的魔杖价格也越高，因此契合度相同时，Alex 希望优先选购便宜的那个。

```
#include <bits/stdc++.h>

using namespace std;

const int N = 1010;

int n, a, b, c, d;
int A[N], B[N], C[N], D[N];

int calc(int pos) { return abs(a - A[pos]) + abs(b - B[pos]) + abs(c - C[pos]) + abs(d - D[pos]); }

int main() {
    cin >> n >> a >> b >> c >> d;
    for (int i = 1; i <= n; ++i) cin >> A[i];
    for (int i = 1; i <= n; ++i) cin >> B[i];
    for (int i = 1; i <= n; ++i) cin >> C[i];
    for (int i = 1; i <= n; ++i) cin >> D[i];
    int ans = -1, delta = 0x3f3f3f3f;
    for (int i = 1; i <= n; ++i)
        if (delta > calc(i))
            delta = calc(i), ans = i;
    cout << ans << endl;
    return 0;
}
```

B

B 卡特兰数

时间限制：1000ms 内存限制：65536kb

通过率：244/250 (97.60%) 正确率：244/444 (54.95%)

题目描述

二叉树是一种重要的数据结构。

有1个节点的二叉树有1种，有2个节点二叉树有2种，有3个节点的二叉树有5种，有n个节点的二叉树记为 $h(n)$ 种， $h(n)$ 又被称为卡特兰数。

现在，给出多组数据，每一组数据会给出一个正整数 n 。对于每一组数据，请输出 $h(n)$ 的值。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int n,i;
LL h[110];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    h[0]=1; h[1]=1;
    for (i=2;i<=25;i++)
        h[i]=(4*(LL)i-2)*h[i-1]/(i+1);
    int T = read();
    while (T--){
        int n = read();
        cout<<h[n]<<endl;
    }
    return 0;
}
```

C 公交排序

时间限制：1000ms 内存限制：65536kb

通过率：184/230 (80.00%) 正确率：184/557 (33.03%)

题目描述

Marvolo 在国庆的时候出去玩了一天，但不幸的是他选择的交通工具是公交车：因为假期出游的原因，车上人满为患，连移动到车门口下车都十分困难。

Marvolo 想：大家在一站下车一开始是无序的，可不可以先在公交车上调整成有序的（也就是单调不降的），这样下车的时候就不用着急往前挤了？如果要调整的话，最少要调整几次呢？因为车上人很多，规定一次调整只能交换相邻两个人的顺序。

为了简化问题，你可以认为公交车上一共有 n 个人，第 i 个人要在第 a_i 个站下车。最开头那个人站的位置是车门位置。

举个例子，假如有 3 个人，这三个人分别要在第 3,1,2 站下车。先让前两个人交换顺序，变成 1,3,2，再让后两个人交换顺序，变成 1,2,3，就可以调整成有序的，此时调整次数最少。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 200010
#define INF 0x3f3f3f3f
using namespace std;

int n,i;
LL Ans;
int a[N],b[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL void Merge(int low,int high){
    reg int i=0,mid=0,q=low,w=0,e=low;
    if (low == high) return;
    mid=(low+high)>>1;
    Merge(low, mid);    Merge(mid+1,high);
    w=mid+1;
    while (q<=mid && w<=high)
        if (a[q]<=a[w])
            b[e++]=a[q++];
        else {
            Ans += mid-q+1;
            b[e++]=a[w++];
        }
}
```



```

    }
    while (q<=mid) b[e++]=a[q++];
    while (w<=high) b[e++]=a[w++];
    for (i=low;i<=high;i++) a[i]=b[i];
}

int main(){
    n=read();
    for (i=1;i<=n;i++) a[i]=read();
    Merge(1,n);
    cout<<Ans<<endl;
    return 0;
}

```

D

D 搬题

时间限制：1000ms 内存限制：65536kb

通过率：219/237 (92.41%) 正确率：219/419 (52.27%)

题目描述

Marvolo不会出题了，于是他决定去搬题。但是他有个奇怪的癖好，在第 k 天时，他必须要搬来 k 道题，否则他当场罢工。

Marvolo现在找到了一个题库，一共有 n 场比赛，每一场比赛中有一定数量的题目。但是他十分挑剔，每场比赛他只会搬一天，每一天也只能搬一场比赛里的题目，之后那场比赛就会被弃之不用。对于每场比赛，他不需要搬来所有的题。

那么问题来了，Marvolo最多会工作几天才罢工呢？

```

#include<bits/stdc++.h>
using namespace std;
int n;
int a[1000006];
int main()
{
    for(int i=0;i<10;i++)
    {
        char s[10]="0a.out";
        char ss[10]="0a.in";
        s[1]='0'+i;
        ss[1]='0'+i;
        FILE *out=fopen(s,"w");
        FILE *in=fopen(ss,"r");
        fscanf(in,"%d",&n);
        for(int j=1;j<=n;j++)
        {
            fscanf(in,"%d",&a[j]);
        }
        sort(a+1,a+n+1);
        int t=1;
        for(int j=1;j<=n;j++)
        {
            if(a[j]>=t)
                t++;
        }
        fprintf(out,"%d",t-1);
    }
}

```

```
    return 0;
}
```

E

E 最大子数组

时间限制：1000ms 内存限制：65536kb

通过率：153/192 (79.69%) 正确率：153/455 (33.63%)

题目描述

定义一个数组的和最大的连续非空子数组为该数组的最大子数组。

给你一个数组，求其最大子数组的和。

```
#include<bits/stdc++.h>
using namespace std;
int n;
long long ans,c,l;
int main()
{
    ans=-1000000000;
    scanf("%d",&n);
    while(n-->0)
    {
        scanf("%lld",&c);
        if(l>0)
            c+=l;
        l=c;
        ans=max(ans,l);
    }
    printf("%lld",ans);
}
```

F

F 二分查找++

时间限制：1000ms 内存限制：65536kb

通过率：109/170 (64.12%) 正确率：109/410 (26.59%)

题目描述

这道题考察二分查找的各种变形，禁用C++。

给一个已经排好序的长度为 n 的数组 a ，第 i 个元素是 $a_{(i)}$ 。

接下来有 m 条询问，每个询问的格式为 $\text{op} \ x$ 。如果 $\text{op}=1$ ，表示问 x 第一次出现的位置是什么；如果 $\text{op}=2$ ，表示询问 x 最后一次出现的位置是什么。如果 x 从未出现过，输出 `Not Found`。

```
#include<stdio.h>
#include<string.h>
#define N 100010
#define INF 0x3f3f3f3f
```

```

int n,m,i,op,x;
int a[N];

int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

void Find1(int x){
    int low=1,high=n,ans=0,mid=0;
    while (low<=high){
        mid = (low+high)>>1;
        if (a[mid] >= x)    ans=mid,high=mid-1;
        else low=mid+1;
    }
    if (a[ans] == x) printf("%d\n",ans);
    else puts("Not Found");
}

void Find2(int x){
    int low=1,high=n,ans=0,mid=0;
    while (low<=high){
        mid = (low+high)>>1;
        if (a[mid] <= x)    ans=mid,low=mid+1;
        else high=mid-1;
    }
    if (a[ans] == x) printf("%d\n",ans);
    else puts("Not Found");
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();    m=read();
    for (i=1;i<=n;i++)    a[i]=read();
    a[0]=-1;
    while (m--){
        op=read();    x=read();
        (op==1)?Find1(x):Find2(x);
    }
    return 0;
}

```

G Alex 和 Marvolo 不会出题了 - 3

时间限制：1000ms 内存限制：65536kb

通过率：40/64 (62.50%) 正确率：40/133 (30.08%)

题目描述

Alex 和 Marvolo 不会出题了。于是他们开始闲聊：

Alex: 感觉他们现在的警觉性很高。

Marvolo: 同感，上一道题的 hint 已经成为表情包了。

Alex: 我决定金盆洗手，改邪归正。出一点**有难度有挑战性的**算法题

Marvolo: 哦吼？

Alex: 你平时看电视嘛？

Marvolo: 不怎么看，我都玩手机的。

Alex: 对，电视上面的节目都是限时的，你过了这个点就看不到了。

Marvolo: 手机上看不就行了。

Alex: no no no，一般来说电视上是最早播出来的，电视放完了手机上才更新。

Marvolo: 所以呢？

```
// Author Alex
#include <bits/stdc++.h>

using namespace std;
const int N = 1000100;

int n;
pair<int, int> q[N];

int main() {
    cin >> n;
    for (int i = 1; i <= n; ++i)
        cin >> q[i].second >> q[i].first;
    sort(q + 1, q + 1 + n);
    int ans = 0, last = -1;
    for (int i = 1; i <= n; ++i)
        if (q[i].second >= last)
            ans++, last = q[i].first;
    cout << ans << endl;
    return 0;
}
```

H

H 浇灌绿藤

时间限制: 1000ms 内存限制: 65536kb

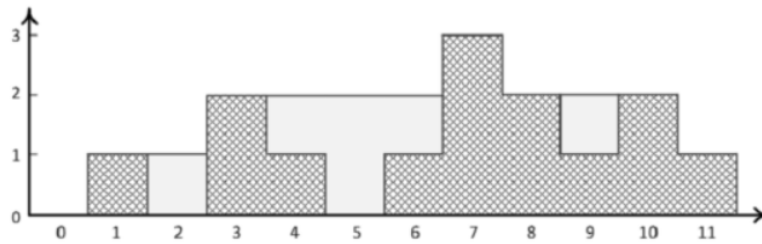
通过率: 20/34 (58.82%) 正确率: 20/81 (24.69%)

题目描述

打开了神秘的盒子之后, 你看到里面放着一株绿藤, 绿藤即将枯萎, 需要神奇的雨水浇灌。

输入 n 个非负整数, 每个整数表示一个宽度为 1 的方形柱子的高度, 计算按此排列的 n 个柱子的高度图, 下雨之后能接多少雨水?

样例: 假设输入的 n 个数为 $[0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]$, 代表如下图所示的柱子高度图, 该图能接的总雨水量为 6。



PicGo

```
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
typedef pair<LL, LL> PII;
inline LL read() {
    LL x = 0, fl = 1; char c = getchar();
    while (c < '0' || c > '9') { if(c == '-') fl = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * fl;
}
const int N = 1000010, M = 100010, mod = 1e9 + 7;
LL n, m, k, top, t;
int a[N], b[N], c[N];
int main()
{
    n = read();
    for (int i = 1; i <= n; i++) a[i] = read();
    for (int i = 1; i <= n; i++) b[i] = max(b[i - 1], a[i]);
    for (int i = n; i >= 1; i--) c[i] = max(c[i + 1], a[i]);
    LL ans = 0;
    for (int i = 1; i <= n; i++)
        ans += min(b[i], c[i]) - a[i];
    printf("%lld", ans);
    return 0;
}
```

I 最近点对

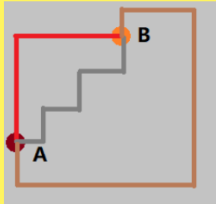
时间限制: 1000ms 内存限制: 65536kb

通过率: 5/14 (35.71%) 正确率: 5/29 (17.24%)

题目描述

某行星上有一种生物, 这种生物有一个特点, 它们只能够沿着平行于x轴和平行于y轴两个方向行走, 现在已知该行星上该物种的种群数量为n, 且知道它们每一个个体的位置, 求它们之间的最小距离。

如下图,



假设x轴为水平方向, y轴为竖直方向。红色、灰色和土黄色都是生物A能够行走走到B的路线, 当然, 这只是其中的一部分, A有无数种方法走到B, 只要沿着平行于x轴或是平行于y轴即可。

```
#include<cstdio>
#include<iostream>
#include<cmath>
#include<algorithm>
#include<cstring>
#include<cstdlib>
#include<vector>
#include<queue>
#include<map>
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int n,i;

struct Data{
    double x,y;
}a[100010];

int b[100010];

IL double Abs(double x){return (x<0)?-x:x;}
IL double Min(double a,double b){return (a<b)?a:b;}

IL int read(){
    register int p=0,f=1;   char c=getchar();
    while (c<'0' || c>'9'){if (c=='-')   f=-1;   c=getchar();}
    while (c>='0' && c<='9'){p=p*10+c-48;   c=getchar();}
    return p*f;
}

IL double Dis(int x,int y){return Abs(a[x].x-a[y].x)+Abs(a[x].y-a[y].y);}

IL bool cmp(Data a,Data b){
    return (a.x==b.x)?a.y<b.y:a.x<b.x;
}

IL bool cmp2(int x,int y){
```

```

        return a[x].y<a[y].y;
    }

    IL double solve(int low,int high){
        int i=0,j=0,mid=(low+high)>>1,cnt=0;
        double Ans=9999999999.99,d1=0,d2=0;
        if (low>=high) return Ans;
        if (high==low+1) return Dis(low,high);
        if (high-low+1<=4){
            for (i=low;i<=high;i++)
                for (j=i+1;j<=high;j++)
                    Ans=Min(Ans,Dis(i,j));
            return Ans;
        }
        d1=solve(low,mid); d2=solve(mid+1,high);
        Ans=Min(d1,d2);
        for (i=low;i<=high;i++)
            if (Abs(a[i].x-a[mid].x) <= Ans) b[++cnt]=i;
        sort(b+1,b+1+cnt,cmp2);
        for (i=1;i<=cnt;i++)
            for (j=i+1; j<=cnt && a[b[j]].y-a[b[i]].y <Ans;j++)
                Ans=Min(Ans,Dis(b[i],b[j]));
        return Ans;
    }

    int main(){
        n=read();
        n=read();
        for (i=1;i<=n;i++) scanf("%lf%lf",&a[i].x,&a[i].y);
        if (n==1){
            puts("0.000");
            return 0;
        }
        if (n==2){
            printf("%.3lf\n",Dis(1,2));
            return 0;
        }
        sort(a+1,a+1+n,cmp);
        printf("%.3lf\n",solve(1,n));
        return 0;
    }

```

J

J Alex 和 Marvolo 不会出题了 - 1 - 番外

时间限制: 1000ms 内存限制: 65536kb

通过率: 2/6 (33.33%) 正确率: 2/13 (15.38%)

题目描述

上回说到, Alex 和 Marvolo 取出了一堆黑球白球, 玩了一个取球的小游戏。结果 Marvolo 被套路了。

但是这个球毕竟是填上掉下来的。只玩一次有点可惜。

于是 Alex 又拉来了 Marvolo。并精心设计了一个新游戏。

初始时, n 个白球 m 个黑球摆成一排

Alex 会在这 $n + m$ 个球上写下一个 $n + m$ 的排列。

游戏开始:

- Marvolo 每次可以选择一个白球和一个黑球, 并交换他们的位置
- Marvolo 不能选择两个同色的球进行操作。
- Alex 希望将这 $n + m$ 个球按从小到大的顺序排序, 而 Marvolo 需要给出他的解法。
- Alex 得到了 Matrix53 的帮助, 已经知道了最优解。因此 Marvolo 必须也给出最优解 (即操作次数最少) 才能赢得游戏。

对了, Matrix53 发现 Alex 胡乱写上去的排列不一定有解, 所以顺手修了一下。

你可以认为输入的数据, 在上述操作下一定可以将所有球按数字大小排序。

```
#include <bits/stdc++.h>

using namespace std;

const int N = 2e5 + 100;

int n, m, L, A[N], fa[N], cntL[N], cntR[N];
inline int find(int x) {return fa[x] == x ? x : fa[x] = find(fa[x]);}

int main() {
    cin >> n >> m; L = n + m;
    for (int i = 1; i <= L; ++i) {
        cin >> A[i]; fa[i] = i;
        cntL[i] = (i <= n);
        cntR[i] = (i > n);
    }
    for (int i = 1; i <= L; ++i) {
        int u = find(i), v = find(A[i]);
        if (u != v) {
            fa[u] = v;
            cntL[v] += cntL[u];
            cntR[v] += cntR[u];
        }
    }
    int cnt = 0, X = 0, Y = 0;
    for (int i = 1; i <= L; ++i) if (fa[i] == i) {
        cnt++; if (cntL[i] + cntR[i] == 1) continue;
        X += (cntL[i] == 0);
        Y += (cntR[i] == 0);
    }
    cout << L - cnt + 2 * max(X, Y) << endl;
    return 0;
}
```


C3

A

A Mat 的签

时间限制：1000ms 内存限制：65536kb

通过率：176/248 (70.97%) 正确率：176/1269 (13.87%)

题目描述

Mat 去旅游了，于断壁残垣处偶遇风水先生 **Alex**，先生有一筒竹签。

Alex: 你我二人于此处偶遇，实属不易，不如我给你算一卦？

Mat: 有劳了。

Alex: 抽签吧。

Alex 的竹筒里有 n 根竹签，每根竹签有一个运气值 v ，一个重量值 w 。

由于 **Alex** 有仙术，所以一根竹签可能很重。

Mat 也不是凡人，**Mat** 能看透所有竹签的运气值和重量值。

Mat 的最大承重为 k ，他能够在承重范围内抽出任意多的竹签。

Mat 所获得的运气值为抽取的竹签的运气值之和，**Mat** 所抽取的竹签的总重量不能超过他的承受能力。

求 **Mat** 能够获得的最大运气值。

```
// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;

long long dp[100005];

int main() {
    int n, k;
    scanf("%d%d", &n, &k);
    while (n--) {
        int v, w;
        scanf("%d%d", &v, &w);
        for (int i = k; i >= w; --i) {
            dp[i] = max(dp[i], dp[i - w] + v);
        }
    }
    printf("%lld", dp[k]);
    return 0;
}
```

B

B Rix 的刀

时间限制：1000ms 内存限制：65536kb

通过率：212/244 (86.89%) 正确率：212/562 (37.72%)

题目描述

Mat 获得了足够的运气值，成了隐士高人。

Rix 是江湖人物，善刀法。

Rix 深知 **Mat** 武艺高强，想去讨教一番。

工欲善其事，必先利其器。

于是 **Rix** 寻来一块巨大的九天陨铁，欲以其铸刀。

陨铁可以被分为任意多块，分割之后，每块陨铁都可以铸刀。

每把刀具具有一定的攻击力，相同体积的陨铁，铸成的刀攻击力相同。

作为武器大师，**Rix** 能够携带任意多的刀，**Rix** 的攻击力为他携带的刀的攻击力之和。

 PicGo

```
// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;

long long k[10005], dp[10005];

int main() {
    int v;
    scanf("%d", &v);
    for (int i = 1; i <= v; ++i) scanf("%d", &k[i]);
    for (int i = 1; i <= v; ++i) {
        for (int j = 1; j <= i; ++j) {
            dp[i] = max(dp[i], dp[i - j] + k[j]);
        }
    }
    printf("%lld", dp[v]);
    return 0;
}
```

C

C 校庆外场

时间限制：1000ms 内存限制：65536kb

通过率：13/20 (65.00%) 正确率：13/44 (29.55%)

题面出完后才接到通知外场取消了，大家当平行时空做就行了.....

题目描述

又到了一年一度校庆的时候。今年的校庆外场格外热闹，大大小小的摊位摆满了一整条路。每个专业都有两个外场摊位，分别位于道路的两端。每个摊位都会有一个小游戏，玩完游戏后可以获得一个印章，集齐**所有的印章**可以兑换一个校庆纪念大奖。

Marvolo 和他的室友自然不会错过这个获奖的机会。他们研究了一下规则后发现，一共有 n 个专业参加了活动，也就是有 $2n$ 个外场摊位。每个专业的两个摊位各持有印章的半个部分，路北的摊位有印章的左半部分，路南的有右半部分。**印章的获取顺序必须是先获得左半部分印章，再获得右半部分印章**。Marvolo 和室友商量好，他们一个人只逛路北的摊位，一个人只逛路南的摊位。一个人每次只能在一个摊位玩游戏。通过调查，他提前知道了玩完每个摊位的小游戏需要多少时间，现在只需要规划好一个逛摊位的方案，使得兑换大奖的时间最短（去的晚可能没了）。

因为他们还要忙着去参加摊位的小游戏，所以找到了擅长编程的你，希望你帮忙解决这个问题。

```
#include<iostream>
```

```

#include<cstdio>
#include<algorithm>
#include<cmath>
using namespace std;
int n,ans;
struct Node{
    int a,b,c;
}x[1500];
bool cmp(Node x,Node y){
    return min(y.b,x.a)<min(x.b,y.a);//这就是Johnson表达式
}
int main(){
    scanf("%d",&n);
    for(int i=1;i<=n;i++) scanf("%d",&x[i].a);
    for(int i=1;i<=n;i++) {scanf("%d",&x[i].b);x[i].c=i;};
    sort(x+1,x+n+1,cmp);
    int ta=0,tb=0;
    for(int i=1;i<=n;i++){
        ta+=x[i].a;
        tb=max(ta,tb)+x[i].b;
    }
    printf("%d\n",tb);
    return 0;
}

```

D

D 生命之水

时间限制：1000ms 内存限制：65536kb

通过率：162/200 (81.00%) 正确率：162/493 (32.86%)

题目描述

经过你辛勤的浇灌和修剪，你的绿藤长势极好，甚至分出了 n 条绿枝，但最近绿藤的生长停滞了，似乎遇到了瓶颈。

你偶然间得到了生命之水，传说生命之水可以滋养万物，提供生命之力，于是你想到了你心爱的绿藤。

每当你使用生命之水滋养一条绿枝时，绿藤可以获得 x_i 的成长增益值，但是你不能同时滋养相邻的两条绿枝，这会触发生命之水的副作用，使绿藤失去生机。

现在，你想知道在不触发副作用的情况下绿藤可以获得的成长增益值最大是多少？

```

#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N = 100010;

LL f[N][2], a[N];

int main(){
    int n, t;
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%lld", &a[i]);
        for(int i = 1; i <= n; i++){
            f[i][0] = max(f[i - 1][0], f[i - 1][1]);
            f[i][1] = f[i - 1][0] + a[i];
        }
        printf("%lld\n", max(f[n][0], f[n][1]));
    }
}

```

```
}  
}
```

E

E 最大值的最小值

时间限制：1000ms 内存限制：65536kb

通过率：46/75 (61.33%) 正确率：46/174 (26.44%)

题目描述

给定一个长度为 n 的正整数数组，把它分成连续的 k 段。

对于每一段，将段内的数值加和作为该段的值。对于一种分段方案，所有段的值的最大值为该方案的值。

求在所有的分段方案中，方案的值最小可能为多少。

```
#include<bits/stdc++.h>  
#define N 100005  
#define ll long long  
#define inf 0x3f3f3f3f  
using namespace std;  
int n,k,a[N];  
bool check(int s)  
{  
    int c=0,cnt=0;  
    for(int i=1; i<=n; i++)  
    {  
        if(a[i]>s)  
            return false;  
        if(c+a[i]>s)  
        {  
            c=a[i];  
            cnt++;  
        }  
        else  
            c+=a[i];  
    }  
    if(c>0)  
        cnt++;  
    return cnt<=k;  
}  
int main()  
{  
    scanf("%d%d",&n,&k);  
    for(int i=1; i<=n; i++)  
        scanf("%d",a+i);  
    int l=0,r=100000000;  
    while(l+1<r)  
    {  
        int m=(l+r)/2;  
        if(check(m))  
            r=m;  
        else  
            l=m;  
    }  
    printf("%d",r);  
}
```

```
    return 0;
}
```

F

F DP一般看规律

时间限制: 1000ms 内存限制: 65536kb

通过率: 85/113 (75.22%) 正确率: 85/222 (38.29%)

题目描述

给定一个整数数列 A_i , 你需要从中挑出一个最长的子序列, 使得该子序列中任意两个相邻的元素不互质。

你只需要输出这个子序列的长度就可以了。

```
// Author: Alex
#include <bits/stdc++.h>

using namespace std;

const int N = 1010;

int n, ans, val[N], dp[N];

int main() {
    cin >> n;
    for (int i = 1; i <= n; ++i)
        cin >> val[i];
    for (int i = 1; i <= n; ++i) {
        dp[i] = 1;
        for (int j = 1; j < i; ++j)
            if (__gcd(val[i], val[j]) > 1)
                dp[i] = max(dp[i], dp[j] + 1);
        ans = max(ans, dp[i]);
    }
    cout << ans << endl;
    return 0;
}
```

G

G 硬币游戏

时间限制：1000ms 内存限制：65536kb

通过率：31/41 (75.61%) 正确率：31/68 (45.59%)

题目描述

Marvolo 和 Alex 在操场上玩一个硬币游戏。游戏的场地是一个画好的矩形，矩形被切分为了很多个小方格，共有2行 n 列。每一个方格内有一定数量的硬币。

现在，两个人都站在 $(1, 1)$ 的位置，且目的地都是 $(2, n)$ 。移动的规则是，一次只能向下或者向右移动一格（注意，坐标轴原点在左上角，向右是x轴正向，向下是y轴正方向。具体参见样例解释）。

一开始，Marvolo先移动，直到他走到 $(2, n)$ 为止。他会拿走他走过的方格上的所有硬币，包含 $(1, 1)$ 和 $(2, n)$ 。

接下来，Alex移动，他也会拿走他走过的方格上的所有硬币。

Marvolo 为了取得游戏的胜利，想让 Alex 能拿到的硬币数量尽可能的少（他自己拿多少硬币无所谓），而 Alex 则是希望让自己拿到的硬币数尽可能的多。

请你帮他们算一下，如果 Marvolo 和 Alex 都采取最优策略，Alex 最后能拿到多少个硬币？

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

int n,i,j,T,Ans;
int a[2][N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    T=1;
    while (T--){
        n=read();
        for (i=1;i<=n;i++)    a[0][i] = read() + a[0][i-1];
        for (i=1;i<=n;i++)    a[1][i] = read() + a[1][i-1];
        a[0][n+1] = a[1][n+1] = 0;
        if (n==1){
            puts("0");
            continue;
        }
    }
}
```

```

    Ans = INF;
    for (i=1;i<=n;i++)
        Ans = Min(Ans, Max(a[0][n] - a[0][i], a[1][\i-1]));
    printf("%d\n",Ans);
}
return 0;
}

```

H

H 公路旅行

时间限制：1000ms 内存限制：65536kb

通过率：3/6 (50.00%) 正确率：3/19 (15.79%)

题目描述

你打算驾驶一辆汽车在环形公路上旅行。

公路上总共有 n 个景点，每个景点可以加 x_i 升汽油，每升油可以让汽车行驶 1 千米。

你必须从某个景点出发，一直按顺时针（或逆时针）方向走遍所有的景点，并回到起点。

在一开始的时候，汽车内油量为零，你每到达一个景点就可以加 x_i 升汽油（起点站亦是如此），行驶过程中不能出现没有油的情况。

现在请你判断以每个景点为起点能否按条件成功旅游一周。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 1000010
#define INF 0x3f3f3f3f
using namespace std;

int n,i;
bool ans[N],Ans[N];
LL res;
LL a[N],b[N];

struct Data{
    LL x;
    int id;
};

priority_queue <Data> Q;

bool operator <(const Data a, const Data b){
    return a.x>b.x;
}

```



```

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL LL read(){
    LL p=0,f=1; char    c=getchar();
    while (c<48||c>57)  {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();
    for (i=1;i<=n;i++){
        a[i]=read();    b[i]=read();
    }
    for (i=1;i<=n;i++){
        Q.push((Data){res, i});
        res += b[i]-a[i];
        while (!Q.empty() && res > Q.top().x){
            ans[Q.top().id] = 1;    Q.pop();
        }
    }
    for (i=1;i<=n;i++){
        res += b[i]-a[i];
        while (!Q.empty() && res > Q.top().x){
            if (Q.top().id > i) ans[Q.top().id] = 1;
            Q.pop();
        }
    }

    while (!Q.empty())    Q.pop();
    res = 0;
    b[0] = b[n];
    for (i=n;i;i--){
        Q.push((Data){res, i});
        res += b[i-1]-a[i];
        while (!Q.empty() && res > Q.top().x){
            Ans[Q.top().id] = 1;
            Q.pop();
        }
    }
    for (i=n;i;i--){
        res += b[i-1]-a[i];
        while (!Q.empty() && res > Q.top().x){
            if (Q.top().id < i) Ans[Q.top().id] = 1;
            Q.pop();
        }
    }

    for (i=1;i<=n;i++)
        puts((ans[i]&Ans[i])?"N0":"YE5");
}

```

```
    return 0;
}
```

I 划分多边形

时间限制: 1000ms 内存限制: 65536kb

通过率: 5/13 (38.46%) 正确率: 5/38 (13.16%)

题目描述

赛后加了一组数据，可能会有部分同学 $1 \rightarrow 0.8$ ，目的是为了同学们真正掌握这道题，上机成绩的统计仍然为比赛结束时的结果

Marvolo 太坏了，他取得游戏胜利的方式居然是想办法让 Alex 拿到的最少。

详情见 -> 硬币游戏

Alex 一眼看穿了 Marvolo 的计谋，因此他决定换一个游戏：

- 给定一个正 n 边形，每个点有点权 val_i 。
- Marvolo 需要将在这个正 n 变形内部加 $n - 3$ 条边，将它拆分成 $n - 2$ 个三角形
- 对于每一个三角形，Alex 可以获得 $val_A \times val_B \times val_C$ 的分数

Marvolo 依然想让 Alex 的得分最少

那么，Alex 得分的最小值是多少呢？

```
// Author: Alex
#include <bits/stdc++.h>

using namespace std;

const int N = 666;

long long val[N], dp[N][N], n, ans = 0x3f3f3f3f3f3f3f;

int main () {
    cin >> n;
    for (int i = 1; i <= n; ++i)
        cin >> val[i], val[i + n] = val[i];
    memset(dp, 0x2f, sizeof dp);
    for (int i = 1; i <= n + n - 2; ++i)
        dp[i][i + 3 - 1] = val[i] * val[i + 1] * val[i + 2];
    for (int i = 1; i <= n + n; ++i)
        for (int j = i; j < i + 2; ++j)
            dp[i][j] = 0;
    for (int L = 4; L <= n; ++L) {
        for (int i = 1; i <= n + n - L + 1; ++i) {
            int j = i + L - 1;
            for (int k = i + 1; k < j; ++k) {
                dp[i][j] = min(dp[i][j], dp[i][k] + dp[k][j] + val[i] * val[k] *
val[j]);
            }
            if (L == n) ans = min(ans, dp[i][i + n - 1]);
        }
    }
    cout << ans << endl;
    return 0;
}
```

防线突破

时间限制: 1000ms 内存限制: 65536kb

通过率: 1/1 (100.00%) 正确率: 1/4 (25.00%)

题目描述

众所周知, Alex 写作业的速度非常快。这天, Marvolo找到了已经写完所有作业的Alex, 打算拉着他玩一下自己刚刚魔改的一版植物大战僵尸。

在这个版本里, 所有的植物种在一条直线上, 编号从1到 n , 第 i 个植物的攻击力是 a_i 。Alex需要操纵一个血量非常大的僵尸依次破坏这些植物。当僵尸在破坏第 i 个植物的时候, 它会受到 $i-1, i, i+1$ 这三个植物的攻击, 累计承受 $a_{i-1} + a_i + a_{i+1}$ 的伤害。植物被破坏之后就不能再攻击了 (你可以认为攻击力变为了0)。

换句话说, 破坏一个植物所要承受的伤害是这个植物以及相邻两个植物的攻击力之和。

现在, Alex想知道, 有多少种破坏植物的方案, 使得破坏所有的植物后, 僵尸承受的伤害总和最小。由于答案可能很大, 所以请输出方案数对1000000007取模的结果。

方便起见, 可以认为 $a_0 = a_{n+1} = 0$ 。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 4010
#define MOD 1000000007
#define INF 0x3f3f3f3f
using namespace std;

int n,i;
int a[N];
LL mi[N];
LL f[N][N],c[N][N];
LL Ans;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL LL Mi(LL x,LL y){
    LL p=x,t=1,Res=1;
    for (;t<=y;(t&y)?Res=(Res*p)%MOD:0,p=(p*p)%MOD,t<<=1);
    return Res;
```

```

}

IL void Ready(){
    reg int i=0,j=0;
    mi[0]=1;
    for (i=1;i<=n;i++) mi[i]=mi[i-1]*i%MOD;
    for (i=1;i<=n;i++)
        for (j=1;j<=i;j++)
            c[i][j]=(mi[i]*Mi(mi[j],MOD-2)%MOD)*Mi(mi[i-j],MOD-2)%MOD;
}

IL LL Dfs(int l,int r){
    if (l>=r) return 1;
    if (f[l][r]) return f[l][r];
    reg int i=0;
    reg LL cnt=0;
    for (i=l+1;i<r;i++){
        if (a[i]<a[i+1] || a[i]<a[i-1]) continue;
        cnt+=(Dfs(l,i-1)*Dfs(i+1,r)%MOD)*c[r-1][i-1]%MOD;
        cnt%=MOD;
    }
    if (a[l]>=a[l+1]) cnt=(cnt+Dfs(l+1,r))%MOD;
    if (a[r]>=a[r-1]) cnt=(cnt+Dfs(l,r-1))%MOD;
    return f[l][r]=cnt;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();
    for (i=1;i<=n;i++) a[i]=read();
    Ready();
    a[0]=a[n+1]=-INF;
    cout<<Dfs(1,n)%MOD<<endl;
    return 0;
}

```

C4

A

A 项目安排

时间限制: 1000ms 内存限制: 65536kb

通过率: 234/248 (94.35%) 正确率: 234/534 (43.82%)

题目描述

北京欢乐谷有许多有趣的游玩项目，每个游玩项目有开始时间和结束时间。每个时间段最多只能游玩一个项目，请问最多可以游玩多少个项目。

输入格式

第一行包含一个整数 $n(1 \leq n \leq 10^6)$ 。

接下来 n 行，每行两个整数 $x_i, y_i(1 \leq x_i < y_i \leq 10^5)$ ，表示每个项目的开始时间和结束时间。

输出格式

输出一行一个整数，表示最多可以游玩的项目数。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 1000010
#define INF 0x3f3f3f3f
using namespace std;

int n,i,x,Ans;

struct Data{
    int x,y;
}a[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

inline bool cmp(const Data &a,const Data &b){return (a.y==b.y)?a.x<b.x:a.y<b.y;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
```

```

        return p*f;
    }

    int main(){
        #ifdef __Marvolo
        freopen("zht.in","r",stdin);
        freopen("zht.out","w",stdout);
        #endif
        scanf("%d",&n);
        for (i=1;i<=n;i++){
            a[i].x=read(); a[i].y=read();
        }
        sort(a+1, a+1+n, cmp);
        for (i=1;i<=n;i++){
            if (a[i].x >= x){
                x=a[i].y; Ans++;
            }
        }
        cout<<Ans<<endl;
        return 0;
    }
}

```

B

B 钢条切割

时间限制: 1000ms 内存限制: 65536kb

通过率: 135/206 (65.53%) 正确率: 135/569 (23.73%)

题目描述

Alex 正在修理小区里的防盗网。他需要 N 根钢条，第 i 根钢条的长度应该是 L_i 。他订购了一根很长的钢条，长度就是这 N 条钢条的总长度，由于约翰没有锯子，所以他找到了 Marvolo，想让他帮忙锯开钢条。

Marvolo 是个生财精，他乐意代劳，但劳动不是无偿的。每开动一次锯子，可以把钢条锯成两段，锯出 N 条钢条就需要开动 $N - 1$ 次机器。Marvolo 说，每次开机都要收加工费，加工费用是需要锯开的钢条长度的两倍。比如要把一段长 3 米的钢条锯开，就要收 6 元。Alex 不得不接受了 Marvolo 的要求，但他发现不同的切割次序会产生不同的费用。请帮 Alex 设计一个锯开钢条的方案，使得 Alex 付出的加工费之和最小。

输入

第一行: 单个整数 N , $1 \leq N \leq 2000$

第二行到第 $N + 1$ 行: 第 $i + 1$ 行有一个整数 L_i , $1 \leq L_i \leq 50000$

```

// Author Alex
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
int n, k;
LL ans;
priority_queue<LL, vector<LL>, greater<LL>> pq;

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; ++i) {
        scanf("%d", &k);
        pq.push(k);
    }
    if (n != 1)
        while (1) {
            LL t = pq.top(); pq.pop();
            t += pq.top(); pq.pop();
        }
}

```

```

        ans += t * 2;
        if (pq.empty()) break;
        pq.push(t);
    }
    printf("%lld\n", ans);
}

```

C

C 等差数列构造

时间限制：1000ms 内存限制：65536kb

通过率：205/239 (85.77%) 正确率：205/524 (39.12%)

题目描述

给出一个长度为 3 的非负整数数列 a, b, c 。对这个数列，可以执行这样一种操作：选择一个数，让它的值变为原来的值 $+1$ 。

问最少执行几次操作后，这个数列可以变成等差数列，也就是 $b-a=c-b$ 。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<map>
#include<queue>
#include<bitset>
#include<cstdlib>
#include<set>
#include<unordered_map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define low(x) (x&(-x))
#define IL inline
#define reg register
#define LL long long
#define N 500010
#define MOD 998244353
#define MO 1000000007
#define INF 0x3f3f3f3f
using namespace std;

LL T,n,m,i,j,k,x,y,z,Ans,sum,cnt;
int a[N],b[N];
char c[N];

struct Data{
    int x,y;
}d[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL LL Min(LL a,LL b){return (a<b)?a:b;}

```

```

IL int Max(int a,int b){return (a>b)?a:b;}
IL bool cmp(const Data a,const Data b){return a.x<b.x;}

IL LL Mi(LL x,LL y){
    LL p=x,t=1,Res=1;
    for (;t<=y;(t&y)?Res=(Res*p)%MOD:0,p=(p*p)%MOD,t<=<=1);
    return Res;
}

IL LL read(){
    LL p=0,f=1; char c=getchar();
    while (c<48||c>57) {if (c=='-') f=-1; c=getchar();}
    while (c>=48&&c<=57) p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __Marvolo
    freopen("5.in","r",stdin);
    freopen("5.out","w",stdout);
    #endif
    x=read(); y=read(); z=read();
    n=2*y; m=x+z;
    if (n<m){
        Ans=(m-n)/2;
        n+=2*Ans;
        while (n<m) n+=2,Ans++;
        while (m<n) m++,Ans++;
        cout<<Ans<<endl;
    } else {
        Ans=n-m;
        cout<<Ans<<endl;
    }
    return 0;
}

```

D

D 出路

时间限制：1000ms 内存限制：65536kb

通过率：137/188 (72.87%) 正确率：137/523 (26.20%)

题目描述

给定一个矩阵，问你从起点走到终点有多少种走法。起点为左上角，终点为右下角，要求只能向右走一格或向下走一格。#表示不可停留，.表示可以停留，保证左上角和右下角为.，要求结果对 1000000007 取模。

输入

第一行两个数为行数 $n(1 \leq n \leq 100)$ 和列数 $m(1 \leq m \leq 100)$

接下来 n 行，每行 m 个字符，表示矩阵。

```

#include<bits/stdc++.h>
#define mod 1000000007
#define N 105
using namespace std;
int n,m,dp[N][N];
char a[N][N];
int main()

```



```

{
    scanf("%d%d",&n,&m);
    for(int i=0; i<n; i++)
        scanf("%s",a[i]);
    for(int i=0; i<n; i++)
    {
        for(int j=0; j<m; j++)
        {
            dp[i][j]=0;
            if(a[i][j]=='#')
                continue;
            if(i==0&&j==0)
                dp[i][j]=1;
            if(i)
                dp[i][j]+=dp[i-1][j];
            if(j)
                dp[i][j]+=dp[i][j-1];
            dp[i][j]%=mod;
        }
    }
    printf("%d",dp[n-1][m-1]);
    return 0;
}

```

E

E 数列划分

时间限制：1000ms 内存限制：65536kb

通过率：116/143 (81.12%) 正确率：116/247 (46.96%)

题目描述

Marvolo 有一个长度为 n 的数列 a ，其中的元素有正有负。双十一快到了，Marvolo觉得一个数列太孤单了，他打算将原数列分成两个数列 b, c ，同时又想让两个数列的平均值之和最大。

现在，Marvolo 将划分数列的任务交给你，请给出一种符合要求的划分方案。当然，因为可能有多种符合条件的划分，Marvolo 又不想写spj，所以你只需要输出划分好的两个数列的平均值之和就好了，保留两位小数。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

int T,n,i;
int a[N];

```

```
double Ans;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    T=1;
    while (T--){
        n=read();
        for (i=1;i<=n;i++)    a[i]=read();
        sort(a+1,a+1+n);
        Ans=0;
        for (i=1;i<n;i++)    Ans+=a[i];
        Ans=Ans/(n-1)+a[n];
        printf("%.2lf\n",Ans);
    }
    return 0;
}
```

F

F 故障的打印机

时间限制：1000ms 内存限制：65536kb

通过率：60/87 (68.97%) 正确率：60/198 (30.30%)

题目描述

在机房里有一台打印机，因为年久失修，所以在打印东西时有一定概率发生故障。具体的说，当打印机在打印英文字母时，可能会将一个 **w** 字母打印成 **uu**，也可能将一个 **m** 字母打印成 **nn**。

现在，给出一段打印机的打印结果，请求出原文一共有多少种可能。由于答案可能很大，所以只需要输出其对 **1000000007** 取模的结果。

输入格式

共一行。一个字符串，保证其中只含有小写英文字母。表示打印机的打印结果。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#define N 100010
#define l(x) (x<<1)
#define r(x) ((x<<1)+1)
#define LL long long
#define INF 0x3f3f3f3f
#define MOD 1000000007
using namespace std;
```

```

int n,m,i,j,k,x,y,z,cnt,sum,T;
LL dp[N];
char c[N];

inline int Abs(int x){return (x<0)?-x:x;}
inline void Swap(int &a,int &b){a^=b^=a^=b;}
inline int Min(int a,int b){return (a<b)?a:b;}
inline int Max(int a,int b){return (a>b)?a:b;}

inline int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57)    {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    scanf("%s",&c);
    n=strlen(c);
    for (i=0;i<n;i++){
        if (c[i]=='m'||c[i]=='w')    {
            puts("0");    return 0;
        }
    }
    for (i=n;i;i--)    c[i]=c[i-1];
    dp[0]=dp[1]=1;    c[0]=0;
    for (i=2;i<=n;i++){
        dp[i]=dp[i-1];
        if (c[i]!='n'&&c[i]!='u')    continue;
        if (c[i]=='n'&&c[i-1]=='n')    dp[i]=(dp[i]+dp[i-2])%MOD;
        if (c[i]=='u'&&c[i-1]=='u')    dp[i]=(dp[i]+dp[i-2])%MOD;
    }
    cout<<dp[n]<<endl;
    return 0;
}

```

G

G 单周期CPU

时间限制：300ms 内存限制：65536kb

通过率：54/67 (80.60%) 正确率：54/119 (45.38%)

题目描述

\$Serein\$、\$Alex\$、\$Matrix53\$ 想要合作做一个单周期 \$CPU\$。

\$Serein\$ 想用 \$Logisim\$ 画 \$CPU\$，但是 \$Matrix53\$ 不喜欢用 \$Logisim\$。

\$Serein\$：\$emm\$...这样吧，我给你们出一道题，你们做出来了我们就用 \$Verilog\$ 来写 \$CPU\$。

\$Serein\$(自顾自地说)：有两个**正整数**数列 \$A\$ 和 \$B\$，每个数列都有 \$n\$ 个**互不相同**的元素，不妨假设为\$a_1,a_2,...a_n\$和\$b_1,b_2,...b_n\$。可以任意次交换数列 \$A\$ 中两个元素的位置，求\$\prod_{i=1}^n(a_i^{b_i})\$的**最大值**。

\$Matrix53\$(一段时间后)：我已经交换好了，先去买瓶可乐，等我回来就给你们讲思路和证明。

其实 \$Alex\$ 也想用 \$Logisim\$，于是他趁着 \$Matrix53\$ 买饮料的时候做了一件事，他重新排列了数列 \$B\$ 中的元素（注意，他并没有动已经交换好的\$A\$数列），现在\$\prod_{i=1}^n(a_i^{b_i})\$的值是**最小**的了，而 \$Serein\$ 对此睁一只眼闭一只眼。

你能看透 \$Alex\$ 重新排列数列 \$B\$ 后，数列 \$B\$ 的样子吗？

```

// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;
const int maxn = 1e5 + 5;

```

```

pair<int, int> b[maxn];
int ans[maxn];
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; ++i) {
        scanf("%d", &b[i].first);
        b[i].second = i;
    }
    sort(b, b + n);
    for (int i = 0; i < n; ++i) ans[b[i].second] = b[n - i - 1].first;
    for (int i = 0; i < n; ++i) printf("%d ", ans[i]);
    return 0;
}

```

H

H 1082实验

时间限制: 1000ms 内存限制: 65536kb

通过率: 2/16 (12.50%) 正确率: 2/41 (4.88%)

题目描述

\$Matrix53\$ 使用逐差法来处理 \$1082\$ 实验的数据，然后他会计算出钠光的波长。

但是 \$Matrix53\$ 其实不会用逐差法，他一用逐差法就停不下来。

具体来说，他本来有 \$n\$ 个实验数据 \$d_1, d_2 \dots d_n\$。

他每次逐差会从当前的数据中拿出两个数据，然后用大的数据减去小的数据，并将这个结果重新加入实验数据中。

也就是说，\$Matrix53\$ 每次逐差，实验数据的数目就会减 \$1\$。

当只剩一个实验数据时，\$Matrix53\$ 会停止逐差，因为这时候他不能逐差了。

求一个取数方案，使得最后剩的这个实验数据的值最小，以便 \$Matrix53\$ 在实验报告上吹嘘。

```

// Author: Matrix53
#include <cstdio>
#define MAX_SUM 250005

bool dp[MAX_SUM];

int main() {
    int n, x, i, sum = 0;
    dp[0] = true;
    scanf("%d", &n);
    while (n--) {
        scanf("%d", &x);
        sum += x;
        for (i = MAX_SUM; i >= x; --i) {
            if (dp[i - x]) dp[i] = true;
        }
    }
    for (i = sum / 2; i >= 0; --i) {
        if (dp[i]) {
            printf("%d", sum - i * 2);
            return 0;
        }
    }
}

```

```
}  
}
```

I

I 观察水位

时间限制：1000ms 内存限制：65536kb

通过率：9/14 (64.29%) 正确率：9/24 (37.50%)

题目描述

Lovarmo决定连续 n 天观察一条河。这条河每天都会有一个水位高度值。

Lovarmo每天都要去河边，在水面处做一个标记，但如果它与以前做的标记相重合，即水位高度与之前的某一天相等，就不会产生新的标记。水不会冲走标记。Lovarmo每天会记录下严格在水位以上的标记的数量，在第 i 天，这个值等于 a_i 。

定义 d_i 为第 i 天严格低于水位的标记数。你要找出 d_i 的最小可能总和。已知起初没有任何标记。

```
#include<bits/stdc++.h>  
#define N 100005  
using namespace std;  
int n,a[N],b[N];  
long long ans;  
int main()  
{  
    scanf("%d",&n);  
    for(int i=0; i<n; i++)  
    {  
        scanf("%d",&a[i]);  
        b[i]=a[i]+1;  
        if(i)  
            b[i]=max(b[i],b[i-1]);  
    }  
    for(int i=n-2; i>=0; i--)  
        b[i]=max(b[i+1]-1,b[i]);  
    for(int i=0; i<n; i++)  
        ans+=b[i]-a[i]-1;  
    printf("%lld",ans);  
    return 0;  
}
```

J

J 刷题计划

时间限制：1000ms 内存限制：204800kb

通过率：0/5 (0.00%) 正确率：0/18 (0.00%)

题目描述

为了备战算法期末考试，Marvolo计划在比赛开始前再刷 n 道题。

假设Marvolo在接下来每天的刷题量为 $a_1, a_2, \dots$ 。为了使刷题不枯燥，相邻的两天刷题数量不应该相同，即 $a_i \neq a_{i+1}$ 。

为了更加科学的训练，刷题数量应该是一天多一天少这样交叉进行。即：

如果 $a_i < a_{i+1}$ ，那么 $a_{i+1} > a_{i+2}$ 。

如果 $a_i > a_{i+1}$ ，那么 $a_{i+1} < a_{i+2}$ 。

计划中不限制刷完 n 道题需要的天数，但是第一天的刷题量是 k 道，即 $a_1 = k$ 。

现在Marvolo想知道，可以有多少种不同的刷题方案来刷完这 n 道题目。方案数可能很大，输出方案数对 998244353 的模数即可。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define N 2010
#define MOD 998244353
using namespace std;

int n,k,i,j;
int down[N][N],up[N][N],sumd[N][N],sumu[N][N];

int main(){
    #ifdef __Marvolo
        freopen("6.in","r",stdin);
        freopen("6.out","w",stdout);
    #endif
    scanf("%d%d",&n,&k);
    if (n==k){
        puts("1");
        return 0;
    }
    // 只做一天题的情况

    for (i=1;i<=n;i++){
        for (j=1;j<i;j++){
            up[i][j] = sumd[i-j][j-1];
            if (j+1 <= i-j) down[i][j] = (sumu[i-j][i-j]-sumu[i-j][j]+MOD)%MOD;
            // 用前缀和进行转移
        }
        down[i][i]=up[i][i]=1;
        // 边界情况

        for (j=1;j<=n;j++){
            sumd[i][j]=(sumd[i][j-1]+down[i][j])%MOD;
            sumu[i][j]=(sumu[i][j-1]+up[i][j])%MOD;
        }
    }
```

```
        // 维护前缀和
    }
    cout<<(down[n][k]+up[n][k])%MOD<<endl;
    return 0;
}
```

C5

A

A 最长路径

时间限制: 1000ms 内存限制: 65536kb

通过率: 187/232 (80.60%) 正确率: 187/697 (26.83%)

题目描述

一个有向图, N 个点编号 $1-N$ 。 M 条边, 每条边有一个权值 c 。

点对 (i, j) 之间的最短路径长度记为 $S_{(i,j)}$ 。 (若 i, j 不连通, 则 $S_{(i,j)} = -1$)

在所有点对 (共有 N^2 对) 中, 输出 $S_{(i,j)}$ 最大的点对。如果存在多个 $S_{(i,j)}$ 最大的情况, 则全部输出 (按 i 的大小由小到大输出, i 相同时按 j 的大小由小到大输出)。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 210
#define INF 0x3f3f3f3f
using namespace std;

int T,n,m,x,y,z,i,j;
int dis[N][N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

inline void Floyd(){
    register int i=0,j=0,k=0;
    for (k=1;k<=n;++k)
        for (i=1;i<=n;++i)
            for (j=1;j<=n;++j)
                dis[i][j]=Min(dis[i][j],dis[i][k]+dis[k][j]);
}
```



```

int main(){
    T=read();
    while (T--){
        n=read();    m=read();
        memset(dis,INF,sizeof(dis));
        for (i=1;i<=n;i++)    dis[i][i]=0;
        for (i=1;i<=m;i++){
            x=read();    y=read();    z=read();
            dis[x][y]=z;
        }
        Floyd();
        x=0;
        for (i=1;i<=n;i++)
            for (j=1;j<=n;j++)
                if (dis[i][j]!=INF) x=Max(x,dis[i][j]);
        for (i=1;i<=n;i++)
            for (j=1;j<=n;j++)
                if (dis[i][j]==x)    printf("%d %d\n",i,j);
    }
    return 0;
}

```

B

B 环路

时间限制：1000ms 内存限制：65536kb

通过率：204/241 (84.65%) 正确率：204/702 (29.06%)

题目描述

给出一个有向图，没有自环和重边，请你判断这个有向图中是否存在环路。

输入格式

第一行两个整数 n 和 m ，分别表示有向图的点数和边数。

接下来 m 行每行两个整数 x 和 y ，表示一条从 x 到 y 的有向边。

输出格式

若给定的有向图中存在环路，则输出 **YES**，否则输出 **NO**。

```

#include<bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> g[N];
int n, m;
int st[N], in[N];
void myExit() {
    puts("YES");
    exit(0);
}
int dfs(int u) {
    st[u] = 1;
    for (auto i : g[u]) {

```

```

        if(st[i] == 1) myExit();
        else if(!st[i]) dfs(i);
    }
    st[u] = 2;
}
bool topSort() {
    queue<int> q;
    for (int i = 1; i <= n; i ++)
        if(in[i] == 0)
            q.push(i);
    vector<int> ans;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);
        for (auto v : g[u]) {
            if(-- in[v] == 0)
                q.push(v);
        }
    }
    if(ans.size() == n) return true;
    return false;
}
int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= m; i ++) {
        int a, b;
        scanf("%d%d", &a, &b);
        g[a].push_back(b);
        in[b] ++;
    }
    for (int i = 1; i <= n; i ++) {
        if(!st[i])
            dfs(i);
    }
    puts("N0");
    return 0;
}

```

题目描述

js13kGames 是面向 HTML5 游戏开发者举办的 JavaScript 编程比赛。

js13kGames 从 2012 年开始举办，到今年(2021)已经举办了 10 届。

这个赛事是国际性的，参赛代码的总大小不能超过 13kb，故以此命名。

今年该比赛的主题是**Space**，冠军是名为 Space Garden 的游戏。

Matrix53 作为玩家，需要穿梭在行星之间。

Matrix53 现在位于行星 p ，需要去往行星 x 和行星 y 收集精灵。

行星之间可能会有太空通道，也可能没有，通道是双向的。

请你求出 Matrix53 在行星间穿梭的最短时间。

```
// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const ll inf = LLONG_MAX / 2;
const int maxv = 1e5 + 5;

struct edge {
    int to;
    ll w;
    edge(int a, ll b) : to(a), w(b) {}
};

struct qnode {
    int id;
    ll ndis;
    qnode(int a, ll b) : id(a), ndis(b) {}
    bool operator<(const qnode& a) const { return ndis > a.ndis; }
};

vector<edge> g[maxv];
int n, m;
ll dis[maxv];
bool done[maxv];

void dijkstra(int s) {
    for (int i = 1; i <= n; ++i) {
        dis[i] = inf;
        done[i] = false;
    }
    dis[s] = 0;
    priority_queue<qnode> q;
    q.push(qnode(s, 0));
    while (!q.empty()) {
        qnode u = q.top();
        q.pop();
        if (done[u.id]) continue;
        done[u.id] = true;
        for (auto v : g[u.id]) {
            if (done[v.to]) continue;
            if (dis[v.to] > u.ndis + v.w) {
```

```

        dis[v.to] = u.ndis + v.w;
        q.push(qnode(v.to, dis[v.to]));
    }
}
}
}

int main() {
    // freopen("D:/workspace/algo/E5-D/data/5.in", "r", stdin);
    int p, x, y;
    scanf("%d%d", &n, &m);
    scanf("%d%d%d", &p, &x, &y);
    for (int i = 0; i < m; ++i) {
        int u, v, t;
        scanf("%d%d%d", &u, &v, &t);
        g[u].push_back(edge(v, t));
        g[v].push_back(edge(u, t));
    }
    dijkstra(p);
    ll px = dis[x], py = dis[y];
    dijkstra(x);
    ll xy = dis[y];
    if (px == inf || py == inf || xy == inf) {
        printf("-1");
    } else {
        printf("%lld", xy + min(px, py));
    }
    return 1;
}

```

D

D 通不通

时间限制：1000ms 内存限制：65536kb

通过率：102/176 (57.95%) 正确率：102/528 (19.32%)

题目描述

一共有 n 个城市。

现在有两种操作：

第一种：在两个城市间新建一条路。（路双向互通，不保证两个城市间只会有一条直接的路）

第二种：询问两个城市间是否存在互通路径。

```

#include<bits/stdc++.h>
#define mod 1000000007
#define N 1000005
using namespace std;
int n,m,f[N];
int find(int k)
{
    if(k==f[k])
        return k;
    f[k]=find(f[k]);
    return f[k];
}

```

```

}
void merge(int l,int r)
{
    f[find(r)]=find(l);
}
int ask(int l,int r)
{
    return find(l)==find(r);
}
void initial()
{
    for(int i=1; i<=n; i++)
        f[i]=i;
}
int main()
{
    scanf("%d%d",&n,&m);
    initial();
    while(m--)
    {
        int op,u,v;
        scanf("%d%d%d",&op,&u,&v);
        if(op==1)
            merge(u,v);
        else
        {
            if(ask(u,v))
                puts("YES");
            else
                puts("NO");
        }
    }
    return 0;
}

```

E

E 间谍

时间限制：1000ms 内存限制：65536kb

通过率：17/22 (77.27%) 正确率：17/51 (33.33%)

题目描述

A 国经过长时间的排查，摸清了国内活动的他国间谍的信息，并准备开展一次行动将他们一网打尽。

可是，行动计划在前一天晚上被内鬼泄露出去了，间谍们打算连夜转移到一些安全屋内，来躲避天亮开始的抓捕行动。已知有 n 个间谍和 n 个安全屋，一个安全屋只能容纳一个间谍，间谍和安全屋均有一个二维坐标。在天亮之前，一个间谍只能移动至多 d 的距离（指欧氏距离）。

已知只有躲进安全屋中的间谍可以避免被抓，且间谍们非常团结，他们会**尽可能让更多的人避免被抓**。请你计算一下，至多有多少个间谍不会被抓获呢？

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>

```

```

#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 210
#define INF 0x3f3f3f3f
using namespace std;

int n,d,i,j,lsum=1,Found,flow,cnt,l,a1;
int head[2*N],gap[2*N],gaps[2*N];

struct Data{
    int x,y;
}a[N],b[N];

struct Flow{
    int t,next,fl,re;
}e[N*N*8];

inline void Add(int s,int t,int fl){
    e[lsum].t=t;    e[lsum].fl=fl;    e[lsum].next=head[s];    e[lsum].re=lsum+1;
head[s]=lsum++;
    e[lsum].t=s;    e[lsum].fl=0;    e[lsum].next=head[t];    e[lsum].re=lsum-1;
head[t]=lsum++;
}

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

inline void Sap(int x){
    int i=0,mingap=cnt-1,F=a1;
    if (x==cnt)    {Found=1;    flow+=a1;    return;}
    for (i=head[x];i;i=e[i].next)
        if (e[i].fl){
            if (gap[e[i].t]+1==gap[x]){
                a1=Min(a1,e[i].fl);    Sap(e[i].t);
                if (gap[1]>=cnt)    return;
                if (Found)    break;
                a1=F;
            }
            mingap=Min(mingap,gap[e[i].t]);
        }
    if (!Found){
        gaps[gap[x]]--;    if (!gaps[gap[x]])    gap[1]=cnt;
        gaps[gap[x]=mingap+1]++;
    }    else e[i].fl-=a1,e[e[i].re].fl+=a1;
}

```

```

int main(){
    n=read();    d=read();
    for (i=1;i<=n;i++){
        a[i].x=read();  a[i].y=read();
    }
    for (i=1;i<=n;i++){
        b[i].x=read();  b[i].y=read();
    }
    d = d*d;
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++){
            l = (a[i].x-b[j].x)*(a[i].x-b[j].x) + (a[i].y-b[j].y)*(a[i].y-
b[j].y);
            if (l <= d){
                Add(1+i,1+n+j,1);
            }
        }
    cnt = 2*n+2;
    for (i=1;i<=n;i++){
        Add(1,1+i,1);    Add(1+n+i,cnt,1);
    }
    gaps[0]=cnt;
    while (gap[1]<cnt){
        Found=0; a1=INF;    Sap(1);
    }
    cout<<flow<<endl;
    return 0;
}

```

F

F 不要在上课前才开始写作业

时间限制: 1000ms 内存限制: 65536kb

通过率: 9/19 (47.37%) 正确率: 9/40 (22.50%)

题目描述

现在是中午 12:30, 下午两点的课要收作业, Alex 此时还一笔没动。

好在 Alex 打字速度比较快, 它可以写成电子版打印出来提交。

在 13:50 的时候, Alex 准备好了 pdf 版的作业, 现在他需要用最短的时间打印出作业并冲到教室。

校园可以抽象成一张无向图, 宿舍在 1 号点, 教室在 n 号点

Alex 知道, 学校里有 k 家打印点, 分别对应图上的 $s_1, s_2, \dots, s_k$ 号点

Alex 需要先跑去一家答应店打印作业, 然后再冲向教室。

如果忽略打印所需要的时间, Alex 最少需要多长时间才能赶到教室呢?

```

// Author: Alex
#include <bits/stdc++.h>

using namespace std;
const int N = 100100;

```

```

int n, m, num, dis1[N], dis2[N];
vector<pair<int, int>> G[N];

bool vis[N];
priority_queue<pair<int, int>> pq;
void dijkstra(int s, int n, int *dis) {
    for (int i = 1; i <= n; ++i)
        dis[i] = 0x3f3f3f3f, vis[i] = false;
    dis[s] = 0; pq.emplace(-dis[s], s);
    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (vis[u]) continue;
        vis[u] = true;
        for (auto &&[v, w] : G[u]) if (dis[u] + w < dis[v]) {
            dis[v] = dis[u] + w;
            pq.emplace(-dis[v], v);
        }
    }
}

int main() {
    cin >> n >> m >> num;
    for (int i = 1, u, v, w; i <= m; ++i) {
        cin >> u >> v >> w;
        G[u].emplace_back(v, w);
        G[v].emplace_back(u, w);
    }
    dijkstra(1, n, dis1);
    dijkstra(n, n, dis2);
    int ans = 0x3f3f3f3f;
    while (num--) {
        int x; cin >> x;
        ans = min(ans, dis1[x] + dis2[x]);
    }
    cout << dis1[n] << endl;
    return 0;
}

```

G

G 演出

时间限制：1000ms 内存限制：65536kb

通过率：1/3 (33.33%) 正确率：1/7 (14.29%)

题目描述

一场演出即将开始，Buttersky迟迟赶来，他想拥有最佳观看位置，但是他又不想费大力气。

表演区域为边长为 n 的正方形区域，而整个场地是一个边长为 $3n$ 的正方形区域，表演区域位于正中央。这场演出的宣传很到位，每一个非表演区域都站了一个人。

如果Buttersky前面没有任何人阻挡他的视线，那么他就获得了他想要的最佳观看位置。

现在Buttersky需要挤出一条路来，每一次他都可以把一个相邻的人挤到他之前的位置，从而到达那个人的位置，但每挤一个人都需要耗费一定的力气。最开始时，Buttersky可以选择去挤任意一个位于边缘的人。

对了，演出即将开始，Buttersky需要尽快准备好，所以他最多只能挤人 $3n$ 次。

那么，为了获得最佳观看位置，Buttersky最少要耗费多少力气呢。

规定只有前后左右四个方向算相邻，斜着的不算，同时只有与表演区域某个位置相邻才算最佳观看位置。


```

#include<bits/stdc++.h>
#define inf 0x3f3f3f3f
#define N 303
using namespace std;
int n,ans,a[N][N],dis[N][N],c[N][N];
int main()
{
    scanf("%d",&n);
    ans=inf;
    for(int i=1; i<=n*3; i++)
    {
        for(int j=1; j<=n*3; j++)
        {
            scanf("%d",&a[i][j]);
            if(i==1||j==1||i==n*3||j==n*3)
                dis[i][j]=a[i][j];
            else
                dis[i][j]=inf;
        }
    }
    int t=n*3-1;
    while(t--)
    {
        for(int i=1; i<=n*3; i++)
        {
            for(int j=1; j<=n*3; j++)
            {
                c[i][j]=dis[i][j];
                if(i!=1)
                    c[i][j]=min(c[i][j],dis[i-1][j]+a[i][j]);
                if(j!=1)
                    c[i][j]=min(c[i][j],dis[i][j-1]+a[i][j]);
                if(i!=n*3)
                    c[i][j]=min(c[i][j],dis[i+1][j]+a[i][j]);
                if(j!=n*3)
                    c[i][j]=min(c[i][j],dis[i][j+1]+a[i][j]);
            }
        }
        for(int i=1; i<=n*3; i++)
            for(int j=1; j<=n*3; j++)
                dis[i][j]=c[i][j];
    }
    for(int i=n+1; i<=n*2; i++)
        for(int j=n+1; j<=n*2; j++)
            ans=min(ans,dis[i][j]);
    printf("%d",ans);
    return 0;
}

```

H 题目搭配

时间限制: 1000ms 内存限制: 65536kb

通过率: 17/26 (65.38%) 正确率: 17/56 (30.36%)

题目背景

一场合适的算法比赛，应该由签到题，中档题，压轴题这三种题型来组成。Marvolo准备出几场恰好有三道题的比赛，也就是说，这三种题型的题目各出一道。

每一种题型都有 m 道候选题目，不妨设第 i 道签到题，中档题，压轴题的难度分别是 e_i, m_i, h_i ，难度用非负整数表示。为了控制难度梯度，要求同一场比赛中，签到题的难度必须小于中档题的难度，中档题的难度必须小于压轴题的难度。

现在，Marvolo想知道，用这一共 $3m$ 道候选题目，最多可以出几场符合要求的比赛呢？

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<map>
#include<queue>
#include<bitset>
#include<cstdlib>
#include<set>
#include<unordered_map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define low(x) (x&(-x))
#define IL inline
#define reg register
#define LL long long
#define N 500010
#define MOD 1000000007
#define INF 0x3f3f3f3f
using namespace std;

int T,n,m,i,j,k,x,y,z,Ans,sum,cnt;
int a[N],b[N],c[N];

struct Data{
    int x,y;
}d[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}
IL bool cmp(const Data a,const Data b){return a.x<b.x;}

IL LL Mi(LL x,LL y){
    LL p=x,t=1,Res=1;
    for (;t<=y;(t&y)?Res=(Res*p)%MOD:0,p=(p*p)%MOD,t<=<1);
    return Res;
}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-') f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}
```

```

}

int main(){
    #ifdef __Marvolo
    freopen("10.in","r",stdin);
    freopen("10.out","w",stdout);
    #endif
    n=read();
    for (i=1;i<=n;i++) a[i]=read();
    for (i=1;i<=n;i++) b[i]=read();
    for (i=1;i<=n;i++) c[i]=read();
    sort(a+1,a+1+n);
    sort(b+1,b+1+n);
    sort(c+1,c+1+n);
    x=y=z=1;
    while (x<=n && y<=n && z<=n){
        while (y<=n && a[x]>=b[y]) y++;
        if (y>n) break;
        while (z<=n && b[y]>=c[z]) z++;
        if (z>n) break;
        Ans++; x++; y++; z++;
    }
    cout<<Ans<<endl;
    return 0;
}

```

I Matrix53的生日礼物

时间限制：1000ms 内存限制：65536kb

通过率：2/4 (50.00%) 正确率：2/10 (20.00%)

题目描述

Alex 送了 Matrix53 一张有 n 个点， m 条边的无向联通图作为生日礼物。

但是 Matrix53 不喜欢这张平平无奇的图，他比较喜欢树，于是他决定从图中取出一棵生成树 T 。

Matrix53 取出第 i 条边会耗费他 a_i 点体力，同时会收获 b_i 点愉悦值，Matrix53 希望他取出来的树满足树中各边的愉悦值之和除以体力之和，即 $\sum_{i \in T} b_i$ 与 $\sum_{i \in T} a_i$ 的比值最大。

但是 Matrix53 忙着做实验，于是把这个问题拜托给了聪明的你。

```

// Author: Marvolo
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstring>
#include <iostream>
#include <map>
#include <queue>
#include <vector>
#define l(x) (x << 1)
#define r(x) ((x << 1) | 1)
#define IL inline
#define reg register
#define LL long long
#define N 100010

```

```

#define EPS 1e-5
#define INF 0x3f3f3f3f
using namespace std;

int n, m, i, tot, x, y;
double p, q, low, high, mid, ans;
int f[N];

struct Data {
    int x, y, a, b;
    double l;
} a[N];

inline bool cmp(const Data &a, const Data &b) { return a.l > b.l; }

IL int Find(int x) { return (x == f[x]) ? x : f[x] = Find(f[x]); }

IL double Abs(double x) { return (x < 0) ? -x : x; }

IL int read() {
    int p = 0, f = 1;
    char c = getchar();
    while (c < 48 || c > 57) {
        if (c == '-') f = -1;
        c = getchar();
    }
    while (c >= 48 && c <= 57) p = (p << 1) + (p << 3) + c - 48, c = getchar();
    return p * f;
}

IL bool kruskal(double p) {
    reg int i = 0, tot = 0, x = 0, y = 0;
    double sum = 0;
    for (i = 1; i <= m; i++) a[i].l = a[i].b - p * a[i].a;
    sort(a + 1, a + 1 + m, cmp);
    for (i = 1; i <= n; i++) f[i] = i;
    for (i = 1; i <= m; i++) {
        x = Find(a[i].x);
        y = Find(a[i].y);
        if (x == y) continue;
        f[x] = y;
        sum += a[i].l;
        if (++tot >= n - 1) break;
    }
    return (sum >= 0);
}

int main() {
#ifdef __Marvolo
    freopen("zht.in", "r", stdin);
    freopen("zht.out", "w", stdout);
#endif
    n = read();
    m = read();
    for (i = 1; i <= m; i++) {
        a[i].x = read();
        a[i].y = read();
        a[i].a = read();
    }
}

```

```

        a[i].b = read();
    }

    low = 0;
    high = 999999999.0;
    while (Abs(low - high) > EPS) {
        mid = (low + high) / 2;
        if (Kruskal(mid))
            low = ans = mid;
        else
            high = mid;
    }
    printf("%.3lf\n", ans);
    return 0;
}

```

J

J 修建公路

时间限制：1000ms 内存限制：65536kb

通过率：98/141 (69.50%) 正确率：98/326 (30.06%)

题目描述

建筑师正在规划城镇道路建设，城镇共有 n 个地点。他想用最少的钱修建道路来让所有地点之间能互相抵达，已知花费的钱和道路的长度成正比，那么道路长度总和最小是多少呢？

输入

第一行一个整数为地点数 $n(1 \leq n \leq 1000)$ 。

接下来 n 行，第 i 行两个整数 $x_i(1 \leq x_i \leq 100)$ 和 $y_i(1 \leq y_i \leq 100)$ ，表示城镇的横纵坐标。保证城市坐标互不相同。

输出

```

#include<bits/stdc++.h>
#define mod 1000000007
#define N 1005
using namespace std;
int n,m,x[N],y[N],vis[N];
double dis[N],ans;
int main()
{
    scanf("%d",&n);
    for(int i=0; i<n; i++)
        scanf("%d%d",&x[i],&y[i]),dis[i]=1e100;
    dis[0]=0;
    while(m!=n)
    {
        m++;
        double mi=1e100;
        int k=0;
        for(int i=0; i<n; i++)
        {
            if(!vis[i]&&dis[i]<mi)
            {
                k=i;
                mi=dis[i];
            }
        }
    }
}

```

```
    ans+=dis[k];
    dis[k]=0;
    vis[k]=1;
    for(int i=0; i<n; i++)
    {
        if(!vis[i])
            dis[i]=min(dis[i],sqrt((x[i]-x[k])*(x[i]-x[k])+(y[i]-y[k])*(y[i]-y[k]))));
    }
}
printf("%.2f",ans);
return 0;
}
```

C6

A

A 非退化三角形

时间限制: 1000ms 内存限制: 65536kb

通过率: 246/246 (100.00%) 正确率: 246/437 (56.29%)

题目描述

给出平面上的 n 个点，请计算出这些点可以构成多少个非退化的三角形。

非退化的定义是，三角形的三个点没有共线。

输入格式

第一行一个正整数 n ，表示点的数量。

接下来 n 行，每行有个2整数，表示一个点的坐标。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define EPS 1e-5
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

struct P{
    LL x,y;
    P operator -(P a){return (P){x-a.x, y-a.y};}
    LL operator *(P a){return x*a.y - y*a.x;}
};

int i,j,k,n,Ans;
P a[N];

int main(){
    scanf("%d",&n);
    for (i=1;i<=n;i++){
        scanf("%lld%lld",&a[i].x,&a[i].y);
    }
    for (i=1;i<=n;i++)
        for (j=i+1;j<=n;j++)
            for (k=j+1;k<=n;k++){
                if ((a[i]-a[j])*(a[i]-a[k]))    Ans++;
            }
}
```

```
cout<<Ans<<endl;
return 0;
}
```

B

B 多边形面积

时间限制：1000ms 内存限制：65536kb

通过率：242/245 (98.78%) 正确率：242/391 (61.89%)

题目描述

给出一个简单多边形(没有缺口)，它的边要么是垂直的，要么是水平的。要求计算多边形的面积。

多边形被放置在一个X-Y的笛卡尔平面上，它所有的边都平行于两条坐标轴之一。然后按逆时针方向给出各顶点的坐标值。所有的坐标值都是整数(因此多边形的面积也为整数)

输入

件第一行给出多边形的顶点数 $n(3 \leq n \leq 100)$ 。接下来的 n 行每行给出多边形一个顶点的坐标值 X 和 Y (都为整数并且用空格隔开)。顶点按逆时针方向逐个给出。并且多边形的每一个顶点的坐标值 $-200 \leq x,y \leq 200$ 。多边形最后是靠从最后一个顶点到第一个顶点画一条边来封闭的。

```
#include<bits/stdc++.h>
#define N 200005
using namespace std;
struct point
{
    int x,y;
} p[N];
int n,ans;
int cproduct(point a1,point a2,point b1,point b2)
{
    return (a2.x-a1.x)*(b2.y-b1.y)-(a2.y-a1.y)*(b2.x-b1.x);
}
int main()
{
    scanf("%d",&n);
    scanf("%d%d",&p[0].x,&p[0].y);
    scanf("%d%d",&p[1].x,&p[1].y);
    for(int i=2; i<n; i++)
    {
        scanf("%d%d",&p[i].x,&p[i].y);
        ans+=cproduct(p[0],p[i-1],p[0],p[i]);
    }
    printf("%d",ans/2);
    return 0;
}
```

C

C 最远的距离

时间限制: 1000ms 内存限制: 65536kb

通过率: 89/204 (43.63%) 正确率: 89/826 (10.77%)

题目描述

给定二维平面点集，求点集中两点间最大距离

输入

第一行一个正整数 n ，为点集中点的数量

接下来 n 行，每行两个浮点数 x, y ，表示点集中一个点的坐标

输出

```
#pragma GCC optimize(3,"Ofast","inline")
#pragma GCC optimize("Ofast")
#pragma GCC optimize("unroll-loops")
#pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,avx,tune=native")
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define EPS 1e-8
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

inline double Min(double a,double b){return (a<b)?a:b;}
inline double Max(double a,double b){return (a>b)?a:b;}
inline double Fabs(double x){return (x<=-EPS)?-x:(x>EPS)?x:0;}
inline bool Equ(double x,double y){return (Fabs(x-y)<EPS);}
inline int Sgn(double x){
    if (Fabs(x)==0) return 0;
    if (x>0) return 1; else return -1;
}

struct Vector{
    double x,y;
    void Clean(void){x=y=0;}
    void write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
    Vector operator +(Vector p){return (Vector){x+p.x,y+p.y};}
    Vector operator -(Vector p){return (Vector){x-p.x,y-p.y};}
    double operator *(Vector p){return x*p.y-y*p.x;}
    double operator &(Vector p){return x*p.x+y*p.y;}
};
typedef Vector V;

struct Point{
    double x,y;
```

```

void Clean(void){x=y=0.0;}
void write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
bool operator ==(Point a){return (Equ(x-a.x,0) && Equ(y-a.y,0));}
Point operator +(Point p){return (Point){x+p.x,y+p.y};}
Point operator /(double l){return (Point){x/l,y/l};}
Vector operator -(Point p){return (Vector){x-p.x,y-p.y};}
};
typedef Point P;

int n,i;
P a[N],b[N],0;

inline void SW(P &a,P &b){P t=a;    a=b;    b=t;}
inline double Dis2(P a,P b){return pow(a.x-b.x,2)+pow(a.y-b.y,2);}
inline double Dis(P a,P b){return pow(pow(a.x-b.x,2)+pow(a.y-b.y,2),0.5);}

inline bool Cmp(P a,P b){
    double l=(a-0)*(b-0);
    if (l>0 || (Equ(l,0) && Fabs(a.x-0.x)<Fabs(b.x-0.x)))
        return 1;
    else return 0;
}

inline void Sort(P a[],int n){
    register int i=0;
    for (i=2;i<=n;i++)
        if (a[i].x<a[1].x || (a[i].x==a[1].x && a[i].y<a[1].y))
            SW(a[1],a[i]);
    0=a[1];
    sort(a+1,a+1+n,Cmp);
}

inline void Convex(P a[],P b[],int n,int &l){
    register int i=0;    l=0;
    Sort(a,n);
    n=unique(a+1,a+1+n)-(a+1);
    if (n<=1){
        b[1]=a[1];    l=1;    return;
    }
    if (Equ((a[2]-a[1])*(a[n]-a[1]),0)){
        b[1]=a[1];    b[2]=a[n];    l=2;    return;
    }
    b[++l]=a[1];    b[++l]=a[2];
    for (i=3;i<=n;i++){
        while ((b[l]-a[i])*(b[l-1]-a[i])>=0 && l>1) l--;
        b[++l]=a[i];
    }
    if (Equ((b[l]-b[l-1])*(b[l]-b[1]),0))    --l;
    //加上这句可以去掉所有凸包边上的点
}

inline double Get_Diameter(P a[],int n){
    register int i=0,j=2;
    double l=0;
    if (n==1)    return 0;
    if (n==2)    return Dis(a[1],a[2]);
    a[0]=a[n];
    for (i=0;i<n;i++){

```

```

        while ((a[j]-a[i])*(a[j]-a[i+1]) < (a[j+1]-a[i])*(a[j+1]-a[i+1]))
            j=(j+1)%n;
        l=Max(l,Max(Dis(a[i],a[j]),Dis(a[i+1],a[j])));
    }
    return l;
}

IL void vio(){
    reg int i=0,j=0;
    double ans=0;
    for (i=1;i<=n;i++)
        for (j=i+1;j<=n;j++){
            ans=Max(ans,Dis(a[i],a[j]));
        }
    printf("%.6lf\n",ans+EPS);
    return;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    scanf("%d",&n);
    for (i=1;i<=n;i++) scanf("%lf%lf",&a[i].x,&a[i].y);
    if (n==2) {
        printf("%.6lf\n",Dis(a[1],a[2]));
        return 0;
    }
    if (n<=2000){
        vio();
        return 0;
    }
    int cnt;
    Convex(a,b,n,cnt);
    if (cnt==1) printf("0.000000\n");
    else printf("%.6lf\n",Get_Diameter(b,cnt)+EPS);
    return 0;
}

```

D

D 弦上交点

时间限制：1000ms 内存限制：65536kb

通过率：3/4 (75.00%) 正确率：3/7 (42.86%)

题目描述

在一个圆周上有 $2n$ 个不同的点。他们以顺时针的顺序被标记为 $1, 2, \dots, 2n$ 。

起初， k 条弦连接了 k 对点，且这 $2k$ 个点互不相同。

你想要增加 $n - k$ 条弦连接剩下的 $2(n - k)$ 个点（每个点都刚好是一条弦的端点）。

请计算这 n 条弦交点数目的最大值。

```

#include <iostream>
#include <vector>

```

```

using namespace std;

bool intersect(pair<int, int> c, pair<int, int> d) {
    if (c.first > d.first) swap(c, d);
    return c.second > d.first and c.second < d.second;
}

void solve() {
    int n, k;
    cin >> n >> k;

    vector<pair<int, int>> chords;
    vector<bool> used(2 * n + 1, false);
    for (int i = 1; i <= k; i++) {
        int x, y;
        cin >> x >> y;
        if (x > y) swap(x, y);
        chords.push_back(make_pair(x, y));
        used[x] = used[y] = true;
    }

    vector<int> unused;
    for (int i = 1; i <= 2 * n; i++)
        if (!used[i]) unused.push_back(i);

    for (int i = 0; i < n - k; i++)
        chords.push_back(make_pair(unused[i], unused[i + n - k]));

    int ans = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            ans += intersect(chords[i], chords[j]);

    cout << ans << endl;
}

int main() {
    #ifdef __Marvolo
    freopen("zht.in", "r", stdin);
    freopen("zht.out", "w", stdout);
    #endif
    int t;
    t=1;

    for (int i = 1; i <= t; i++)
        solve();

    return 0;
}

```

E 哈希冲突

时间限制: 1000ms 内存限制: 128000kb

通过率: 5/9 (55.56%) 正确率: 5/19 (26.32%)

题目描述

现你需要给一个指定集合 $S = \{a_1, a_2, \dots, a_n\}$ 设计一个哈希函数 $h_m(x) = x \bmod m$ ，使得对于集合 S ，哈希函数 $h_m(x)$ 没有哈希冲突，也即 $\forall 1 \leq i < j \leq n, h_m(a_i) \neq h_m(a_j)$ 。

请问在满足上述条件的哈希函数中， m 的最小值是多少。

```
#include <bits/stdc++.h>
#define maxn 1100086

using namespace std;

const double pi = acos(-1);

struct Complex{
    double x, y;
    Complex(double xx = 0, double yy = 0){
        x = xx, y = yy;
    }
};

Complex operator + (Complex a, Complex b){
    return Complex(a.x + b.x, a.y + b.y);
}

Complex operator - (Complex a, Complex b){
    return Complex(a.x - b.x, a.y - b.y);
}

Complex operator * (Complex a, Complex b){
    return Complex(a.x * b.x - a.y * b.y, a.x * b.y + a.y * b.x);
}

int pos[maxn];

void FFT(Complex *a, int lim, int type){
    for(int i = 0; i < lim; i++){
        pos[i] = (pos[i >> 1] >> 1) | ((i & 1) ? (lim >> 1) : 0);
        if(i < pos[i]) swap(a[i], a[pos[i]]);
    }
    for(int mid = 1; mid < lim; mid <= 1){
        Complex wn(cos(pi / mid), type * sin(pi / mid));
        for(int i = mid << 1, j = 0; j < lim; j += i){
            Complex w(1, 0);
            for(int k = 0; k < mid; k++, w = w * wn){
                Complex x = a[j + k], y = w * a[j + k + mid];
                a[j + k] = x + y;
                a[j + k + mid] = x - y;
            }
        }
    }
    if(type == -1) for(int i = 0; i < lim; i++) a[i].x /= lim;
}

int n;
```

```

int x;
Complex a[maxn], b[maxn];
bool tag[maxn];

int main(){
// freopen("data5.in", "r", stdin);
// freopen("data5.out", "w", stdout);
scanf("%d", &n);
for(int i = 1;i <= n;i++) scanf("%d", &x), a[x].x = b[(int)5e5 - x].x = 1;
int lim = 1 << 20;
FFT(a, lim, 1), FFT(b, lim, 1);
for(int i = 0;i < lim;i++) a[i] = a[i] * b[i];
FFT(a, lim, -1);
for(int i = 5e5;i < lim;i++) if(fabs(a[i].x) > 1e-3) tag[i - (int)5e5] =
true;
for(int i = 1;;i++){
    bool flag = false;
    for(int j = i;j <= 5e5;j += i){
        if(tag[j]){
            flag = true;
            break;
        }
    }
    if(!flag) return printf("%d", i), 0;
}
}

```

F

F 简单的A+B Problem

时间限制：2000ms 内存限制：128000kb

通过率：222/239 (92.89%) 正确率：222/602 (36.88%)

题目描述

这是一个模板题。

给出两个大整数，输出他们的乘积。

输入格式

输入一共有两行，每行一个**正整数**，保证**没有前导0**。

输出格式

输出一行，表示这两个数的乘积，不能有前导0。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)

```

```

#define IL inline
#define reg register
#define LL long long
#define N 4000010
#define MOD 998244353
#define INF 0x3f3f3f3f
using namespace std;

int n,i,j,len;
char c[N],s[N];
LL a[N],b[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(LL &a,LL &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

inline LL Mi(LL x,LL y){
    LL p=x,t=1,Res=1;
    for (;t<=y;(t&y)?Res=(Res*p)%MOD:0,p=(p*p)%MOD,t<=<=1);
    return Res;
}

IL void Rader(LL y[]){
    int i=0,j=0,k=0;
    for (i=1,j=len>>1;i<len-1;i++){
        if (i<j)    Swap(y[i],y[j]);
        k=len>>1;
        while (j>=k)    j-=k,k>=>=1;
        if (j<k)    j+=k;
    }
}

IL void NTT(LL a[],int f){
    int i=0,j=0,k=0;
    LL wn=0,w=0,u=0,t=0;
    Rader(a);
    for (k=2;k<=len;k<=<=1){
        wn=Mi(3,(MOD-1)/k);
        if (f==-1)    wn=Mi(wn,MOD-2);
        for (i=0;i<len;i+=k){
            w=1;
            for (j=i;j<i+k/2;j++){
                u=a[j]; t=(w*a[j+k/2])%MOD;
                a[j]=(u+t)%MOD; a[j+k/2]=(u-t+MOD)%MOD;
                w=(w*wn)%MOD;
            }
        }
    }
    if (f==-1)
        for (i=0;i<len;i++) a[i]=(a[i]*Mi(len,MOD-2))%MOD;
}

```

```

}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    scanf("%s",s);
    scanf("%s",c);

    n=strlen(c);
    for (i=n-1;i>=0;i--)    a[n-1-i]=c[i]-48;

    n=strlen(s);
    for (i=n-1;i>=0;i--)    b[n-1-i]=s[i]-48;
    n=Max(strlen(c),strlen(s))*2;
    for (len=1;len<=n;len<=<=1);
    NTT(a,len); NTT(b,len);
    for (i=0;i<=len;i++)    a[i]*=b[i];
    NTT(a,-1);
    for (i=0;i<=len || a[i];i++)    a[i+1]+=a[i]/10,a[i]%=10;
    for (i=len+10;i>=0;i--)
        if (a[i]){
            n=i;    break;
        }
    for (i=n;i>=0;i--)    putchar(a[i]+48);
    putchar(10);

    return 0;
}

```

G

G Matrix53的数列 - Hard

时间限制: 1000ms 内存限制: 65536kb

通过率: 7/16 (43.75%) 正确率: 7/41 (17.07%)

题目描述

Matrix53 和 Marvolo 发财啦，因为他们上次解决了 Medium 难度的数列问题，获得了“世纪难题”委员会的奖金。

“世纪难题”委员会惊讶于 Matrix53 和 Marvolo 的解题能力，对他们大加称赞。

“计算姬”委员会知道了他们的研究成果，想让他们对 Easy 难度的数列问题的时间复杂度做进一步的优化。

给定一正整数数列，统计出**数列中的三角形**的总数

数列中的三角形：若有三个数处于数列中三个不同的位置，且这三个数可以作为某一个三角形的三条边，则这三个数构成一个“数列中的三角形”

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)

```



```

#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 400010
#define INF 0x3f3f3f3f
#define PI 3.14159265358979323846
using namespace std;

LL T,i,n,len,x,Ans;
LL b[N],c[N],s[N],d[N];

struct Cp{
    double a,b;
}a[N];

Cp operator +(Cp x,Cp y){return (Cp){x.a+y.a,x.b+y.b};}
Cp operator -(Cp x,Cp y){return (Cp){x.a-y.a,x.b-y.b};}
Cp operator *(Cp x,Cp y){return (Cp){x.a*y.a-x.b*y.b,x.a*y.b+x.b*y.a};}

IL void Swap(Cp &a,Cp &b){Cp c;c=a;    a=b; b=c;}

IL LL read(){
    LL p=0,f=1;    char    c=getchar();
    while (c<48||c>57)    {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

inline void Rader(Cp *y){
    int i=0,j=len>>1,k=0;
    for (i=1;i<len-1;i++){
        if (i<j)    swap(y[i],y[j]);
        k=len>>1;
        while (j>=k)    j-=k,k>=>1;
        if (j<k)    j+=k;
    }
}

inline void FFT(Cp *y,int f){
    int i=0,j=0,k=0;
    Cp w,a,b,wn;
    Rader(y);
    for (i=2;i<=len;i<=>1){
        wn=(Cp){cos(2*f*PI/i),sin(2*f*PI/i)};
        for (j=0;j<len;j+=i){
            w=(Cp){1,0};
            for (k=j;k<j+i/2;k++){
                a=y[k];    b=w*y[k+i/2];
                y[k]=a+b;    y[k+i/2]=a-b;
                w=w*wn;
            }
        }
    }
    if (f==1)
        for (i=0;i<=len;i++)    y[i].a/=len;
}

```

```

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    //T=read();
    //while (T--){
        n=read();
        memset(a,0,sizeof(a));
        memset(c,0,sizeof(c));
        for (i=1;i<=n;i++){
            x=read();    a[x].a++;    c[x]++;    d[i]=x;
        }
        len=262144;
        FFT(a,1);
        for (i=0;i<len;i++)    a[i]=(a[i]*a[i]);
        FFT(a,-1);
        for (i=1;i<=200000;i++)    s[i]=(LL)(a[i].a+0.5);
        for (i=1;i<=100000;i++)
            if (c[i])    s[2*i]-=c[i];
        for (i=1;i<=200000;i++)    s[i]/=2;
        //    for (i=0;i<=200000;i++)
        //        if (s[i])    printf("%d\n",s[i]);

        b[0]=0;
        for (i=1;i<=200000;i++)    b[i]=b[i-1]+s[i];
        Ans=0;
        sort(d+1,d+1+n);
        for (i=1;i<=n;i++){
            x=b[200000]-b[d[i]];
            x-=(n-1);
            x-=(i-1)*(n-i);
            x-=(n-i)*(n-i-1)/2;
            Ans+=x;
        }
        printf("%lld\n",Ans);
    //}
    return 0;
}

```

H

H 笛卡尔坐标系下的莫卡定理

时间限制：1000ms 内存限制：65536kb

通过率：16/52 (30.77%) 正确率：16/113 (14.16%)

题面

筹备校赛的过程中，*Bazoka13* 突发奇想——是不是任意的二维点集都可以被两条直线覆盖？他决定画一些图判断一下，虽然问题的答案显然是否，但是 *Bazoka13* 捏数据的能力太差了，画的 114514 组数据都能够被两条直线覆盖。他认为自己发现了一个神奇的定理，把它命名为“莫卡定理”并将它分享给了队友们，*JJLeo* 一眼就看出了“莫卡定理”的错误并随手捏了几个数据，但是还没有计算就被拉去筹备校赛了，于是他把这些数据给了你，请你帮他判断每组数据的点集能否被某两条直线覆盖。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>

```

```

#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define EPS 1e-5
#define N 200010
#define INF 0x3f3f3f3f
using namespace std;

inline double Min(double a,double b){return (a<b)?a:b;}
inline double Max(double a,double b){return (a>b)?a:b;}
inline double Fabs(double x){return (x<=-EPS)?-x:(x>EPS)?x:0;}
inline bool Equ(double x,double y){return (Fabs(x-y)<EPS);}
inline int Sgn(double x){
    if (Fabs(x)==0)    return 0;
    if (x>0)    return 1;    else return -1;
}

struct Vector{
    double x,y;
    void Clean(void){x=y=0;}
    void write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
    Vector operator +(Vector p){return (Vector){x+p.x,y+p.y};}
    Vector operator -(Vector p){return (Vector){x-p.x,y-p.y};}
    double operator *(Vector p){return x*p.y-y*p.x;}
    double operator &(Vector p){return x*p.x+y*p.y;}
};
typedef Vector V;

struct Point{
    double x,y;
    void Clean(void){x=y=0.0;}
    void write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
    bool operator ==(Point a){return (Equ(x-a.x,0) && Equ(y-a.y,0));}
    Point operator +(Point p){return (Point){x+p.x,y+p.y};}
    Point operator /(double l){return (Point){x/l,y/l};}
    Vector operator -(Point p){return (Vector){x-p.x,y-p.y};}
};
typedef Point P;

inline void SW(P &a,P &b){P t=a;    a=b;    b=t;}
inline double Dis2(P a,P b){return pow(a.x-b.x,2)+pow(a.y-b.y,2);}
inline double Dis(P a,P b){return pow(pow(a.x-b.x,2)+pow(a.y-b.y,2),0.5);}

inline bool Point_On_Line(P x,P y,P z){return (Fabs((x-y)*(x-z)) == 0);}

int n,i,j;
P a[N],b[N];

IL bool Check(int x, int y){
    reg int i=0,j=0,cnt=0;
    for (i=1;i<=n;i++)
        if (!Point_On_Line(a[i], a[x], a[y]))    b[++cnt]=a[i];
}

```

```

        for (i=1;i<=cnt;i++)
            if (!Point_On_Line(b[i], b[1], b[2]))    return 0;
        return 1;
    }

    int main(){
        #ifdef __Marvolo
        freopen("zht.in","r",stdin);
        freopen("zht.out","w",stdout);
        #endif
        scanf("%d", &n);
        for (i=1;i<=n;i++)    scanf("%lf%lf",&a[i].x,&a[i].y);
        if (n<=4){
            puts("YES");
            return 0;
        }
        if (Check(1,2) || Check(1,3) || Check(2,3)) puts("YES");
        else puts("NO");
        return 0;
    }

```

I 铺设信标

时间限制：1000ms 内存限制：65536kb

通过率：42/82 (51.22%) 正确率：42/227 (18.50%)

题目描述

提示非常重要，请认真阅读

铺设信标的区域是一个三角形区域，区域位于一个二维笛卡尔坐标系上。

信标只需要铺设在**区域内部**的所有**横纵坐标均为整数**的点上，即“格点”上，位于**区域边界**的**格点**不铺设信标。

为了减少负重，“我们”需要知道需要多少个信标。

```

#include<cstdio>
#include<iostream>
#include<cmath>
#include<algorithm>
#include<cstring>
#include<cstdlib>
#include<vector>
#include<queue>
#include<map>
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

LL T;

struct P{
    LL x,y;

```

```

}a,b,c;

IL LL Abs(LL x){return (x<0)?-x:x;}

IL LL read(){
    register LL p=0,f=1;    char c=getchar();
    while (c<'0' || c>'9'){if (c=='-') f=-1;    c=getchar();}
    while (c>='0' && c<='9'){p=p*10+c-48;    c=getchar();}
    return p*f;
}

IL LL Gcd(LL a,LL b){return (b==0)?a:Gcd(b,a%b);}

IL LL C(P a,P b,P c){
    LL sum=0;
    sum=Abs((b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-a.x))+2;
    sum-=Gcd(Abs(a.x-b.x),Abs(a.y-b.y));
    sum-=Gcd(Abs(a.x-c.x),Abs(a.y-c.y));
    sum-=Gcd(Abs(c.x-b.x),Abs(c.y-b.y));
    return sum/2;
}

int main(){
    //freopen("in.txt","r",stdin);
    //freopen("out.txt","w",stdout);
    T=read();
    while (T--){
        scanf("%lld%lld%lld%lld%lld%lld",&a.x,&a.y,&b.x,&b.y,&c.x,&c.y);
        printf("%lld\n",C(a,b,c));
    }
    return 0;
}

```

J

J 光与镜 - 青春版

时间限制：1000ms 内存限制：65536kb

通过率：21/49 (42.86%) 正确率：21/166 (12.65%)

题目描述

正如题目所说，这是个青春版的题目。

(或许需要重新看一下 [光与镜](#) 的题面)

Alex 觉得这个题太难了，因此他不打算问你^{能不能}经过点 K 。

他想知道这个点还能不能回到点 A 。

浮点数的输入也实在是太难了，因此 Alex 不打算给你输入坐标。

看到 $\angle ABO$ 了嘛，Alex 觉得把这个告诉你就够了。

似乎角度也是个浮点数，那他就把浮点数 $\frac{a}{b}$ 拆成两个整数 a, b 告诉你吧。

```
#include <bits/stdc++.h>

using namespace std;
using i64 = long long;

int main() {
    i64 T, a, b; cin >> T;
    while (T--) {
        cin >> a >> b; a = 90 * b - a;
        cout << 180 * b / __gcd(a, 180 * b) - 1 << endl;
    }
    return 0;
}
```

E1

A

A 瓷砖

时间限制: 1000ms 内存限制: 65536kb

通过率: / (%) 正确率: / (%)

题目描述

有一个宽度为 $2n$ ，长度为 n 的地板需要贴上瓷砖。有2种瓷砖可供选择：长度为1，宽度为2的瓷砖；长度为2，宽度为2的瓷砖。瓷砖可以随意旋转，但不能重叠，不能被切割，且必须恰好把地板铺满。

问一共有多少种铺瓷砖的方案，可以恰好把地板铺满。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define MOD 998244353
#define INF 0x3f3f3f3f
using namespace std;

int n,i;
LL f[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    n=read();
    f[0]=1; f[1]=1; f[2]=3;
    for (i=3;i<=n;i++){
        f[i]=f[i-1];
        f[i]=(f[i]+2*f[i-2])%MOD;
    }
}
```

```
cout<<f[n]<<endl;
return 0;
}
```

B

B 排列

时间限制：1000ms 内存限制：65536kb

通过率：237/245 (96.73%) 正确率：237/624 (37.98%)

题目描述

对于一个数列 $1, 2, 3, \dots, n$ ，它有 $n!$ 种排列方式。例如，当 $n = 3$ 时，有如下6种排列方式，按照从小到大的顺序依次为：

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

两个排列比大小的方式是，从头到尾依次比较两个排列的元素，如果相等则继续向后比较，直到其中一个排列的元素比另一个排列对应位置的元素小，则前者小于后者。

对于一个排列 A ，若 B 是它的下一个排列，当且仅当 $A < B$ 且不存在 C 满足 $A < C < B$ 。

现在，给出这 n 个正整数的一种排列，请输出它的下一个排列。

 PicGo

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

int n,i;
int a[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    n=read();
    for (i=1;i<=n;i++)    a[i]=read();
```



```
next_permutation(a+1,a+1+n);
for (i=1;i<=n;i++) printf("%d ",a[i]);
return 0;
}
```

C

C Ashane算日期 (hard version)

时间限制: 1000ms 内存限制: 65536kb

通过率: 232/244 (95.08%) 正确率: 232/1117 (20.77%)

题目描述

Ashane 喜欢用 8 位数字表示一个日期，其中前 4 位代表年，接下来 2 位代表月，最后 2 位代表日。

现在，他想知道指定两个日期之间（包含这两个日期本身），有多少个日期是幸运日期。

一个日期是幸运的当且仅当表示这个日期的 8 位数字是回文的。

例如：

- 2021年9月10日的数字表示是20210910，它不是回文的，所以这个日期不是幸运日期。
- 2021年12月2日的数字表示是20211202，它是回文的，所以这个日期是幸运日期。

```
#include <bits/stdc++.h>

using namespace std;

int MON[13] = {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

int calc(int M, int D) {
    int _t = M * 100 + D, Y = 0;
    for (int i = 0; i < 4; ++i) Y = Y * 10 + (_t % 10), _t /= 10;
    return Y * 10000 + M * 100 + D;
}

int main() {
    int T; cin >> T; while (T--) {
        int A, B, res = 0; scanf("%d%d", &A, &B);
        for (int i = 1; i <= 12; ++i) {
            for (int j = 1; j <= MON[i]; ++j) {
                int t = calc(i, j);
                if (min(A, B) <= t && t <= max(A, B))
                    res++;
            }
        }
        printf("%d\n", res);
    }
    return 0;
}
```

D

D Alex 和 Marvolo 不会出题了 - 1

时间限制：1000ms 内存限制：65536kb

通过率：220/225 (97.78%) 正确率：220/423 (52.01%)

题目描述

Alex 和 Marvolo 不会出题了。于是他们开始闲聊：

Marvolo: 要有一个木桶！ (于是天上掉下来一个木桶)

Alex: 木桶里面放点球吧！

Marvolo: 先扔他 100 个黑球进去！

Alex: 唔，那我扔 50 个白球！

Marvolo: (随手从桶里拿了两个球出来)

Alex: 好！我有了一个绝妙的想法，你先拿着别动

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    int T, b, w; scanf("%d", &T); while(T--) {
        scanf("%d%d", &b, &w);
        puts((b & 1) ? "white" : "Black");
    }
}
```

E

E Matrix53的数列 - Easy

时间限制：1000ms 内存限制：65536kb

通过率：187/209 (89.47%) 正确率：187/932 (20.06%)

题目描述

中秋节下围棋时，由于有高人相助，所以 Matrix53 赢了 Alex。

Alex 很不服气：“我能在很短的时间内统计出**数列中的三角形**的总数，你可以吗？”

Matrix53 仔细想了想，但还是对此毫无头绪，你能帮他解答出这道题吗？

数列中的三角形：若有三个数处于数列中三个不同的位置，且这三个数可以作为某一个三角形的三条边，则这三个数构成一个“数列中的三角形”

```
// Author: Matrix53
#include <bits/stdc++.h>

using namespace std;

int a[10005];

int main() {
    ios::sync_with_stdio(false);
    long long ans = 0;
    int n;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    sort(a, a + n);
```

```

for (int i = 2; i < n; ++i) {
    int index = lower_bound(a, a + i, a[i] / 2 + 1) - a;
    if (index == i) {
        continue;
    } else if (index == 0) {
        ans += i * (i - 1) / 2;
    } else {
        for (int l = index - 1, r = index; l >= 0; --l) {
            while (r < i && a[l] + a[r] <= a[i]) r++;
            if (r == i) break;
            ans += i - r;
        }
        ans += (i - index) * (i - index - 1) / 2;
    }
}
cout << ans;
return 0;
}

```

F

F 双陆棋

时间限制：1000ms 内存限制：65536kb

通过率：152/184 (82.61%) 正确率：152/793 (19.17%)

题目描述

为了缓解考试前的压力，学校为每位同学发放了一副解压双陆棋。

双陆棋的棋盘由 $N+1$ 个方格拍成一排，方格从左到右依次编号为 0 到 $N+1$ 。最开始的时候 0 号和 $N+1$ 号方格被标记为 $\times$ ，其它的方格被标记为 $\#$ 或 $\cdot$ 。

这个游戏最开始的时候一个棋子放在编号为 A 的位置，方向朝右，每一个单位时间，棋子都会往它所朝向的方向移动一格。

游戏规则如下：

- 若棋子移动之后所在的格子标记是 $\times$ ，会反转棋子的朝向。
- 若棋子移动之后所在的格子标记是 $\#$ ，会将该格子的标记改为 $\cdot$ 并反转棋子的朝向。
- 若棋子移动之后所在的格子标记是 $\cdot$ ，不会发生任何变化。

改变标记和敢转棋子朝向的时间可以忽略不计，当所有的 $\#$ 标记被清空，游戏结束。

现在，Marvolo想知道，给定了棋盘与初始位置后，在多久之后游戏会结束？

 PicGo

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define LL long long
#define N 200010
using namespace std;

int n,x,i,pos,cnt,w;
LL Ans;
char c[N];
int pre[N],nxt[N];

int main(){
    scanf("%d%d",&n,&x);

```

```

scanf("%s",c+1);
pos=0;
for (i=0;i<=n+1;i++){
    pre[i]=pos;
    if (c[i]=='#') pos=i,cnt++;
}
if (cnt==0){
    puts("0"); return 0;
}
pos=n+1;
for (i=n+1;i>=0;i--){
    nxt[i]=pos;
    if (c[i]=='#') pos=i;
}
// 仿照链表预处理出前后#的位置

w=1;
Ans+=abs(x-nxt[x]); x=nxt[x];
if (x == n+1){
    Ans+=abs(x-pre[x]); x=pre[x];
    w=-1;
}
// 从初始位置走到#上面

while (cnt){
    cnt--;
    if (!cnt) break;
    nxt[pre[x]] = nxt[x];
    pre[nxt[x]] = pre[x];
    // #变. 删除链表节点
    if (w==1){
        Ans+=abs(x-pre[x]);
        w=-1; x=pre[x];
        if (x == 0) Ans+=abs(x-nxt[x]),x=nxt[x],w=1;
    } else {
        Ans+=abs(x-nxt[x]);
        w=1; x=nxt[x];
        if (x == n+1) Ans+=abs(x-pre[x]),x=pre[x],w=-1;
    }
    // 根据方向直接跳转
}
cout<<Ans<<endl;
return 0;
}

```

G 四则运算

时间限制: 1000ms 内存限制: 65536kb

通过率: 139/154 (90.26%) 正确率: 139/430 (32.33%)

题目描述

Marvolo 和 Alex 经常在一起玩24点的游戏。他们发现, 对于任意的四个整数, 不一定能够恰好算出24点, 所以他们打算研究一下这四个整数最大能算出来多少。

Marvolo 每次都会给出四个正整数, 在不改变这四个数顺序的前提下, Alex 要在这四个正整数中间插入三个运算符, $(+, -, *, /)$; 同时, 他还可以插入任意数量的左右括号。

Marvolo 的要求是, 在1s的时间内, 回答出所有合法的表达式中最小的结果是多少。如果 Alex 答不上来, 就要被抓去写文档。

由于 Alex 还要忙着出题, 他找到了擅长编程的你, 希望你帮他解决这个问题。

注意1: 你不可以将 $+$ 和 $-$ 用于改变数的符号, 例如不可以将 $1\ 2\ 3\ 4$ 变为 $-1+2*(3+4)$, 而只能变为 $1+2*(3+4)$

注意2: 在计算表达式值时, 如果计算过程中出现了以0为除数的情况, 那么此表达式**非法**, 例如 $12+6/(3-3)$ 是一个非法的表达式, 因为计算过程中出现了 $6/0$

注意3: “除”指整数除法, 运算规则与C语言中的整数除法规则一致, 也就是说, 结果一定是整数

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int T,i,Ans;
int a[5];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

vector <int> v;

IL void Dfs(int a,int b,int c,int d,int len){
    if (len==1){
        Ans=Min(Ans,a);
        return;
    }
    if (len==2){
        Dfs(a+b,0,0,0,1);
        Dfs(a-b,0,0,0,1);
        Dfs(a*b,0,0,0,1);
        if (b) Dfs(a/b,0,0,0,1);
        return;
    }
```

```

    }
    if (len==3){
        Dfs(a+b,c,0,0,2);
        Dfs(a-b,c,0,0,2);
        Dfs(a*b,c,0,0,2);
        if (b) Dfs(a/b,c,0,0,2);
        Dfs(a,b+c,0,0,2);
        Dfs(a,b-c,0,0,2);
        Dfs(a,b*c,0,0,2);
        if (c) Dfs(a,b/c,0,0,2);
        return;
    }
    Dfs(a+b,c,d,0,3);
    Dfs(a-b,c,d,0,3);
    Dfs(a*b,c,d,0,3);
    if (b) Dfs(a/b,c,d,0,3);
    Dfs(a,b+c,d,0,3);
    Dfs(a,b-c,d,0,3);
    Dfs(a,b*c,d,0,3);
    if (c) Dfs(a,b/c,d,0,3);
    Dfs(a,b,c+d,0,3);
    Dfs(a,b,c-d,0,3);
    Dfs(a,b,c*d,0,3);
    if (d) Dfs(a,b,c/d,0,3);
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    T=read();
    while (T--){
        Ans=INF;
        for (i=1;i<=4;i++) scanf("%d",&a[i]);
        for (i=1;i<=4;i++) v.push_back(a[i]);
        Dfs(a[1],a[2],a[3],a[4],4);
        printf("%d\n",Ans);
    }
    return 0;
}

```



模拟只会猜题意

时间限制: 1000ms 内存限制: 65536kb

通过率: 139/154 (90.26%) 正确率: 139/416 (33.41%)

题目描述

Alex 的英语很不好, 他从来没做过英语语法题。为了考过四六级, 他决定好好学英语。

他写下了一些句子, 但是不知到这些句子是否符合英语语法, 希望你能告诉他。

英语的语法较为复杂, 我们这里仅考虑简化版本:

- 一个句子由**主语加谓语**组成
- 主语是一个**名词短语**
- 名词短语可以是**单一的名词**, 也可以在名词前加**任意多个形容词**
- 谓语由**动词加直接宾语**组成
- 直接宾语由**冠词加名词短语**组成
- 形容词仅考虑: big, brown, roasted
- 名词仅考虑: John, peanut
- 动词仅考虑: ate
- 冠词仅考虑: the

按照上述规则, 我们可以认为: John ate the peanut, John ate the big brown peanut 都是一个语法正确的句子。

```
#include <bits/stdc++.h>

using namespace std;

bool noun(string *s, int l, int r);
bool adjective(string *s, int l, int r);
bool verb(string *s, int l, int r);
bool article(string *s, int l, int r);
bool directObject(string *s, int l, int r);
bool nounPhrase(string *s, int l, int r);
bool predicate(string *s, int l, int r);
bool subject(string *s, int l, int r);
bool sentence(string *s, int l, int r);

int main() {
    int cnt = 0; string s[110];
    while(cin >> s[cnt + 1])
        cnt++;
    puts(sentence(s, 1, cnt) ? "Yes" : "No");
    return 0;
}

bool noun(string *s, int l, int r) {
    if (l > r) return false;
    if (l != r) return false;
    if (s[l] == "John") return true;
    if (s[l] == "peanut") return true;
    return false;
}

bool adjective(string *s, int l, int r) {
    if (l > r) return false;
    if (l != r) return false;
    if (s[l] == "brown") return true;
    if (s[l] == "big") return true;
    if (s[l] == "roasted") return true;
    return false;
}
```

```

}

bool verb(string *s, int l, int r) {
    if (l > r) return false;
    if (l != r) return false;
    if (s[l] == "ate") return true;
    return false;
}

bool article(string *s, int l, int r) {
    if (l > r) return false;
    if (l != r) return false;
    if (s[l] == "the") return true;
    return false;
}

bool directObject(string *s, int l, int r) {
    if (l > r) return false;
    return article(s, l, l) && nounPhrase(s, l + 1, r);
}

bool nounPhrase(string *s, int l, int r) {
    if (l > r) return false;
    if (noun(s, l, r)) return true;
    return adjective(s, l, l) && nounPhrase(s, l + 1, r);
}

bool predicate(string *s, int l, int r) {
    if (l > r) return false;
    return verb(s, l, l) && directObject(s, l + 1, r);
}

bool subject(string *s, int l, int r) {
    if (l > r) return false;
    return nounPhrase(s, l, r);
}

bool sentence(string *s, int l, int r) {
    if (l > r) return false;
    for (int mid = l; mid < r; ++mid)
        if (subject(s, l, mid) && predicate(s, mid + 1, r))
            return true;
    return false;
}

```


I 贴瓷砖

时间限制: 1000ms 内存限制: 65536kb

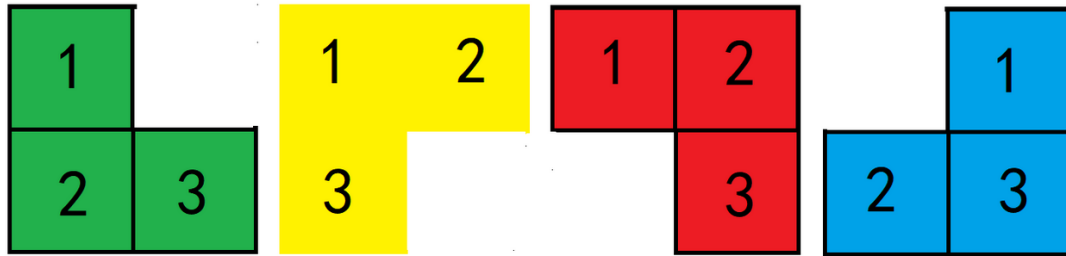
通过率: 103/117 (88.03%) 正确率: 103/422 (24.41%)

题目描述

你是一个装修公司的CEO, 今天你接到了一单生意, 让你在月球上贴瓷砖, 客户给出的价格不菲, 并且愿意承担来往的费用, 你很心动, 但是客户提出了一些要求, 你的任务是给出满足客户要求的方案。

客户主要的要求如下:

- 只能使用如下的瓷砖



- 要布满一个边长为 n 的矩形区域, 除了其中一个用来放置人类月球纪念碑的位置。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 2050
#define INF 0x3f3f3f3f
using namespace std;

int n,tot,i,j,x,y;
int a[N][N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL void Fill(int x,int y,int l,int p,int q){
    if (l==2){
        tot++;
        a[x][y]=tot;    a[x+1][y]=tot;
        a[x][y+1]=tot;    a[x+1][y+1]=tot;
        a[p][q]=0;
        return;
    }
}
```

```

if (p<=x+(l>>1)-1 && q<=y+(l>>1)-1){
    Fill(x,y,l>>1,p,q);
    Fill(x+(l>>1),y,l>>1,x+(l>>1),y+(l>>1)-1);
    Fill(x,y+(l>>1),l>>1,x+(l>>1)-1,y+(l>>1));
    Fill(x+(l>>1),y+(l>>1),l>>1,x+(l>>1),y+(l>>1));
    tot++;
    a[x+(l>>1)][y+(l>>1)-1]=tot;
    a[x+(l>>1)-1][y+(l>>1)]=tot;
    a[x+(l>>1)][y+(l>>1)]=tot;
} else if (p>=x+(l>>1) && q<=y+(l>>1)-1){
    Fill(x,y,l>>1,x+(l>>1)-1,y+(l>>1)-1);
    Fill(x+(l>>1),y,l>>1,p,q);
    Fill(x,y+(l>>1),l>>1,x+(l>>1)-1,y+(l>>1));
    Fill(x+(l>>1),y+(l>>1),l>>1,x+(l>>1),y+(l>>1));
    tot++;
    a[x+(l>>1)-1][y+(l>>1)-1]=tot;
    a[x+(l>>1)-1][y+(l>>1)]=tot;
    a[x+(l>>1)][y+(l>>1)]=tot;
} else if (p<=x+(l>>1)-1 && q>=y+(l>>1)){
    Fill(x,y,l>>1,x+(l>>1)-1,y+(l>>1)-1);
    Fill(x+(l>>1),y,l>>1,x+(l>>1),y+(l>>1)-1);
    Fill(x,y+(l>>1),l>>1,p,q);
    Fill(x+(l>>1),y+(l>>1),l>>1,x+(l>>1),y+(l>>1));
    tot++;
    a[x+(l>>1)-1][y+(l>>1)-1]=tot;
    a[x+(l>>1)][y+(l>>1)-1]=tot;
    a[x+(l>>1)][y+(l>>1)]=tot;
} else {
    Fill(x,y,l>>1,x+(l>>1)-1,y+(l>>1)-1);
    Fill(x+(l>>1),y,l>>1,x+(l>>1),y+(l>>1)-1);
    Fill(x,y+(l>>1),l>>1,x+(l>>1)-1,y+(l>>1));
    Fill(x+(l>>1),y+(l>>1),l>>1,p,q);
    tot++;
    a[x+(l>>1)-1][y+(l>>1)-1]=tot;
    a[x+(l>>1)][y+(l>>1)-1]=tot;
    a[x+(l>>1)-1][y+(l>>1)]=tot;
}
}

int main(){
#ifdef __Marvolo
freopen("zht.in","r",stdin);
freopen("zht.out","w",stdout);
#endif
n=read();
y=read()+1; x=read()+1;
if (n==1) return 0;
Fill(1,1,n,x,y);
for (i=1;i<=n;i++)
    for (j=1;j<=n;j++){
        if (!a[i][j]) continue;
        if (a[i][j]==a[i+1][j] && a[i][j]==a[i][j+1])
            printf("y (%d,%d) (%d,%d) (%d,%d)\n",j-1,i-1,j,i-1,j-1,i);

        if (a[i][j]==a[i+1][j] && a[i][j]==a[i+1][j+1])
            printf("g (%d,%d) (%d,%d) (%d,%d)\n",j-1,i-1,j-1,i,j,i);

        if (a[i][j]==a[i][j+1] && a[i][j]==a[i+1][j+1])

```

```

        printf("r (%d,%d) (%d,%d) (%d,%d)\n",j-1,i-1,j,i-1,j,i);

        if (a[i][j]==a[i+1][j-1] && a[i][j]==a[i+1][j])
            printf("b (%d,%d) (%d,%d) (%d,%d)\n",j-1,i-1,j-2,i,j-1,i);
    }
    /*for (i=1;i<=n;i++){
        for (j=1;j<=n;j++){
            printf("%5d ",a[i][j]);
            //cout<<a[i][j]<<" ";
        }
        cout<<endl;
    }*/
    return 0;
}

```

J

J 密切接触者

时间限制：1000ms 内存限制：65536kb

通过率：54/91 (59.34%) 正确率：54/648 (8.33%)

题目描述

你想为遏制疫情蔓延做些贡献吗？我们的医护工作者在统计密切接触者时遇到了难题，现在就请你来帮帮他们吧。

情景如下。

一共有 n 个人，假设所有人沿着一水平数轴做匀速直线运动，每个人的行动轨迹通过起始位置和移动速度描述，正数表示正方向，负数表示负方向。

当两个人的距离小于等于某个给定值时，这两个人存在密切接触关系。

假设时间可以无限长，请你给出所有潜在的密切接触关系的数量。

```

#include<bits/stdc++.h>
#define N 100006
#define ll long long
using namespace std;
int n,d,x[N],s[N];
ll ans;
struct person
{
    int x,v;
}p[N];
int cmp(person a,person b)
{
    if(a.v==b.v)
        return a.x<b.x;
    return a.v<b.v;
}
void mergesort(int l,int r)
{
    if(l==r) return ;
    int m=(l+r)/2;
    mergesort(l,m);
    mergesort(m+1,r);
    int k=l-1;
    for(int i=m+1;i<=r;i++)
    {

```

```

        while(k+1<=m&&x[k+1]+d<x[i]) k++;
        ans+=m-k;
    }
    int i=1,j=m+1;
    k=1;
    while(k<=r)
    {
        if(i>m) s[k]=x[j++];
        else if(j>r) s[k]=x[i++];
        else if(x[i]<=x[j]) s[k]=x[i++];
        else s[k]=x[j++];
        k++;
    }
    for(i=1;i<=r;i++)
        x[i]=s[i];
}
int main()
{
    scanf("%d%d",&n,&d);
    for(int i=0;i<n;i++)
        scanf("%d%d",&p[i].x,&p[i].v);
    sort(p,p+n,cmp);
    for(int i=0;i<n;i++)
    {
        x[i]=p[i].x;
    }
    mergesort(0,n-1);
    printf("%lld",ans);
}

```

A 海拉鲁王国的勇士

时间限制：1000ms 内存限制：65536kb

通过率：241/241 (100.00%) 正确率：241/429 (56.18%)

题目描述

为夺回被恶龙抢走的城堡，海拉鲁王国的国王罗姆召集了 n 位勇士，每一位勇士有一个力量值 a_i 。国王将会从他们中间，挑选出一批人组成小队，潜入城堡去和恶龙战斗。

已知恶龙的力量值为 s ，而想要打败恶龙，必须保证派出的勇士的力量值之和**大于等于**恶龙的力量值。为了保证勇士们在潜入城堡时不会暴露，国王希望派出的勇士数量尽可能的少。请你帮国王计算一下，最少派出几位勇士，就可以打败恶龙了呢？

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 200010
#define INF 0x3f3f3f3f
using namespace std;

LL n,i,s,x,y,T,att,def;
LL a[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL LL read(){
    LL p=0,f=1; char c=getchar();
    while (c<48||c>57) {if (c=='-') f=-1; c=getchar();}
    while (c>=48&&c<=57) p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    n=read(); x=read();
    for (i=1;i<=n;i++) a[i]=read();
    sort(a+1,a+1+n);
    reverse(a+1,a+1+n);
    for (i=1;i<=n;i++){
        s+=a[i];
```

```

        if (s>=x){
            cout<<i<<endl;
            return 0;
        }
    }
    puts("Lose");
    return 0;
}

```

B

B 魔术球

时间限制：1000ms 内存限制：65536kb

通过率：240/240 (100.00%) 正确率：240/414 (57.97%)

题目描述

Marvolo 是一位魔术师。众所周知，魔术师的道具都是很神奇的，Marvolo 也不例外，他有一个神奇的魔术球和一个配套的魔术桶。

在表演魔术的时候，Marvolo 会先让观众随便说一个正整数 x ，然后他就会开始表演。首先，他会展示魔术桶，桶里一开始什么都没有；然后，他会在上面盖一层布，遮住桶的开口。接下来，他会在上面施展一定次数的魔法（没错，魔术师其实都会魔法的），每施展一次，他可以让桶里魔术球的数量加一，或者翻倍。施展了一定次数的魔法后，Marvolo 就会揭开布，让观众数一下桶里球的数量，观众会发现恰好是 x 个，魔术到这里就结束了。

为了确保节目效果，Marvolo 希望施展魔术的时间尽可能短一点，省得观众等的不耐烦要求退钱。请你帮他规划一下，最少需要施展几次魔法，就可以让魔术桶里的魔术球恰好有 x 个呢？

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 2000010
#define INF 0x3f3f3f3f
using namespace std;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL LL read(){
    LL p=0,f=1; char c=getchar();
    while (c<48||c>57) {if (c=='-') f=-1; c=getchar();}
    while (c>=48&&c<=57) p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL int Dfs(LL x){
    if (x==1) return 1;
    if (x&1) return Dfs(x-1)+1;
}

```

```

        else return Dfs(x>>1)+1;
    }

    int main(){
        LL n = read();
        cout<<Dfs(n)<<endl;
        return 0;
    }

```

C

C 数矩形

时间限制：1000ms 内存限制：65536kb

通过率：192/209 (91.87%) 正确率：192/482 (39.83%)

题目描述

在二维平面上散乱地分布着 n 个坐标点，编号从1到 n 。第 i 个坐标点的坐标是 (x_i, y_i) 。

Marvolo 想知道，如果从这些点中选4个点出来，能够构成多少种不同的矩形呢？（两个矩形不同当且仅当他们有至少一个顶点不同）。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define mk make_pair
#define IL inline
#define reg register
#define LL long long
#define N 2010
#define INF 0x3f3f3f3f
using namespace std;

int n,i,j,Ans;

struct Data{
    int x,y;
}a[N];

map <pair<int,int>, bool> M;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57)  {if (c=='-')    f=-1;    c=getchar();}

```

```

while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
return p*f;
}

int main(){
    n=read();
    for (i=1;i<=n;i++){
        a[i].x=read(); a[i].y=read();
        M[mk(a[i].x, a[i].y)] = 1;
    }
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++){
            if (i == j) continue;
            if (a[j].x<=a[i].x) continue;
            if (a[j].y<=a[i].y) continue;
            if (!M[mk(a[i].x, a[j].y)]) continue;
            if (!M[mk(a[j].x, a[i].y)]) continue;
            Ans++;
        }
    cout<<Ans<<endl;
    return 0;
}

```

D

D 非降序排列

时间限制：1000ms 内存限制：65536kb

通过率：93/136 (68.38%) 正确率：93/533 (17.45%)

题目描述

给定一个序列，求最多可以分成多少个连续的组，使对每个组组内非降序排列后，整个序列仍是非降序的。

输入格式

第一行包含两个整数 $n(1 \leq n \leq 10^6)$ ，表示序列的数量。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n(1 \leq a_i \leq 10^9)$ ，代表这个序列。

```

#include<bits/stdc++.h>
using namespace std;
const int N = 1e6 + 10;
int n, a[N];
int maxx[N], minn[N];
int main() {
    scanf("%d", &n);
    for(int i = 1; i <= n; i++)
        scanf("%d", &a[i]);
    minn[n] = a[n];
    for(int i = 1; i <= n; i++)
        maxx[i] = max(maxx[i - 1], a[i]);
    for(int i = n - 1; i >= 1; i--)
        minn[i] = min(minn[i + 1], a[i]);
    int ans = 1;
    for(int i = 1; i < n; i++)

```



```
        if(maxx[i] <= minn[i + 1])
            ans ++;
    printf("%d", ans);
}
```

E

E 海拉鲁王国的勇士2

时间限制：1000ms 内存限制：65536kb

通过率：163/208 (78.37%) 正确率：163/715 (22.80%)

题目描述

本题的题目背景详见A题。

经过观察，国王发现，恶龙的力量值并不是一成不变的，例如早上时恶龙的力量值最低，深夜恶龙的力量值最高。因此，国王打算挑选一个合适的时机再派出勇士前去挑战恶龙。

国王经过观测，得到了 m 个不同的时刻下恶龙的力量值数据。现在，他想知道，对于每一个时刻，最少需要派出几位勇士才可以打败恶龙。如果派出所有勇士都无法打败，输出 `Lose`。

```
#include <bits/stdc++.h>

using namespace std;
typedef long long ll;

ll n, m, s, a[100010];

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    sort(a + 1, a + 1 + n);
    reverse(a + 1, a + 1 + n);
    for (int i = 1; i <= n; ++i) a[i] += a[i - 1];
    while (m--) {
        cin >> s;
        ll ans = (lower_bound(a + 1, a + 1 + n, s) - a);
        if (ans > n) puts("Lose");
        else cout << ans << endl;
    }
    return 0;
}
```

F

F 神秘的盒子

时间限制：1000ms 内存限制：65536kb

通过率：47/59 (79.66%) 正确率：47/193 (24.35%)

题目描述

你得到了一个神秘的盒子，可是盒子被上了锁，想打开盒子必须解开这个锁。

你看到这个锁上有 $n \times m$ 个方块，每个方块可以显示 1 和 0 两个数字。

方块是可按动的，每按动一个方块，这个方块显示的数字就会改变，同时这个方块上下左右相邻的方块显示的数字也会相应改变。

以 5×5 为例，比如初始状态如下：

```
10101
01010
10101
01010
10101
```

```
#include<bits/stdc++.h>
using namespace std;
const int INF = 1e9;
int n, m;
char g[20][20];
char backup[20][20];
int dx[5] = {0, -1, 0, 1, 0}, dy[5] = {0, 0, 1, 0, -1};
void turn(int x, int y){
    for(int i = 0; i < 5; i++){
        int a = x + dx[i], b = y + dy[i];
        if(a >= 0 && a < n && b >= 0 && b < m){
            g[a][b] ^= 1;
        }
    }
}
int work(){
    int ans = INF, hasAns = 0;
    for(int k = 0; k < 1 << m; k++){
        int res = 0;
        memcpy(g, backup, sizeof(g));
        for(int j = 0; j < m; j++){
            if(k >> j & 1){
                res ++;
                turn(0, j);
            }
        }
        for(int i = 0; i < n - 1; i++){
            for(int j = 0; j < m; j++){
                if(g[i][j] == '0'){
                    res++;
                    turn(i + 1, j);
                }
            }
        }
        bool fl = 1;
        for(int j = 0; j < m; j++){
            if(g[n - 1][j] == '0'){
                fl = 0;
                break;
            }
        }
    }
}
```

```

    }
    hasAns |= f1;
    if(f1) ans = min(ans, res);
}
if(hasAns) return ans;
return -1;
}
int main(){
    scanf("%d%d", &n, &m);
    for(int i = 0 ; i < n; i++) scanf("%s", backup[i]);
    printf("%d", work());
}

```

G

G Alex 和 Marvolo 不会出题了 - 2

时间限制：1000ms 内存限制：65536kb

通过率：139/146 (95.21%) 正确率：139/265 (52.45%)

题目描述

Alex 和 Marvolo 不会出题了。于是他们开始闲聊：

Alex: 听说了吗，C2 只有两个小时做题了。

Marvolo: 就应该这样做 (x

Alex: 两个小时 10 个题，做得完吗。

Marvolo: 不太容易。

Alex: 我有了一个精妙的想法。

Marvolo: 巧了我也有。

Alex & Marvolo: 不如把难度降低一点吧（出题压力--）。

Marvolo: 出一点大家都学过的。

Alex: 高等代数！

Marvolo: ? ? ?

```

// Author: Alex
#include <bits/stdc++.h>

using namespace std;

int main() {
    int n, m, T; cin >> T; while (T--) {
        cin >> n >> m;
        for (int i = 0, x; i < n; ++i) cin >> x;
        for (int i = 0, x; i < m; ++i) cin >> x;
        puts(n == m ? "Draw" : n > m ? "Alex" : "Marvolo");
    }
    return 0;
}

```

H

H Alex 和 Marvolo 造题的 4 秒钟

时间限制: 1000ms 内存限制: 65536kb

通过率: 34/56 (60.71%) 正确率: 34/297 (11.45%)

题目描述

Alex 和 Marvolo 不会出题了。于是他们开始闲聊:

Marvolo: 要有书! (于是填上掉下来一堆课本)

Alex: 要有分类! (于是课本分为了 n 类)

Marvolo: 要精简! (于是每一类书都只剩下来了一本)

Alex: 要更精简! (于是 he 可以将不同类的课本进行合并了)

具体而言, 书本将会按照如下规则进行合并:

- 初始状态下, 每本书各自为营, 合并后的书属于同一类别。用一个队列表示。
- 如果 Alex 希望合并 x, y 两本书
 - Alex 会分别找到 x, y 所属的队列, 假设他们已经有图书 $[x_1, x_2, \dots, x_p], [y_1, y_2, \dots, y_q]$
 - Alex 会将两套书交叉合并, 如果有某一类有剩余的, 则会直接放于队尾, 具体的, 若 $p < q$, 则新的图书序列为 $[x_1, y_1, x_2, y_2, \dots, x_p, y_p, y_{p+1}, \dots, y_q]$, 反之亦然。
 - 当然, 如果 x, y 在合并之前已经属于同一队列了, 那么 Alex 不会做任何修改。

初始状态下总共有 n 本书, Alex 想知道他在操作 m 次之后, 不同的书所属的类别以及摆放的顺序

```
// Author: Alex
#include <bits/stdc++.h>
using namespace std;

const int N = 3e5 + 100;

int n, m, fa[N], op, x, y;
int find(int x) {
    int p = x; while (p != fa[p]) p = fa[p];
    while (x != p) {
        int t = fa[x]; fa[x] = p; x = t;
    }
    return p;
}
struct List {
    int nxt;
} L[N];

int main() {
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; ++i)
        L[i].nxt = 0,
        fa[i] = i;
    for (int i = 1; i <= m; ++i) {
        scanf ("%d%d", &x, &y);
        x = find(x); y = find(y); if (x == y) continue;
        fa[y] = x;
        while (x && y) {
            int nx = L[x].nxt;
            L[x].nxt = y; x = nx; swap(x, y);
        }
    }
    for (int i = 1; i <= n; ++i) if (i == find(i)) {
        int now = i;
        while (now) {
            printf("%d ", now);
            now = L[now].nxt;
        }
        puts("");
    }
```

```
}  
return 0;  
}
```

I Matrix53的数列 - Medium

时间限制：1000ms 内存限制：65536kb

通过率：18/45 (40.00%) 正确率：18/171 (10.53%)

题目描述

且说道那 Matrix53 不仅下棋胜了 Alex，还顺利地在规定时间内统计出了数列中的三角形的总数，扬名四方。

数列大师 Marvolo 听说了 Matrix53 的事迹，于是决定和他共同研究一个数列问题，他给 Matrix53 发了一封信：

Matrix53 亲启：

我是数列研究者 Marvolo，近期听闻你在很短的时间内计算出了数列中的三角形的总数。

我也在研究一个类似的问题，希望能够与您讨论一下，问题是这样的——

已知数列 $a_1, a_2, a_3 \dots a_n$ ，试统计同时满足下列条件的三元组 (i, j, k) 的总数：

1. $1 \leq i < j < k \leq n$
2. $a_i \leq a_j \leq a_k$
3. a_i, a_j, a_k 可构成某一三角形的三边

书不尽意，余容后叙。

来自：Marvolo

```
// Author: Matrix53  
#include <bits/stdc++.h>  
  
using namespace std;  
  
const int N = 3505;  
  
int a[N], b[N], c[N];  
int pa, pb, pc;  
  
int main() {  
    long long ans = 0;  
    scanf("%d", &pa);  
    for (int i = 0; i < pa; ++i) scanf("%d", &a[i]);  
    for (int j = 1; j < pa - 1; ++j) {  
        pc = pb = 0;  
        for (int i = 0; i < j; ++i) {  
            if (a[i] <= a[j]) b[pb++] = a[i];  
        }  
        for (int k = j + 1; k < pa; ++k) {  
            if (a[j] <= a[k]) c[pc++] = a[k];  
        }  
        if (pb == 0 || pc == 0) continue;  
        sort(b, b + pb);  
        sort(c, c + pc);  
        int i = 0, k = 0;  
        while (i < pb && b[i] + a[j] <= c[k]) i++;  
        while (i < pb) {  
            while (k < pc - 1 && b[i] + a[j] > c[k + 1]) k++;  
            ans += k + 1;  
            i++;  
        }  
    }  
    printf("%lld", ans);  
}
```

```

    }
}
printf("%lld", ans);
return 0;
}

```

J

J 考古

时间限制：1000ms 内存限制：65536kb

通过率：10/19 (52.63%) 正确率：10/137 (7.30%)

题目描述

Marvolo 是一个考古学家。这天，他来到了一个文明古城遗址，这个遗址非常的大，可以视为一个二维坐标平面。在遗址下的一些地方埋藏的有文物，由于文物体积相对遗址很小，所以可以视为坐标平面上的一个点。

通过一个神奇的探测工具，Marvolo已经探明了地下一共有 m 件文物，第 i 件的考古价值为 s_i ，坐标为 (x_i, y_i) 。由于国庆假期短暂，Marvolo并不能把所有的文物都带回研究所研究，他只能选择一个小区间，然后将这个小区间中的所有文物带走（含边界）。

由于Marvolo忙着出土文物，他找到了擅长编程的你，并给出了他预设好的 m 种考古方案，希望你帮忙计算一下，每一种考古方案中，带走的文物的总考古价值为多少。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<map>
#include<unordered_map>
#define low(x) (x&(-x))
#define mk make_pair
#define IL inline
#define reg register
#define LL long long
#define N 100010
using namespace std;

int n,m,i,tot,cnt;
LL t[N*6];
int l[N*6];
vector<pair<int,int>> v[N*6],V[N*6];
unordered_map<int,int> M;

struct Data{
    int x,y,p;
}a[N];

struct Data2{
    int x1,y1,x2,y2;
    LL ans;
}b[N];

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
}

```

```

        return p*f;
    }

    IL void Add(int x,int val){
        reg int i=0;
        for (i=x;i<=tot;i+=low(i)) t[i]+=val;
    }

    IL LL Query(int x){
        reg int i=0;
        reg LL Ans=0;
        for (i=x;i-=low(i)) Ans+=t[i];
        return Ans;
    }

    IL void Ready(){
        reg int i=0;
        sort(l+1,l+1+tot);
        for (i=1;i<=tot;i++){
            if (M.count(l[i])) continue;
            M[l[i]] = ++cnt;
        }
        for (i=1;i<=n;i++){
            a[i].x=M[a[i].x]; a[i].y=M[a[i].y];
            v[a[i].x].push_back(mk(a[i].y, a[i].p));
        }
        for (i=1;i<=m;i++){
            b[i].x1=M[b[i].x1]; b[i].x2=M[b[i].x2];
            b[i].y1=M[b[i].y1]; b[i].y2=M[b[i].y2];
            v[b[i].x1].push_back(mk(i,-1));
            v[b[i].x2].push_back(mk(i,1));
        }
    }

    IL void Work(){
        reg int i=0,j=0;
        for (i=1;i<=cnt;i++){
            for (auto j : v[i]) Add(j.first,j.second);
            for (auto j : v[i]){
                auto k = b[j.first];
                b[j.first].ans+=(LL)j.second*(Query(k.y2)-Query(k.y1));
            }
        }
        for (i=1;i<=m;i++) printf("%lld\n",b[i].ans);
    }

    int main(){
        #ifdef __Marvolo
        freopen("zht.in","r",stdin);
        freopen("zht.out","w",stdout);
        #endif
        n=read(); m=read();
        for (i=1;i<=n;i++){
            a[i].x=read(); a[i].y=read(); a[i].p=read();
            l[++tot]=a[i].x; l[++tot]=a[i].y;
        }
        for (i=1;i<=m;i++){
            b[i].x1=read()-1; b[i].y1=read()-1;

```

```
        b[i].x2=read();    b[i].y2=read();
        l[++tot]=b[i].x1;  l[++tot]=b[i].x2;
        l[++tot]=b[i].y1;  l[++tot]=b[i].y2;
    }
    Ready();    work();
    return 0;
}
```


E3

A

A 堆-初见

时间限制: 1000ms 内存限制: 65536kb

通过率: 199/207 (96.14%) 正确率: 199/610 (32.62%)

题目描述

堆是一种重要的数据结构，它可以在 $O(\log n)$ 的时间内，维护一些元素的最大值或最小值。堆可以分为大根堆和小根堆，在这道题中你的任务就是实现一个小根堆。

本题禁用C++

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int n,x,op;
priority_queue <int> Q;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    n=read();
    while (n--){
        op=read();
        if (op == 1){
            x=read();    Q.push(-x);
        } else if (op == 2)
            Q.pop();
        else
            printf("%d\n",-Q.top());
    }
}
```

```

    }
    while (!Q.empty()){
        printf("%d ",-Q.top());
        Q.pop();
    }
    return 0;
}

```

B

B Is a heap?

时间限制：1000ms 内存限制：65536kb

通过率：199/204 (97.55%) 正确率：199/385 (51.69%)

题目描述

在算法导论以及网上的各类实现堆的代码中，他们都选择使用一维数组去模拟堆的结构。如果在堆构建好之后，将这个一维数组输出，可以观察到他们有一些有趣的性质（详见算导P85）。我们将小根堆构建完成后所对的数列，称为小根堆数列，大根堆构建完成后所对的数列称为大根堆数列。

现在，给出一个长度为 n 的数列，请你判断它是否可以作为一个小根堆或者大根堆的数列。如果可以是小根堆数列，输出 `Min heap`；如果可以是大根堆数列，输出 `Max heap`；如果不可能小根堆也不可能大根堆，输出 `Not a heap!`。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

int T,n,i,x;
int a[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    T=read();
    while (T--){

```

```

n=read();
for (i=1;i<=n;i++) a[i]=read();
x=0;
for (i=1;i<=n;i++){
    if (l(i) > n) break;
    if (a[l(i)]>a[i]){
        if (x>=0) x=1;
        else {
            puts("Not a heap!");
            goto END;
        }
    }
    if (a[l(i)]<a[i]){
        if (x<=0) x=-1;
        else {
            puts("Not a heap!");
            goto END;
        }
    }
}

if (r(i) > n) break;
if (a[r(i)]>a[i]){
    if (x>=0) x=1;
    else {
        puts("Not a heap!");
        goto END;
    }
}
if (a[r(i)]<a[i]){
    if (x<=0) x=-1;
    else {
        puts("Not a heap!");
        goto END;
    }
}
}
if (x==1) puts("Min heap");
else puts("Max heap");
END;;
}
return 0;
}

```

题目描述

一辆名叫漫游者的宇宙探索车出现了故障。为了恢复其性能，需要为其充电。

漫游者的电量是由一个正整数给出的，目前的电池电量等于 a ，为了使其能够重新运行起来，应该将其充电至 b 。

漫游者自带的充电模式有两种操作 X 和 Y，执行 X 操作可以使当前的电池电量增加 1；执行 Y 操作可以使当前的电池电量增加 2。但是由于漫游者的特殊设计，若在充电过程中电池电量恰巧是 c 的倍数，那么电池电量会瞬间变为 0。

为了尽快让漫游者重新运行起来，你想要通过最少的操作使其充电到 b ，请计算最少的操作次数。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
using namespace std;

int a,b,c,ans,up,down;

int main(){
    scanf("%d%d%d",&a,&b,&c);
    if (a/c == b/c){
        cout<<(b-a+1)/2<<endl;
        return 0;
    }
    // 没有c的倍数，特判

    down = (a+c-1)/c*c;
    up = b/c*c;
    ans = (down-a)/2 + (b-up)/2 + 1;
    // 第一部分

    if (c%2 == 0){
        ans += (up-down)/2;
    } else {
        ans += (up-down)/c * (c+1)/2;
    }
    // 分奇偶讨论
    cout<<ans<<endl;
    return 0;
}
```

D 滑动窗口

时间限制：1000ms 内存限制：65536kb

通过率：138/165 (83.64%) 正确率：138/403 (34.24%)

题目描述

给定一个大小为 n 的数组。有一个大小为 k 的滑动窗口，它从数组的最左边移动到最右边。

你只能在窗口中看到 k 个数字，每次滑动窗口向右移动一个位置。

以下是一个例子：

该数组为 [1 3 -1 -3 5 3 6 7]， k 为 3。

窗口位置	最小值	最大值
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

```
// Author: Serein
#include<bits/stdc++.h>
using namespace std;

const int N = 2000010;

int n, k;
int a[N], q[N];

int main(){
    scanf("%d%d", &n, &k);
    for(int i = 0; i < n; i++) scanf("%d", &a[i]);
    int hh = 0, tt = -1;
    for(int i = 0; i < n; i++){
        if(hh <= tt && i - k + 1 > q[hh]) hh++;
        while(hh <= tt && a[q[tt]] >= a[i]) tt--;
        q[++tt] = i;
        if(i >= k - 1) printf("%d ", a[q[hh]]);
    }
    puts("");
    hh = 0, tt = -1;
    for(int i = 0; i < n; i++){
        if(hh <= tt && i - k + 1 > q[hh]) hh++;
        while(hh <= tt && a[q[tt]] <= a[i]) tt--;
        q[++tt] = i;
        if(i >= k - 1) printf("%d ", a[q[hh]]);
    }
}
```

E 加赛

时间限制: 1000ms 内存限制: 65536kb

通过率: 101/122 (82.79%) 正确率: 101/245 (41.22%)

题目描述

Buttersky和Serein正在进行一场多轮比赛。每人在每轮都可以获得0到100的得分。

共有 n 轮比赛，比赛结束后，每个人选取 $n - \lfloor n/4 \rfloor$ 个最高得分，通过加和得到总分，总分高的一方获胜，总分相等时算为平局。

明明 n 轮已经结束，Buttersky却要耍起了赖皮，他想通过加赛来取得胜利或者平局。那么，他最少要申请多少轮加赛才有可能获得胜利或者平局呢？注意，加赛之后，选出的最高得分的数目也会变（可以认为是 n 变大了）。

```
// Author: Marvolo
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

int n,i,ans,x,p,q;
int a[N],b[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-') f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();
    for (i=1;i<=n;i++) a[i]=read();
    for (i=1;i<=n;i++) b[i]=read();
    sort(a+1,a+1+n);
    sort(b+1,b+1+n);
    for (i=1;i<=n;i++) a[i]+=a[i-1];
    for (i=1;i<=n;i++) b[i]+=b[i-1];
    for (i=0;i<=n;i++){
        x = (n+i)-(n+i)/4;
```

```

        if (x>=n)    q=b[n];
        else q=b[n]-b[n-x];
        p=a[n]-a[n-(x-i)]+i*100;
        if (p>=q){
            cout<<i<<endl;
            break;
        }
    }
    return 0;
}

```

F

F 猫捉老鼠

时间限制：1000ms 内存限制：65536kb

通过率：125/129 (96.90%) 正确率：125/179 (69.83%)

题目描述

在一个坐标轴上，有一只猫， k 只老鼠和一个老鼠洞。猫位于坐标原点，也就是 $x = 0$ 处。老鼠洞位于 $x = m$ 处。所有的老鼠都在猫和老鼠洞之间，第 i 只老鼠的坐标是 $x_i (0 < x_i < m)$ ，每一个点上可以有很多只老鼠。

在一秒钟之内，会发生以下两个事件：首先，只有一只老鼠向右移动1个单位距离。如果老鼠到达洞口，它就会藏起来(也就是说，这个老鼠不会再移动到任何地方，也不会被猫吃掉)。然后，猫向右移动1个单位距离。如果猫在到达一个新的位置后，这个位置上有老鼠，那么它就会把老鼠都吃掉。

换句话说，每一秒中，第一部分是老鼠移动。如果老鼠已经到达洞口，它就安全了。然后猫开始移动。猫会吃掉它所到之处的老鼠(如果猫已经到达老鼠洞，它也不会吃在洞里的老鼠)。

每一秒钟，你都可以指定一个老鼠移动。问最多可以有几只老鼠可以不被吃掉？

```

// Author: Serein
#include<bits/stdc++.h>
using namespace std;

typedef long long LL;
typedef pair<LL, LL> PII;
const int N = 400010, M = 100010, mod = 1e9 + 7;

inline LL read() {
    LL x = 0, fl = 1; char c = getchar();
    while (c < '0' || c > '9') { if(c == '-') fl = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * fl;
}

LL n, m, k, top, t;
int a[N];

int main()
{
    t = read();
    while(t --) {
        n = read(), k = read();
        for (int i = 1; i <= k; i ++) a[i] = read();
        sort (a + 1, a + 1 + k);
        int ans = 0, sum = 0;
        for (int i = k; i >= 1; i --) {
            if(sum >= a[i]) break;

```

```

        sum += n - a[i];
        ans ++;
    }
    printf("%d\n", ans);
}
return 0;
}

```

G

G 均值维护

时间限制：1000ms 内存限制：65536kb

通过率：59/95 (62.11%) 正确率：59/470 (12.55%)

题目描述

Marvolo 有一个长度为 n 的整数数列。这天，他觉得这个数列太长了，不方便存储，他打算从数列中删去两个元素，并且不想破坏这个数列的一些性质，比如平均值。

现在他想知道，在删去数之后，保证新数列的平均值和原来的数列一样的方案有多少种？

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 200010
#define INF 0x3f3f3f3f
using namespace std;

int T,n,i,j,tot,low,high;
int a[N],b[N];
LL Ans;
double avg,x;

map <int,LL> M;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

```



```

int main(){
    T=read();
    while (T--){
        n=read();
        for (i=1;i<=n;i++) a[i]=read();
        sort(a+1,a+1+n);
        avg = tot = 0; M.clear();
        for (i=1;i<=n;i++){
            avg+=a[i]; M[a[i]]++;
            if (M[a[i]]==1) b[++tot]=a[i];
        }
        avg /= 1.0*n;
        low=1; high=tot;
        Ans=0;
        while (low<high){
            x = (b[low]+b[high])*0.5;
            if (x > avg){
                high--; continue;
            } else if (x < avg){
                low++; continue;
            }
            Ans += M[b[low]]*M[b[high]];
            low++; high--;
        }
        for (i=1;i<=tot;i++)
            if (b[i]*1.0 == avg){
                Ans += M[b[i]]*(M[b[i]]-1)/2;
                break;
            }
        printf("%lld\n",Ans);
    }
    return 0;
}

```

H

H 修剪绿藤

时间限制：1000ms 内存限制：65536kb

通过率：52/74 (70.27%) 正确率：52/257 (20.23%)

题目描述

在得到了神奇的绿藤后，你开始批量种植，绿藤的长势喜人，但也离不开你辛勤的修剪。

维护一个绿藤集合，初始时集合为空，支持如下几种操作：

1. **I x**，插入一个数 x ，表示你刚刚种植了一个高度为 x 的绿藤；
2. **PM**，输出当前绿藤集合中绿藤高度的最小值；
3. **DM**，删除当前绿藤集合中的最小值（数据保证此时的最小值唯一）；
4. **D k**，删除第 k 个种植的绿藤；
5. **C k x**，修改第 k 个种植的绿藤的高度，将其变为 x ；

— 共要进行 N 次操作。

```

// Author: Marvolo
#include<cstring>

```

```

#include<cstdio>
#include<iostream>
#include<cmath>
#include<ctime>
#include<cstdlib>
#define IL inline
#define N 200010
using namespace std;

int T,x,cnt,root,total;
int p[N];
char op[10];
int val[N],key[N],size[N],son[N][2];

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57)    {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL void Update(int x){
    size[x]=size[son[x][0]]+size[son[x][1]]+1;
}

IL void Split(int now,int k,int &x,int &y){
    if (!now){x=y=0;    return;}
    if (val[now]<=k)    {
        x=now;    Split(son[now][1],k,son[now][1],y);
    }    else {
        y=now;    Split(son[now][0],k,x,son[now][0]);
    }
    Update(now);
}

IL int Merge(int x,int y){
    if (!x || !y)    return x+y;
    if (key[x]<key[y]){
        son[x][1]=Merge(son[x][1],y);
        Update(x);    return x;
    }    else {
        son[y][0]=Merge(x,son[y][0]);
        Update(y);    return y;
    }
}

IL int New(int x){
    key[++cnt]=rand();    val[cnt]=x;    size[cnt]=1;
    return cnt;
}

IL void Insert(int k){
    int x=0,y=0;
    Split(root,k,x,y);
    root=Merge(Merge(x,New(k)),y);
}

IL void Delete(int k){

```

```

        int x=0,y=0,z=0;
        Split(root,k,x,y);    Split(x,k-1,x,z);
        z=Merge(son[z][0],son[z][1]);
        root=Merge(x,Merge(z,y));
    }

    IL int Rank(int k){
        int x=0,y=0,ans=0;
        Split(root,k-1,x,y);
        ans=size[x]+1;
        root=Merge(x,y);
        return ans;
    }

    IL int Find(int x,int k){
        while (1){
            if (k<=size[son[x][0]])    x=son[x][0];
            else if (k>size[son[x][0]]+1)    k-=size[son[x][0]]+1,x=son[x][1];
            else return val[x];
        }
    }

    IL int Pre(int k){
        int x=0,y=0,ans=0;
        Split(root,k-1,x,y);
        ans=Find(x,size[x]);    root=Merge(x,y);
        return ans;
    }

    IL int Nex(int k){
        int x=0,y=0,ans=0;
        Split(root,k,x,y);
        ans=Find(y,1);    root=Merge(x,y);
        return ans;
    }

    int main(){
        #ifdef __Marvolo
        freopen("zht.in","r",stdin);
        freopen("zht.out","w",stdout);
        #endif
        srand(time(NULL));
        T=read();
        while (T--){
            scanf("%s",op);
            if (op[0] == 'I'){
                x=read();
                Insert(x);
                p[++total] = x;
            }
            else if (op[0] == 'P') printf("%d\n",Find(root,1));
            else if (op[1] == 'M') Delete(Find(root,1));
            else if (op[0] == 'D') Delete(p[read()]);
            else {
                int k = read();
                x=read();
                Delete(p[k]);
                p[k]=x; Insert(x);
            }
        }
    }

```

```
    }  
}  
return 0;  
}
```

I 关于一个既不返回最大值也不返回最小值的奇奇怪怪的堆居然被放到了 I 题这档事

时间限制：1000ms 内存限制：65536kb

通过率：29/46 (63.04%) 正确率：29/161 (18.01%)

题目描述

这是一个奇奇怪怪的堆，它不是最小堆，也不是最大堆，这个堆的堆顶是堆中第 k 小的元素。

对于给定的 k ，你需要实现如下三个操作：

- `top()`：返回第 k 小的元素，若不足 k 个，则返回堆中最大的元素，若堆为空，则返回 -1。
- `push(x)`：将数字 x 压入堆中
- `pop()`：将第 k 小的元素从堆中弹出，若不足 k 个，则弹出堆中最大的元素，若堆为空，则不做任何操作

```
// Author: Alex  
#include <bits/stdc++.h>  
  
using namespace std;  
  
struct Priority_Queue {  
    int k;  
    priority_queue<int> q1;  
    priority_queue<int, vector<int>, greater<int> > q2;  
    Priority_Queue() { }  
    Priority_Queue(int _k) : k(_k) {}  
  
    void push(int v) {  
        q1.push(v);  
        while (q1.size() >= k) q2.push(q1.top()), q1.pop();  
    }  
  
    void pop() {  
        if (!q2.empty()) q2.pop();  
        else if (!q1.empty()) q1.pop();  
    }  
  
    int top() {  
        if (!q2.empty()) return q2.top();  
        else if (!q1.empty()) return q1.top();  
        else return -1;  
    }  
};  
  
int n, m, k;  
  
int main() {  
    scanf("%d%d", &n, &k);  
    Priority_Queue Q = Priority_Queue(k);  
    while (n--) {
```

```

int op, x;
scanf("%d", &op);
if (op == 2) {
    scanf("%d", &x);
    Q.push(x);
} else if (op == 3) {
    Q.pop();
} else if (op == 1) {
    printf("%d\n", Q.top());
}
}
return 0;
}

```

J

J Matrix53 也不会出题了

时间限制：1000ms 内存限制：65536kb

通过率：10/17 (58.82%) 正确率：10/54 (18.52%)

题目描述

Matrix53 也不会出题了，于是他找来 Serein 和 ButterSky 来帮他出题 ↓

Matrix53: 实在不会出题了，给定范围内的题我实在想不出来了 **QAQ**

Serein: 那就出重复的题吧，秦九韶算法怎么样？

ButterSky: 要不不改一改秦九韶算法的内容，出一道题吧 **O.O**

Matrix53: emm...有道理，我再想想好了。

过了一天，Matrix53 终于出了一道题：

给定一个式子： $a_n \times (\prod_{i=1}^n x_i) + a_{n-1} \times (\prod_{i=1}^{n-1} x_i) + \dots + a_1 \times x_1 + a_0$

Matrix53: 这个式子里有 n 个 x ，有 $n+1$ 个 a ，我再额外给一个 x ，为 x_{n+1}

给定一个可以任意次使用的操作： 交换 a_i 和 a_j ，与此同时交换 x_{i+1} 和 x_{j+1}

```

// Author: Matrix53
#include <bits/stdc++.h>

using namespace std;

typedef long long ll;
typedef pair<ll, ll> pll;

const ll mod = 998244353;
const int maxn = 1e5 + 5;

pll data[maxn];

bool isLower(pll a, pll b) {
    return a.first * b.second + b.first > b.first * a.second + a.first;
}

int main() {
    // freopen("D:\\workspace\\algo\\BUAAOJ-X\\Matrix53-5\\data\\1.in", "r",
    // stdin);

```

```
int n;
scanf("%d", &n);
for (int i = 0; i <= n; ++i) scanf("%lld", &data[i].first);
for (int i = 0; i <= n; ++i) scanf("%lld", &data[i].second);
sort(data, data + n + 1, isLower);
ll ans = 0;
for (int i = 0; i <= n; ++i)
    ans = (ans * data[i].second + data[i].first) % mod;
printf("%lld", ans);
return 0;
}
```

E4

A

A DNA螺旋

时间限制：1000ms 内存限制：65536kb

通过率：232/240 (96.67%) 正确率：232/613 (37.85%)

题目描述

在生物学应用中，经常需要比较两个（或更多）不同有机体的DNA。一个DNA螺旋由一串被称为基的分子组成，可能的基包括腺嘌呤，鸟嘌呤、胞嘧啶和胸腺嘧啶。若分别以它们的首字母来表示这些基，一个DNA螺旋可以表示为在有穷集合 {A, G, C, T} 上的一个串。

现给出两个表示DNA螺旋的串，请你求出它们的最长公共子序列。

有限集合 S 的串是 S 中元素的一个序列。例如 AGCTTCGA 是一个表示DNA螺旋的串。

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;
const int N = 1010;
int c[N][N], b[N][N];
int n, m, k;
string x, y;
void printLCS(int i, int j) {
    if(i == 0 || j == 0) return;
    if(b[i][j] == 1) {
        printLCS(i - 1, j - 1);
        printf("%c", x[i - 1]);
    }
    else if(b[i][j] == 2)
        printLCS(i - 1, j);
    else
        printLCS(i, j - 1);
}
int main()
{
    scanf("%d", &k);
    cin >> x >> y;
    n = x.size();
    m = y.size();
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if(x[i - 1] == y[j - 1]) {
                c[i][j] = c[i - 1][j - 1] + 1;
                b[i][j] = 1;
            }
            else if(c[i - 1][j] >= c[i][j - 1]) {
                c[i][j] = c[i - 1][j];
                b[i][j] = 2;
            }
            else {
                c[i][j] = c[i][j - 1];
            }
        }
    }
    printLCS(n, m);
}
```

```

        b[i][j] = 3;
    }
}
}
if(k == 0) printf("%d", c[n][m]);
else    printLCS(n, m);
return 0;
}

```

B

B 深度学习

时间限制：1000ms 内存限制：65536kb

通过率：225/231 (97.40%) 正确率：225/746 (30.16%)

题目描述

在深度学习领域，经常会对一些矩阵进行乘法运算。如果计算顺序规划不合理，会带来极大的时间开销，影响训练效率。

给出 n 个矩阵的行、列大小，请你计算将这 n 个矩阵乘起来，最少和最多分别需要几次乘法运算。注意，不可以改变矩阵顺序，只能改变乘法顺序。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define LL inline
#define reg register
#define LL long long
#define N 1010
#define INF 0x3f3f3f3f3f3f3f11
using namespace std;

LL n,i,j,k,l;
LL a[N],f[N][N];

LL LL Min(LL a,LL b){return (a<b)?a:b;}
LL LL Max(LL a,LL b){return (a>b)?a:b;}

LL LL read(){
    LL p=0,f=1; char c=getchar();
    while (c<48||c>57) {if (c=='-') f=-1; c=getchar();}
    while (c>=48&&c<=57) p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    n=read();
    for (i=1;i<=n+1;i++) a[i]=read();
}

```



```

memset(f,INF,sizeof(f));
for (i=1;i<=n;i++) f[i][i] = 0;
for (l=1;l<n;l++)
    for (i=1;i<=n;i++){
        j = i+l;
        for (k=i;k<j;k++){
            f[i][j] = Min(f[i][j], f[i][k] + f[k+1][j] + a[i] * a[k+1] *
a[j+1]);
        }
    }
cout<<f[1][n]<<endl;

memset(f,0,sizeof(f));
for (l=1;l<n;l++)
    for (i=1;i<=n;i++){
        j = i+l;
        for (k=i;k<j;k++){
            f[i][j] = Max(f[i][j], f[i][k] + f[k+1][j] + a[i] * a[k+1] *
a[j+1]);
        }
    }
cout<<f[1][n]<<endl;

return 0;
}

```

C

C Matrix53只会发表情包

时间限制：1000ms 内存限制：65536kb

通过率：56/64 (87.50%) 正确率：56/126 (44.44%)

题目描述

Matrix53 喜欢在微信群里潜水，他只会发表情包。

Matrix53 有 n 个表情可以发送，这 n 个表情拥有严格递增的正整数标号，分别为 $s_1, s_2 \dots s_n$ 。

在发送表情的前一刻，*Matrix53* 会灵机一动，从脑海里找到表情的标号。

但是 *Matrix53* 的记忆不一定是正确的，他有可能想到错误的标号，这时由于 *Matrix53* 找不到对应标号的表情，所以这时他不会发表情包。

Matrix53 想到正确的标号 $s_1, s_2 \dots s_n$ 的频率分别为 $f_1, f_2 \dots f_n$ 。

Matrix53 想到错误的标号的频率分别为 $p_0, p_1 \dots p_n$ ，设 *Matrix53* 想到的标号为 t ，有：

$p_0 = P(t < s_1)$, $p_n = P(t > s_n)$, $p_i (0 < i < n) = P(s_i < t < s_{i+1})$ ，其中 $P(x)$ 表示事件发生的频率。

Matrix53 上过算法课，他将这 n 个表情以标号为键构建了一颗二叉搜索树。

也就是说，对于这棵树的任意一颗子树，若记子树树根为 x ，则：

x 的标号大于 x 的左子树的所有结点的标号， x 的标号小于 x 的右子树的所有结点的标号。

对于一次搜索，*Matrix53* 会用标号从树根开始检索表情，当他找到表情或者能够判定找不到表情时，他会结束这次搜索，本次搜索的搜索代价为搜索的深度。

```

// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;
const int maxn = 1e3 + 5;

long long dp[maxn][maxn], w[maxn][maxn];
int f[maxn], p[maxn], root[maxn][maxn];

```

```

int main() {
    // freopen("D:/Workspace/algo/BUAAOJ-X/Matrix53-E4-C/data/1.in", "r", stdin);
    int n;
    scanf("%d", &n);
    for (int i = 1; i <= n; ++i) scanf("%lld", &f[i]);
    for (int i = 0; i <= n; ++i) scanf("%lld", &p[i]);
    for (int i = 1; i <= n; ++i) {
        root[i][i] = i;
        w[i][i] = f[i] + p[i - 1] + p[i];
        dp[i][i] = w[i][i];
    }
    for (int len = 1; len < n; ++len) {
        for (int l = 1; l <= n - len; ++l) {
            int r = l + len;
            dp[l][r] = LONG_LONG_MAX;
            w[l][r] = w[l][r - 1] + f[r] + p[r];
            for (int mid = root[l][r - 1]; mid <= root[l + 1][r]; ++mid) {
                long long sum = dp[l][mid - 1] + dp[mid + 1][r] + w[l][r];
                if (dp[l][r] > sum) {
                    dp[l][r] = sum;
                    root[l][r] = mid;
                }
            }
        }
    }
    printf("%lld", dp[1][n]);
    return 0;
}

```

D

D 道歉的35xirtaM

时间限制: 1000ms 内存限制: 65536kb

通过率: 214/222 (96.40%) 正确率: 214/565 (37.88%)

题目描述

olovraM 生气了，原因是 35xirtaM 偷吃了 olovraM 的奥利奥小饼干。

35xirtaM 很抱歉，于是他决定向 olovraM 道歉。

35xirtaM 带 olovraM 来到了一家饼干店。

饼干店里有 n 种饼干，每种饼干的存货都是充足的，且都是一块一块零售的。

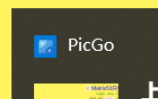
35xirtaM：你随便挑吧！

olovraM：我不客气了！

35xirtaM 的零花钱为 m ，olovraM 最多只能花光这些钱。

olovraM 对于每种饼干有不同的满意度，olovraM 的满意度为他所买的饼干的满意度之和。

求 olovraM 的最大满意度。



```

// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;

```

```

long long dp[100005];

int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    while (n--) {
        int v, w;
        scanf("%d%d", &v, &w);
        for (int i = 0; i <= m - w; ++i) {
            dp[i + w] = max(dp[i + w], dp[i] + v);
        }
    }
    printf("%lld", dp[m]);
    return 0;
}

```

E

E 最长波动子数组

时间限制：1000ms 内存限制：65536kb

通过率：159/168 (94.64%) 正确率：159/269 (59.11%)

题目描述

对于一个数组，如果对于其中任意一个元素，它邻近的元素要么都比它大要么都比它小（只有一个亦然），那么称这个数组为波动数组。

现在给你一个数组，让你求出他的所有为波动数组的子数组（不一定连续）中，最长数组的长度。

输入

第一行一个数，为数组长度 $n(1 \leq n \leq 1000)$ 。

接下来一行 n 个正整数 $a_1, a_2, \dots, a_n(1 \leq a_i \leq 100)$ ，表示数组。

```

#include<bits/stdc++.h>
#define N 1005
using namespace std;
int n,a[N],dp[2][N],ans;
int main()
{
    scanf("%d",&n);
    for(int i=1; i<=n; i++)
        scanf("%d",a+i);
    for(int i=1; i<=n; i++)
    {
        dp[0][i]=1;
        dp[1][i]=1;
        for(int j=1; j<i; j++)
        {
            if(a[j]<a[i])
                dp[0][i]=max(dp[1][j]+1,dp[0][i]);
            if(a[j]>a[i])
                dp[1][i]=max(dp[0][j]+1,dp[1][i]);
        }
        ans=max(ans,dp[0][i]);
        ans=max(ans,dp[1][i]);
    }
    printf("%d",ans);
}

```

```
    return 0;
}
```

F

F 更改字符串

时间限制: 1000ms 内存限制: 65536kb

通过率: 152/158 (96.20%) 正确率: 152/314 (48.41%)

题目描述

给定两个字符串, 请你对第一个字符串进行操作, 使它变成第二个字符串, 每次操作可以在任意一个位置添加或删除一个字符。求最少操作次数。

```
#include<bits/stdc++.h>
#define N 2005
using namespace std;
int n,m,dp[N][N];
char a[N],b[N];
int main()
{
    scanf("%s",a+1);
    scanf("%s",b+1);
    n=strlen(a+1);
    m=strlen(b+1);
    for(int i=0; i<=n; i++)
    {
        for(int j=0; j<=m; j++)
        {
            if(i==0||j==0)
                dp[i][j]=i+j;
            else
            {
                if(a[i]==b[j])
                    dp[i][j]=dp[i-1][j-1];
                else
                    dp[i][j]=min(dp[i][j-1]+1,dp[i-1][j]+1);
            }
        }
    }
    printf("%d",dp[n][m]);
    return 0;
}
```

G

G 贪心只能过样例

时间限制：1000ms 内存限制：65536kb

通过率：39/49 (79.59%) 正确率：39/133 (29.32%)

题目描述

本题采用 SPJ

Alex 发现他睡过头了没有报上六级考试，现在继续学习语法性价比不高。

于是他投入的小学奥数的怀抱。

这是他上小学二年级的弟弟问他的一道题：

给定 n 和 k ，你的任务是找到一个长度为 k 的数列，满足：

- $a_1 + a_2 + \dots + a_k = n$
- $LCM(a_1, a_2, \dots, a_k) \leq \frac{n}{2}$

LCM 是最小公倍数的缩写，比如 $LCM(2, 3) = 6$, $LCM(5, 10) = 10$

可以证明答案一定存在，如果有多个解可以输出任意解。

Alex 准备去夏威夷恶补一下数学知识，这道题就留给你了。

```
// Author: Alex
#include <bits/stdc++.h>

using namespace std;

int main() {
    int T; cin >> T; while (T--) {
        int n, k; cin >> n >> k;
        if (k > 3) {
            for (int i = 1; i <= k - 3; ++i) cout << 1 << " ";
            n = n - k + 3; k = 3;
        }
        if (n & 1)
            cout << 1 << " " << n / 2 << " " << n / 2 << endl;
        else if (n % 4)
            cout << 2 << " " << (n - 2) / 2 << " " << (n - 2) / 2 << endl;
        else
            cout << n / 4 << " " << n / 4 << " " << n / 2 << endl;
    }
    return 0;
}
```

H

H 双人成行

时间限制：1000ms 内存限制：65536kb

通过率：87/95 (91.58%) 正确率：87/189 (46.03%)

题目描述

有一款名为双人成行的双人游戏，两个玩家分别控制游戏中的两个人物。每个人物有一个能力，可以用 a_1, a_2 表示。初始时，两人的能力都为0。

游戏是以天为单位进行的，每一天这两个人的能力都会分别提升 b_1, b_2 ；此外，每天可以获得一个天赋点，让 b_1 或者 b_2 的值增加1。 b_1, b_2 的初始值一开始也为0。

等两人的能力提升到一定的程度后，就可以打败 Boss 通关。通过查询攻略可知，只要满足 $a_1 \geq l_1, a_2 \geq l_2$ ，就能通关。

现在请你计算出，最快几天才可以通关。注意，每一天都是先增加天赋点，再提升的能力。

```
#include<cstring>
```

```

#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 200010
#define INF 0x3f3f3f3f
using namespace std;

int T,n,i,x;
int a[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    T=read();
    x=1;    n=1;
    for (i=1;i<=200000;i++){
        if (n >= i) a[i]=x;
        else {
            x+=1;    n+=x;    a[i]=x;
        }
    }
    while (T--){
        n=read()+read();
        printf("%d\n",a[n]);
    }
    return 0;
}

```

I 什么是算法

时间限制: 1000ms 内存限制: 65536kb

通过率: 26/33 (78.79%) 正确率: 26/98 (26.53%)

题目描述

Alex 至今也不知道，什么是算法。

他虚心的向 Marvolo 请教，Marvolo 扔给他一本《算法导论》（一下简称算导）。告诉你你只要把这本书读懂，你就懂算法了。

Alex 依然不明白，但他至少有了方向。

Alex 决定用 $n(1 \leq n \leq 10^5)$ 天来学习算导，如果心情好的话每天都会从 $k(1 \leq k \leq 10^5)$ 章中随机挑一章学习。但是心情不好的时候会跳过最后一章，因为那一章难度太大了看完会让心情变糟糕。

已知 Alex 一共有 $m(1 \leq m \leq n - 2)$ 天心情不好（不会是第一天或最后一天），分别是第 $a_1, a_2, \dots, a_m$ 天。由于 Alex 不喜欢连续作战，Alex 相邻的两天不会选择同一章学习。那么， n 天后，Alex 可能有多少种不同的学习路线呢？

答案可能很大，你只需要输出答案对 998244353 取模后的结果即可。

```
// Author Alex
#include <bits/stdc++.h>

using namespace std;

const int N = 100100;
const int mod = 998244353;

int n, k, m, a[N];
long long dp[N][2];
bool vis[N];

int main() {
    cin >> n >> m >> k;
    for (int i = 1; i <= m; ++i)
        cin >> a[i], vis[a[i]] = true;
    sort(a + 1, a + 1 + m);
    dp[1][0] = 1; dp[1][1] = k - 1;
    for (int i = 2; i <= n; ++i) {
        if (!vis[i]) {
            dp[i][0] = dp[i - 1][1];
            dp[i][1] = dp[i - 1][1] * (k - 2) + dp[i - 1][0] * (k - 1);
        } else {
            dp[i][0] = 0;
            dp[i][1] = dp[i - 1][1] * (k - 2) + dp[i - 1][0] * (k - 1);
        }
        dp[i][0] %= mod;
        dp[i][1] %= mod;
    }
    cout << (dp[n][0] + dp[n][1]) % mod << endl;
    return 0;
}
```

J 电梯

时间限制: 500ms 内存限制: 65536kb

通过率: 8/21 (38.10%) 正确率: 8/124 (6.45%)

题目描述

有 n 个乘客会来到位于第 0 层的地下停车场, 等待电梯将他们带到上层。第 i 个乘客会在 t_i 时刻来到电梯并想要到达 a_i 层。只要电梯在第 0 层, 乘客总是可以进入电梯且电梯容量无限。

电梯在第 0 层总会保持打开状态, 直到你派它去运送乘客, 然后它移动到它需要的最高楼层 (即当前在电梯中的所有乘客中的最大 a_i) 并返回停车场。电梯在每层楼之间的移动需要 1 个单位的时间。电梯开门和关门的时间, 以及乘客进出电梯的时间, 忽略不计。

在 0 时刻, 电梯在第 0 层。求电梯在运送完所有人后返回第 0 层的最短时间。

```
#include<bits/stdc++.h>
#define ll long long
#define N 200005
using namespace std;
ll n,cnt;
ll pret[N],prea[N],t[N],a[N];
ll dp[N];
int main()
{
    while(~scanf("%lld",&n))
    {
        for(int i=1;i<=n;i++)
        {
            scanf("%lld%lld",&pret[i],&prea[i]);
        }
        cnt=0;
        for(int i=1;i<=n;i++)
        {
            while(prea[i]>=a[cnt]&&cnt>0)
                cnt--;
            cnt++;
            a[cnt]=prea[i];
            t[cnt]=pret[i];
        }
        n=cnt;
        deque<int> q;
        for(int i=1;i<=n;i++)
        {
            int k = upper_bound(dp,dp+i,t[i])-dp-1;
            dp[i] = t[i] + a[k+1]*2;
            while(!q.empty()&&q.front()<=k)
                q.pop_front();
            if(!q.empty())
            {
                dp[i]=min(dp[i],dp[q.front()+1]+a[q.front()+1]*2);
            }
            while(!q.empty()&&dp[q.back()+1]+a[q.back()+1]*2>=dp[i]+a[i+1]*2)
                q.pop_back();
            q.push_back(i);
        }
        printf("%lld\n",dp[n]);
    }
    return 0;
}
```


E5

A

A 黑白

时间限制: 1000ms 内存限制: 65536kb

通过率: 239/242 (98.76%) 正确率: 239/661 (36.16%)

题目描述

这天你来到一个神秘的宫殿，宫殿的地面是由黑色与白色两种颜色的瓷砖贴成的。

你可以选择其中的一块瓷砖作为起点，但每次只能向相邻（上下左右四个方向）且颜色相同的瓷砖移动。

请问你在选择一个起点后，最多能够到达多少块瓷砖？（注意，瓷砖是可以重复走的）

```
// Author: Marvolo
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 110
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,j,Ans;
int a[N][N];
char c[N][N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57)  {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL int Dfs(int x,int y){
    reg int sum = 1;
    if (a[x][y])    return 0;
    a[x][y] = 1;
    if (c[x+1][y] == c[x][y])    sum+=Dfs(x+1,y);
```

```

        if (c[x-1][y] == c[x][y])    sum+=Dfs(x-1,y);
        if (c[x][y+1] == c[x][y])    sum+=Dfs(x,y+1);
        if (c[x][y-1] == c[x][y])    sum+=Dfs(x,y-1);
        return sum;
    }

    int main(){
        #ifdef __Marvolo
        freopen("zht.in","r",stdin);
        freopen("zht.out","w",stdout);
        #endif
        scanf("%d%d",&n,&m);
        getchar();
        for (i=1;i<=n;i++)
            scanf("%s",c[i]+1);
        for (i=1;i<=n;i++)
            for (j=1;j<=m;j++)
                if (a[i][j])    continue;
                else Ans=Max(Ans, Dfs(i,j));
        cout<<Ans<<endl;
        return 0;
    }

```

B

B 克鲁斯卡尔与普莱姆

时间限制：1000ms 内存限制：65536kb

通过率：220/242 (90.91%) 正确率：220/906 (24.28%)

题目描述

克鲁斯卡尔有一个图

而普莱姆想要一个树

克鲁斯卡尔说 he 可以从图中挖出一个树

但普莱姆更喜欢边权和更小的树

克鲁斯卡尔小心翼翼的挖出了一棵树

但普莱姆却说那不是最小的。

克鲁斯卡尔懵了

最小的树究竟是有小呢？

 PicGo

```

// Author: Alex
#include<bits/stdc++.h>

using namespace std;

const int N = 30000 + 100;
const int M = 50000 + 300;

struct Road {
    int u, v, val;
    inline bool operator < (const Road &a) const {
        return val < a.val;
    }
} A[M];

```

```

int n, m, fa[N], ans = 0;
int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]);}

long long kruskal() {
    sort(A + 1, A + 1 + m);
    for(int i = 1; i <= n; ++i)
        fa[i] = i;
    long long res = 0;
    for(int i = 1; i <= m; ++i) {
        int u = find(A[i].u);
        int v = find(A[i].v);
        if(u == v) continue;
        fa[u] = v; res += A[i].val;
    }
    return res;
}

int main() {
    scanf("%d%d", &n, &m);
    for(int i = 1; i <= m; ++i)
        scanf("%d%d%d", &A[i].u, &A[i].v, &A[i].val);
    cout << kruskal() << endl;
    return 0;
}

```

C

C 最大拓扑排序

时间限制: 1000ms 内存限制: 65536kb

通过率: 186/218 (85.32%) 正确率: 186/734 (25.34%)

题目描述

给定 n 个点, m 条有向边, 求拓扑排序。

P.S. 对于拓扑排序不唯一的情况, 先输出序号大的点, 再输出序号小的点。

输入

第一行两个正整数 n m

接下来 m 行, 每行两个正整数 u v , 表示 u 到 v 存在有向边

```

// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;

vector<int> g[100005];
priority_queue<int> pq;
int inDegree[100005];

int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < m; ++i) {
        int u, v;

```

```

scanf("%d%d", &u, &v);
g[u].emplace_back(v);
inDegree[v]++;
}
for (int i = 1; i <= n; ++i) {
    if (inDegree[i] == 0) {
        pq.push(i);
    }
}
while (!pq.empty()) {
    int now = pq.top();
    pq.pop();
    printf("%d ", now);
    for (auto v : g[now]) {
        inDegree[v]--;
        if (inDegree[v] == 0) {
            pq.push(v);
        }
    }
}
return 0;
}

```

D

D Floyd-Warshall

时间限制：1000ms 内存限制：65536kb

通过率：164/183 (89.62%) 正确率：164/1017 (16.13%)

题目描述

给定 n 个点, m 条有向边, p 个询问。

每个询问查询 u 到 v 的最短路径长度, 若不存在路径则输出 -1 。

P.S. 点的编号从 1 到 n

```

// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll dp[505][505];

int main() {
    int n, m, p;
    scanf("%d%d%d", &n, &m, &p);
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            dp[i][j] = LONG_LONG_MAX;
        }
        dp[i][i] = 0;
    }
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        scanf("%d%d%d", &u, &v, &w);
        dp[u][v] = min(dp[u][v], ll(w));
    }
}

```

```

}
for (int k = 1; k <= n; ++k) {
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (dp[i][k] != LONG_LONG_MAX && dp[k][j] != LONG_LONG_MAX &&
                dp[i][k] + dp[k][j] < dp[i][j]) {
                dp[i][j] = dp[i][k] + dp[k][j];
            }
        }
    }
}
}
while (p--) {
    int u, v;
    scanf("%d%d", &u, &v);
    printf("%lld\n", dp[u][v] == LONG_LONG_MAX ? -1 : dp[u][v]);
}
return 0;
}

```

E

E Stream

时间限制: 1000ms 内存限制: 65536kb

通过率: 81/97 (83.51%) 正确率: 81/417 (19.42%)

题目描述

给定 n 个点和 m 条有向边，给定每条边的容量，求从点 s 到点 t 的最大流。

输入

第一行四个整数 n, m, s, t

接下来的 m 行，每行三个整数 u, v, w , 表示 u 到 v , 流量为 w 的一条边

输出

输出点 s 到点 t 的最大流

```

// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const int N = 1e4 + 5, M = 2e5 + 5;
const ll inf = LLONG_MAX;

int tot = 1, lnk[N], ter[M], nxt[M], dep[N], cur[N];
ll val[M];

void add(int u, int v, ll w) {
    ter[++tot] = v;
    nxt[tot] = lnk[u];
    lnk[u] = tot;
    val[tot] = w;
}

void addedge(int u, int v, ll w) {
    add(u, v, w);
    add(v, u, 0);
}

```

```

}

int bfs(int s, int t) {
    memset(dep, 0, sizeof(dep));
    memcpy(cur, lnk, sizeof(lnk));
    queue<int> q;
    q.push(s);
    dep[s] = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int i = lnk[u]; i; i = nxt[i]) {
            int v = ter[i];
            if (val[i] && !dep[v]) {
                q.push(v);
                dep[v] = dep[u] + 1;
            }
        }
    }
    return dep[t];
}

ll dfs(int u, int t, ll flow) {
    if (u == t) return flow;
    ll ans = 0;
    for (int &i = cur[u]; i && ans < flow; i = nxt[i]) {
        int v = ter[i];
        if (val[i] && dep[v] == dep[u] + 1) {
            ll x = dfs(v, t, min(val[i], flow - ans));
            if (x) {
                val[i] -= x;
                val[i ^ 1] += x;
                ans += x;
            }
        }
    }
    if (ans < flow) dep[u] = -1;
    return ans;
}

ll dinic(int s, int t) {
    ll ans = 0;
    while (bfs(s, t)) {
        ll x;
        while ((x = dfs(s, t, inf))) ans += x;
    }
    return ans;
}

int main() {
    int n, m, p, t;
    scanf("%d%d%d%d", &n, &m, &p, &t);
    for (int i = 0; i < m; ++i) {
        int u, v, c;
        scanf("%d%d%d", &u, &v, &c);
        addedge(u, v, c);
    }
    printf("%lld", dinic(p, t));
}

```

```
    return 0;
}
```

F

F 卷卷卷

时间限制：1000ms 内存限制：65536kb

通过率：47/59 (79.66%) 正确率：47/152 (30.92%)

题目描述

有 n 个同学在玩一种叫做“卷卷卷”的游戏。游戏内容是这样的：第 i 个人卷的能力为 i ，这些人中一开始存在着 m 对朋友关系， a 和 b 可能是朋友（注意，朋友关系没有传递性， a 和 b 是朋友， b 和 c 是朋友，不能得到 a 和 c 是朋友）。

在游戏开始后，有三种事件可能发生：

1. 新增一对朋友关系。
2. 删除一对朋友关系。
3. 所有人开卷。开卷之后，如果一个人的所有朋友的卷的能力都比他高，那么他就会被卷出局，其他人继续卷，直到没有人再出局为止，此时剩下的人为获胜者。

现在，给出初始时的朋友关系以及接下来要发生的 q 个事件。请算出每次开卷之后，有多少个获胜者。注意，两次开卷之间是独立的，一个人的出局只是暂时的，是相对于这一次开卷而言的。具体参见样例。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 200010
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,Ans,x,y,op;
int a[N];
map <pair<int,int>,bool> M;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
```



```

#endif
n=read(); m=read();
for (i=1;i<=m;i++){
    x=read(); y=read();
    if (x>y) Swap(x,y);
    M[make_pair(x,y)]=1;
    a[x]++;
    if (a[x]==1) Ans++;
}
m=read();
while (m--){
    op=read();
    if (op==3){
        printf("%d\n",n-Ans);
        continue;
    }
    x=read(); y=read();
    if (x>y) Swap(x,y);
    if (op==1){
        if (M[make_pair(x, y)]) return 0;
        a[x]++;
        if (a[x]==1) Ans++;
        M[make_pair(x, y)] = 1;
    } else {
        if (!M[make_pair(x, y)]) return 0;
        a[x]--;
        if (!a[x]) Ans--;
        M[make_pair(x, y)] = 0;
    }
}
return 0;
}

```

G

G 塔防游戏

时间限制：1000ms 内存限制：65536kb

通过率：62/65 (95.38%) 正确率：62/116 (53.45%)

题目描述

Marvolo最近又迷上了一款类塔防游戏.之所以说是类塔防,是因为别的塔防游戏主要在建塔,而这款游戏却是在拆塔.

已知一张地图中有 n 座防御塔,编号为1至 n .第 i 座防御塔还有一个攻击力 $a[i]$.这些防御塔之间构建了 m 条守护关系.游戏的目的是把所有的防御塔都拆除掉.假如果说 a 号塔和 b 号塔有守护关系,那么,在拆除 a 号塔的时候,会受到来自 b 号塔的伤害,伤害值等同于 b 号塔的攻击力.

一座防御塔在被拆除了之后是不会再攻击的,可以认为防御塔的攻击力被清零了.Marvolo想知道,如果要把这 n 座塔全部拆除,最少要受到多少点伤害?因为Marvolo忙着玩游戏,所以他找到了擅长Python的你,希望你帮他解决这个问题.

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline

```

```

#define reg register
#define LL long long
#define N 1010
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,x,y,Ans;
int a[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();    m=read();
    for (i=1;i<=n;i++)    a[i]=read();
    for (i=1;i<=m;i++){
        x=read();    y=read();
        Ans+=Min(a[x],a[y]);
    }
    cout<<Ans<<endl;
    return 0;
}

```

H

H 约翰的农场

时间限制：1000ms 内存限制：65536kb

通过率：11/18 (61.11%) 正确率：11/73 (15.07%)

注意，本题仅开放C语言提交

题目描述

农民约翰有 n 座农场，标号为 1 到 n ，约翰的家在 1 号农场，现在他想修一条从家到 n 号农场的道路。

修路公司给了约翰 m 条双向道路候选，每条道路连接农场 s_i 和 d_i ，费用为 w_i 。

约翰觉得这个方案代价太大了，于是和修路公司商议，最终双方各退一步。每条道路的花费不变，但是修路公司可以免费帮助修建其中的 k 条，而约翰需要支付剩下的道路中费用最高的那一条道路的费用。

请你帮助约翰来做决定，修一条从家到 n 号农场的道路的最少花费是多少？

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define N 100010

```

```

#define M 200010
int h[N], e[M], w[M], ne[M], idx;
int st[N];
int dist[N];
int n, m, k;
void add(int a, int b, int c) {
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx++;
}
int q[M], hh = 0, tt = 0;
int check(int bound) {
    memset(dist, 0x3f, sizeof dist);
    memset(st, 0, sizeof st);
    dist[1] = 0;
    q[tt++] = 1;
    while(hh != tt) {
        int t = q[hh];
        hh++;
        if(hh == M) hh = 0;
        if(st[t]) continue;
        st[t] = 1;
        for(int i = h[t]; ~i; i = ne[i]) {
            int j = e[i], c = w[i] > bound;
            if(dist[j] > dist[t] + c) {
                dist[j] = dist[t] + c;
                if(!c) {
                    hh--;
                    if(hh < 0) hh += M;
                    q[hh] = j;
                }
            }
            else {
                q[tt] = j;
                tt++;
                if(tt == M) tt = 0;
            }
        }
    }
    return dist[n] <= k;
}

int main() {
    scanf("%d%d%d", &n, &m, &k);
    memset(h, -1, sizeof h);
    while(m--) {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        add(a, b, c), add(b, a, c);
    }
    int l = 0, r = 1e6 + 1;
    while(l < r) {
        int mid = l + r >> 1;
        if(check(mid)) r = mid;
        else l = mid + 1;
    }
    if(r > 1e6) r = -1;
    printf("%d\n", r);
}

```

I 路径

时间限制: 1000ms 内存限制: 65536kb

通过率: 60/72 (83.33%) 正确率: 60/171 (35.09%)

题目描述

给出一棵树，所有边的边长都是1。现在有 q 个询问，每一个询问会给出两个正整数 a, b ，代表着一个人从树上的节点 a 出发，另一个人从 b 出发，双方均沿着最短路向对方走去，每秒移动的距离也为1。问能否恰好在节点处相遇。

对于每个询问，如果能，输出 YES，否则输出 NO。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 100010
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,x,y,lsum=1,Ans,cnt,z;
int head[N],c[N];

struct Edge{
    int t,next,l;
}e[N*8];

inline void Add(int s,int t){
    e[lsum].t=t;    e[lsum].next=head[s];    head[s]=lsum++;
}

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL void Dfs(int x,int fa){
    reg int i=0;
    for (i=head[x];i;i=e[i].next){
        if (e[i].t == fa)    continue;
```

```

        c[e[i].t]=c[x]^1;
        Dfs(e[i].t,x);
    }
}

int main(){
    /*
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    */
    n=read();
    for (i=1;i<n;i++){
        x=read();    y=read();
        Add(x,y);    Add(y,x);
    }
    c[1]=1; Dfs(1,0);
    m=read();
    for (i=1;i<=m;i++){
        x=read();    y=read();
        if (c[x] == c[y])    puts("YE5");
        else puts("N0");
    }
    return 0;
}

```

J

J Alex 和他的矩阵

时间限制：1000ms 内存限制：131072kb

通过率：8/10 (80.00%) 正确率：8/85 (9.41%)

题目描述

众所周知，二维矩阵可以用来存储一张图。

听机房里的大佬们说，存一张图的方法有很多，而用二维矩阵存是耗费空间最多的。

但是 Alex 一贯奉行的宗旨是算法能用就行，代码能跑就行。

Alex 有一个 n 行 m 列的矩阵，在 (i, j) 处的值为 a_{ij}

Alex 打算在这个矩阵上玩一个染色游戏：

- 可以花费 a_{ij} 数量的金币，将 (i, j) 这一格染成黑色
- 对于任意的两行两列相交得到的四个点，如果其中有三个已经染成黑色，则可以不花费任何代价将其染成黑色

那么，如果要把每一个格子都染黑，最少要花多少钱呢？

(这很重要，如果最少花费大于游戏的奖金的话，Alex 就不会玩这个游戏了)

```

// Author: Alex
#include <stdio.h>
#include <stdlib.h>

typedef long long ll;
#define maxn 3030

ll n, m, a, b, c, d, p, dis[maxn + maxn], vis[maxn + maxn], A[maxn][maxn], ans =
0, fa[maxn + maxn];

```

```

11 nxt() {return (a = (a * a * b + a * c + d) % p);}
11 find(int x) {return x == fa[x] ? x : (fa[x] = find(fa[x]));}
11 min(11 a, 11 b) {return a > b ? b : a;}

int main() {
    scanf("%lld%lld%lld%lld%lld%lld", &n, &m, &a, &b, &c, &d, &p);
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            A[i][j] = nxt();
    for (int i = 1; i <= n + m; ++i) fa[i] = i, dis[i] = 0x3f3f3f3f3f3f3f3f;
    int now = 1; dis[now] = 0; vis[now] = 1;
    for (int i = 2; i <= n + m; ++i) {
        if (now <= n)
            for (int j = n + 1; j <= n + m; ++j)
                dis[j] = min(dis[j], A[now][j - n]);
        else
            for (int j = 1; j <= n; ++j)
                dis[j] = min(dis[j], A[j][now - n]);
        11 mincost = 0x3f3f3f3f3f3f3f3f, id = -1;
        for (int j = 1; j <= n + m; ++j)
            if (mincost > dis[j] && !vis[j])
                mincost = dis[j], id = j;
        ans += dis[id]; vis[id] = 1; now = id;
    }
    printf("%lld\n", ans);
    return 0;
}

```

E6

A

A 全部加起来

时间限制：1000ms 内存限制：65536kb

通过率：224/234 (95.73%) 正确率：224/906 (24.72%)

题目描述

给出 n 条线段，求 n 条线段两两之间共有多少对是相交的（有一个或多个交点都视为相交）。

输入

第一行一个整数为线段总数 n ($1 \leq n \leq 1000$)。

接下来 n 行，每行4个整数 x_a ($1 \leq x_a \leq 100$)， y_a ($1 \leq y_a \leq 100$)， x_b ($1 \leq x_b \leq 100$)， y_b ($1 \leq y_b \leq 100$)分别表示线段两个端点横纵坐标。

输出

输出一个数为答案。

```
// Author: Marvolo
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define EPS 1e-5
#define N 1010
#define INF 0x3f3f3f3f
using namespace std;

inline double Min(double a,double b){return (a<b)?a:b;}
inline double Max(double a,double b){return (a>b)?a:b;}
inline double Fabs(double x){return (x<=-EPS)?-x:(x>EPS)?x:0;}
inline bool Equ(double x,double y){return (Fabs(x-y)<EPS);}
inline int Sgn(double x){
    if (Fabs(x)==0)    return 0;
    if (x>0)    return 1;    else return -1;
}

struct Vector{
    double x,y;
    void Clean(void){x=y=0;}
    void write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
    Vector operator +(Vector p){return (Vector){x+p.x,y+p.y};}
    Vector operator -(Vector p){return (Vector){x-p.x,y-p.y};}
    double operator *(Vector p){return x*p.y-y*p.x;}
    double operator &(Vector p){return x*p.x+y*p.y;}
```

```

};
typedef Vector V;

struct Point{
    double x,y;
    void Clean(void){x=y=0.0;}
    void Write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
    bool operator ==(Point a){return (Equ(x-a.x,0) && Equ(y-a.y,0));}
    Point operator +(Point p){return (Point){x+p.x,y+p.y};}
    Point operator /(double l){return (Point){x/l,y/l};}
    Vector operator -(Point p){return (Vector){x-p.x,y-p.y};}
};
typedef Point P;

inline void SW(P &a,P &b){P t=a;    a=b;    b=t;}
inline double Dis2(P a,P b){return pow(a.x-b.x,2)+pow(a.y-b.y,2);}
inline double Dis(P a,P b){return pow(pow(a.x-b.x,2)+pow(a.y-b.y,2),0.5);}

int n,i,j,Ans;
P a[N],b[N];

inline bool Point_On_Seg(P x,P y,P z){
    double s;
    s=(x-y)*(x-z);
    if (Fabs(s)>EPS)    return 0;
    if (Min(y.x,z.x)-EPS>x.x||Max(y.x,z.x)+EPS<x.x)    return 0;
    if (Min(y.y,z.y)-EPS>x.y||Max(y.y,z.y)+EPS<x.y)    return 0;
    return 1;
}

inline bool Seg_Cross(P a,P b,P c,P d){
    if (Point_On_Seg(a,c,d) || Point_On_Seg(b,c,d) || Point_On_Seg(c,a,b) ||
    Point_On_Seg(d,a,b))    return 1;
    if (Sgn((b-a)*(c-a)) * Sgn((b-a)*(d-a)) >= 0)    return 0;
    if (Sgn((d-c)*(a-c)) * Sgn((d-c)*(b-c)) >= 0)    return 0;
    return 1;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    scanf("%d",&n);
    for (i=1;i<=n;i++)
        scanf("%lf%lf%lf%lf",&a[i].x,&a[i].y,&b[i].x,&b[i].y);
    for (i=1;i<=n;i++)
        for (j=i+1;j<=n;j++){
            if (Seg_Cross(a[i],b[i],a[j],b[j])){
                Ans++;
            }
        }
    cout<<Ans<<endl;
    return 0;
}

```


B 挑战多项式

时间限制: 1000ms 内存限制: 204800kb

通过率: 227/229 (99.13%) 正确率: 227/492 (46.14%)

题目描述

这是一道模板题。

输入两个多项式，输出这两个多项式的乘积。

输入格式

第一行两个整数 n 和 m ，分别表示两个多项式的次数。

第二行 $n+1$ 个整数，分别表示第一个多项式的 0 到 n 次项前的系数。

第三行 $m+1$ 个整数，分别表示第二个多项式的 0 到 m 次项前的系数。

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#define l(x) (x<<1)
#define r(x) ((x<<1)+1)
#define LL long long
#define N 800010
#define MOD 998244353
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,len;
LL a[N],b[N];

inline void Swap(LL &a,LL &b){a^=b^=a^=b;}

inline LL Mi(LL x,LL y){
    LL p=x,t=1,Res=1;
    for (;t<=y;(t&y)?Res=(Res*p)%MOD:0,p=(p*p)%MOD,t<<=1);
    return Res;
}

inline void Rader(LL y[]){
    int i=0,j=0,k=0;
    for (i=1,j=len>>1;i<len-1;i++){
        if (i<j) Swap(y[i],y[j]);
        k=len>>1;
        while (j>=k) j-=k,k>>=1;
        if (j<k) j+=k;
    }
}

inline void NTT(LL a[],int f){
    int i=0,j=0,k=0;
    LL wn=0,w=0,u=0,t=0;
```

```

Rader(a);
for (k=2;k<=len;k<=1){
    wn=Mi(3,(MOD-1)/k);
    if (f==-1) wn=Mi(wn,MOD-2);
    for (i=0;i<len;i+=k){
        w=1;
        for (j=i;j<i+k/2;j++){
            u=a[j]; t=(w*a[j+k/2])%MOD;
            a[j]=(u+t)%MOD; a[j+k/2]=(u-t+MOD)%MOD;
            w=(w*wn)%MOD;
        }
    }
}
if (f==-1)
    for (i=0;i<len;i++) a[i]=(a[i]*Mi(len,MOD-2))%MOD;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    scanf("%d%d",&n,&m);
    for (i=0;i<=n;i++) scanf("%lld",&a[i]);
    for (i=0;i<=m;i++) scanf("%lld",&b[i]);
    for (len=1;len<=(n+m);len<=1);
    NTT(a,1); NTT(b,1);
    for (i=0;i<len;i++) a[i]=(a[i]*b[i])%MOD;
    NTT(a,-1);
    for (i=0;i<=n+m;i++) printf("%lld ",a[i]);
    return 0;
}

```

C

C 部落冲突

时间限制：1000ms 内存限制：65536kb

通过率：32/45 (71.11%) 正确率：32/193 (16.58%)

题目描述

Ins 星系中有两颗适宜人类居住的星球。

这次的星际移民中包含了 n 个不同的部落，他们会分散居住在两颗星球上。这些部落之间可能存在仇恨，如果存在仇恨的两个部落居住在一个星球上那么他们之间就会发生部落冲突。部落冲突可大可小，严重冲突是指所有部落冲突中最大的那一个。

如何安排部落的居住方案才能使得部落间发生的严重冲突尽可能的小呢？

输入格式

第一行两个整数 n, m ($1 \leq n \leq 2 \cdot 10^4, 1 \leq m \leq 10^5$)，分别表示部落的数量和部落间存在的仇恨数量。

接下来 m 行每行三个整数 x_i, y_i, w_i ($1 \leq x_i, y_i \leq n, 0 \leq w_i \leq 10^9$)，表示 x_i 与 y_i 部落间存在仇恨，部落冲突的大小是 w_i 。

输出格式

```

#include<bits/stdc++.h>
#define bug printf("!!!!\n");
using namespace std;
typedef long long LL;
typedef pair<LL, LL> PII;

```

```

inline LL read() {
    LL x = 0, fl = 1; char c = getchar();
    while (c < '0' || c > '9') { if(c == '-') fl = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * fl;
}

const int N = 100010, M = 200010, mod = 1e9 + 7;
int n, m, k, top, t, flag;
int h[N], e[M], w[M], ne[M], idx;
void add(int a, int b, int c){
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx ++;
}

int color[N];
bool dfs(int u, int c, int mid) {
    color[u] = c;
    for(int i = h[u]; ~i; i = ne[i]) {
        int j = e[i];
        if(w[i] <= mid) continue;
        if(color[j]) {
            if(color[j] == c) return 0;
        }
        else if(!dfs(j, 3 - c, mid)) return 0;
    }
    return 1;
}

bool check(int mid) {
    memset(color, 0, sizeof color);
    for (int i = 1; i <= n; i ++)
        if(!color[i])
            if(!dfs(i, 1, mid))
                return 0;
    return 1;
}

bool peaceCheck() {
    check(0);
    for (int u = 1; u <= n; u ++) {
        for(int i = h[u]; ~i; i = ne[i]) {
            int j = e[i];
            if(color[u] == color[j]) return 0;
        }
    }
    return 1;
}

int main()
{
    // freopen("ok//zero1.in", "r", stdin);
    // freopen("ok//zero1.out", "w", stdout);
    n = read(), m = read();
    memset(h, -1, sizeof h);
    int l = 0, r = 0;
    while (m --) {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        add(a, b, c);
        add(b, a, c);
        r = max(r, c);
    }
    while (l < r) {

```

```

        int mid = l + r >> 1;
        if(check(mid)) r = mid;
        else l = mid + 1;
    }
    if(!l && peaceCheck()) puts("Peace And Love!");
    else printf("%d\n", l);
    return 0;
}

```

D

D 游戏代练 - Easy

时间限制：1000ms 内存限制：65536kb

通过率：20/29 (68.97%) 正确率：20/104 (19.23%)

题目描述

简单版和困难版的唯一区别是每日任务的数量
简单版只有一个任务
困难版有两个

机缘巧合之下，Alex 进入了一个代练工作室

他需要帮助号主刷取游戏内的金币。

每天上线，Alex 都可以从游戏系统中获取 一个 每日任务，根据任务的难度，会有不同数量的金币奖励。

任务共有 n 个，每个任务完成后可以收获 $coin_i$ 的奖励。

Alex 作为一个资深老玩家，完成每日任务自然不在话下。因此不论任务多难，他都能拿到满额奖励。

这个游戏还有一个机制，每个人每天都有 k 次机会，可以重新抽取每日任务。

每天每个任务出现的概率是固定的，重新抽取也是在所有任务中等概率抽取。

Alex 想知道，如果他充分利用这 k 次抽取的机会，它每天期望可以刷到多少金币？

```

// Author: Alex
#include <bits/stdc++.h>

using namespace std;
const int N = 100100;

int n, k, Q, val[N];
double sum = 0, s[N];

int main() {
    cin >> n >> k >> Q;
    for (int i = 1; i <= n; ++i) {
        cin >> val[i];
        sum += val[i];
    }
    s[0] = 1.0 * sum / n;
    for (int i = 1; i <= k; ++i) {
        sum = 0;
        for (int j = 1; j <= n; ++j)
            sum += max(1.0 * val[j], s[i - 1]);
        s[i] = sum / n;
    }
    printf("%.4lf\n", s[k]);
    for (int i = 1, u, c; i <= Q; ++i){
        scanf("%d%d", &u, &c);
        if (c == 0) puts("accept");
    }
}

```

```

        else {
            double s1 = val[u], s2 = S[c - 1];
            if (fabs(s1 - s2) < 1e-4) puts("both");
            else if (s1 < s2) puts("reselect");
            else puts("accept");
        }
    }
    return 0;
}

```

E

E 结果算出来

时间限制: 1000ms 内存限制: 65536kb

通过率: 219/223 (98.21%) 正确率: 219/446 (49.10%)

题目描述

给定 n 个点，求其凸包周长，本题保证凸包不退化。

输入

第一行一个整数为点数 $n(3 \leq n \leq 200000)$ 。

接下来 n 行，每行两个小数 $x(-10000 \leq x \leq 10000)$, $y(-10000 \leq y \leq 10000)$ 表示横纵坐标。

```

// Author: Marvolo
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define EPS 1e-5
#define N 200010
#define INF 0x3f3f3f3f
using namespace std;

inline double Min(double a,double b){return (a<b)?a:b;}
inline double Max(double a,double b){return (a>b)?a:b;}
inline double Fabs(double x){return (x<=-EPS)?-x:(x>EPS)?x:0;}
inline bool Equ(double x,double y){return (Fabs(x-y)<EPS);}
inline int Sgn(double x){
    if (Fabs(x)==0)    return 0;
    if (x>0)    return 1;    else return -1;
}

struct Vector{
    double x,y;
    void Clean(void){x=y=0;}
}

```

```

void write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
Vector operator +(Vector p){return (Vector){x+p.x,y+p.y};}
Vector operator -(Vector p){return (Vector){x-p.x,y-p.y};}
double operator *(Vector p){return x*p.y-y*p.x;}
double operator &(Vector p){return x*p.x+y*p.y;}
};
typedef Vector V;

struct Point{
    double x,y;
    void Clean(void){x=y=0.0;}
    void Write(void){printf("%.3f %.3f\n",x+EPS,y+EPS);}
    bool operator ==(Point a){return (Equ(x-a.x,0) && Equ(y-a.y,0));}
    Point operator +(Point p){return (Point){x+p.x,y+p.y};}
    Point operator /(double l){return (Point){x/l,y/l};}
    Vector operator -(Point p){return (Vector){x-p.x,y-p.y};}
};
typedef Point P;

inline void SW(P &a,P &b){P t=a;    a=b;    b=t;}
inline double Dis2(P a,P b){return pow(a.x-b.x,2)+pow(a.y-b.y,2);}
inline double Dis(P a,P b){return pow(pow(a.x-b.x,2)+pow(a.y-b.y,2),0.5);}

P O;

inline bool Cmp(P a,P b){
    double l=(a-O)*(b-O);
    if (l>0 || (Equ(l,0) && Fabs(a.x-O.x)<Fabs(b.x-O.x)))
        return 1;
    else return 0;
}

inline void Sort(P a[],int n){
    register int i=0;
    for (i=2;i<=n;i++){
        if (a[i].x<a[1].x || (a[i].x==a[1].x && a[i].y<a[1].y))
            SW(a[1],a[i]);
        O=a[1];
        sort(a+1,a+1+n,Cmp);
    }
}

inline void Convex(P a[],P b[],int n,int &l){
    register int i=0;    l=0;
    Sort(a,n);
    n=unique(a+1,a+1+n)-(a+1);
    if (n<=1){
        b[1]=a[1];    l=1;    return;
    }
    if (Equ((a[2]-a[1])*(a[n]-a[1]),0)){
        b[1]=a[1];    b[2]=a[n];    l=2;    return;
    }
    b[++l]=a[1];    b[++l]=a[2];
    for (i=3;i<=n;i++){
        while ((b[l]-a[i])*(b[l-1]-a[i])>=0 && l>1)    l--;
        b[++l]=a[i];
    }
    if (Equ((b[l]-b[l-1])*(b[l]-b[1]),0))    --l;
    //加上这句可以去掉所有凸包边上的点

```

```

}

inline double Convex_Perimeter(P a[],int n){
    double len=0;
    for (int i=1;i<n;i++)    len+=Dis(a[i],a[i+1]);
    if (n>2)    len+=Dis(a[1],a[n]);
    return len;
}

int n,i,m;
P a[N],b[N];

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    scanf("%d",&n);
    for (i=1;i<=n;i++)
        scanf("%lf%lf",&a[i].x,&a[i].y);
    Convex(a,b,n,m);
    printf("%.2lf\n",Convex_Perimeter(b, m));
    return 0;
}

```

F

F Word Chain

时间限制：1000ms 内存限制：65536kb

通过率：26/41 (63.41%) 正确率：26/137 (18.98%)

题目描述

相信大家都玩过成语接龙的游戏，比如说**心心相印**→**印贼做父**→**父相伤害**→.....我们可以将它们拼接起来，如**心心相印贼做父相伤害**。

为了简化问题，避免处理宽字符，下面我们用英文单词来模拟这一过程。规定如果一个单词a的最后一个字符和另一个单词b的第一个字符相同，则可以将单词b接在单词a的后面。且拼接后的结果衔接部分只出现一次，并大写。

比如说单词"abandon"和单词"naive"就可以拼接，因为"abandon"的最后一个字母与"naive"的第一个字母相同，都为'n'。于是得到拼接后的结果为"abandoNaive"。

下面给出n个单词，请判断这些单词是否能够拼接成一个环。要求每个单词都必须使用且只能使用一次。

本题采用 spj。

```

#include<bits/stdc++.h>
using namespace std;
#define rep(i, a, n) for(int i = a; i <= n; i++)
#define _rep(i, n, a) for(int i = n; i >= a; i--)
const int maxn = 100, inf = 2147483647;
int n, x, y, f1, f2, cnt, st, top, len, fl;
char s[1010][20];
int f[maxn], in[maxn], g[maxn][maxn][1010], ans[1010];
inline void dfs(int x){
    rep(y, 0, maxn - 10){
        if(g[x][y][0] > 0){
            int i = g[x][y][g[x][y][0]];
            g[x][y][0]--;
            dfs(y);
            ans[top++] = i;
        }
    }
}

```

```

    }
}
}
inline int find(int x){
    return f[x] == x ? x : f[x] = find(f[x]);
}
char op;
int main(){
    scanf("%d", &n);
    rep(i, 0, maxn - 10) f[i] = i;
    rep(i, 1, n){
        scanf("%s", s[i]);
        len = strlen(s[i]);
        x = s[i][0] - 'a', y = s[i][len - 1] - 'a';
        g[x][y][0]++;
        g[x][y][g[x][y][0]] = i;
        f1 = find(x), f2 = find(y);
        if(f1 != f2) f[f1] = f2;
        in[x]++; in[y]++;
        if(!f1) st = x, f1 = 1;
    }
    rep(i, 0, maxn - 10) if(f[i] == i && in[i]) cnt++;
    if(cnt > 1){
        cout << "None";
        exit(0);
    }
    rep(i, 0, maxn - 10){
        if(in[i] & 1){
            cout << "None";
            exit(0);
        }
    }
    dfs(st);
    f1 = 0;
    _rep(i, top - 1, 0){
        len = strlen(s[ans[i]]);
        printf("%c", s[ans[i]][0] - 'a' + 'A');
        if(!f1){
            f1 = 1; op = s[ans[i]][0] - 'a' + 'A';
        }
        if(len > 2){
            rep(j, 1, len - 2) printf("%c", s[ans[i]][j]);
        }
    }
    printf("%c", op);
}

```


G Label Segments

时间限制: 1000ms 内存限制: 65536kb

通过率: 15/24 (62.50%) 正确率: 15/127 (11.81%)

题目描述

Marvolo 刚改完他的 sampler 论文, 现在他想做一个玩具线段数据集来补实验。

首先他需要一个简单的程序来标注一些平面内线段之间的关系; 注意, 这个程序需要同时标注**位置关系**和**相交关系**。

平面内两条线段的位置关系有三种:

- 平行或共线: 描述为 **Parallel or Collinear**
- 垂直 (不一定要相交): 描述为 **Perpendicular**
- 普通 (非第一种且非第二种): 描述为 **Vanilla**

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define EPS 1e-5
#define INF 0x3f3f3f3f
using namespace std;

inline double Min(double a,double b){return (a<b)?a:b;}
inline double Max(double a,double b){return (a>b)?a:b;}
inline double Fabs(double x){return (x<=-EPS)?-x:(x>EPS)?x:0;}
inline bool Equ(double x,double y){return (Fabs(x-y)<EPS);}

inline int Sgn(double x){
    if (Fabs(x)==0) return 0;
    if (x>0)    return 1;    else return -1;
}

struct Vector{
    double x,y;
    void Clean(void){x=y=0;}
    void write(void){printf("%.31f %.31f\n",x+EPS,y+EPS);}
    Vector operator +(Vector p){return (Vector){x+p.x,y+p.y};}
    Vector operator -(Vector p){return (Vector){x-p.x,y-p.y};}
    double operator *(Vector p){return x*p.y-y*p.x;}
    double operator &(Vector p){return x*p.x+y*p.y;}
};
typedef Vector V;

struct Point{
    double x,y;
    void Clean(void){x=y=0.0;}
    void write(void){printf("%.31f %.31f\n",x+EPS,y+EPS);}
    bool operator ==(Point a){return (Equ(x-a.x,0) && Equ(y-a.y,0));}
    Point operator +(Point p){return (Point){x+p.x,y+p.y};}
```

```

    Point operator /(double l){return (Point){x/l,y/l};}
    Vector operator -(Point p){return (Vector){x-p.x,y-p.y};}
};
typedef Point P;

inline void SW(P &a,P &b){P t=a;    a=b;    b=t;}
inline double Dis2(P a,P b){return pow(a.x-b.x,2)+pow(a.y-b.y,2);}
inline double Dis(P a,P b){return pow(pow(a.x-b.x,2)+pow(a.y-b.y,2),0.5);}

struct Circle{
    Point o;
    double r;
    void Clean(void){o.x=o.y=0.0;}
    void write(void){printf("%.31f %.31f %.31f\n",o.x,o.y,r);}
    bool operator &(Circle a){return (Dis(o,a.o)+r <= a.r+EPS);}
};
typedef Circle C;

inline double Convex_Size(P a[],int n){
    register int i=0,t=0;
    double s=0;
    P o;
    o.x=o.y=0;
    for (i=1;i<=n;i++){
        t=(i<n)?i+1:1;
        s+=(a[i]-o)*(a[t]-o);
    }
    return s/2;
}

inline bool Point_On_Seg(P x,P y,P z){
    double s;
    s=(x-y)*(x-z);
    if (Fabs(s)>EPS)    return 0;
    if (Min(y.x,z.x)-EPS>x.x||Max(y.x,z.x)+EPS<x.x) return 0;
    if (Min(y.y,z.y)-EPS>x.y||Max(y.y,z.y)+EPS<x.y) return 0;
    return 1;
}

inline bool Seg_Cross(P a,P b,P c,P d){
    if (Point_On_Seg(a,c,d) || Point_On_Seg(b,c,d) || Point_On_Seg(c,a,b) ||
    Point_On_Seg(d,a,b))    return 1;
    if (Sgn((b-a)*(c-a)) * Sgn((b-a)*(d-a)) >= 0)    return 0;
    if (Sgn((d-c)*(a-c)) * Sgn((d-c)*(b-c)) >= 0)    return 0;
    return 1;
}

int T,i;
P a,b,c,d;

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    scanf("%d",&T);
    while (T--){

```

```
scanf("%lf%lf%lf%lf%lf%lf%lf%lf", &a.x, &a.y, &b.x, &b.y, &c.x, &c.y, &d.x, &d.y);
    if (Equ((b-a)*(d-c), 0)) printf("Parallel or Collinear; ");
    else if (Equ((b-a)&(d-c), 0))    printf("Perpendicular; ");
    else printf("Vanilla; ");

    if (!Seg_Cross(a, b, c, d))    printf("None\n");
    else {
        int s=0;
        s+=Point_On_Seg(a, c, d);
        s+=Point_On_Seg(b, c, d);
        s+=Point_On_Seg(c, a, b);
        s+=Point_On_Seg(d, a, b);

        if (s==0)    printf("Unique (X)\n"); else
        if (s==1)    printf("Unique (T)\n"); else{
            if (s==2 && (a==c || a==d || b==c || b==d)) printf("Unique
(V)\n"); else
                if (s==2 && !Equ((b-a)*(d-c), 0))    printf("Unique (V)\n");
                else printf("Infinite\n");
        }
    }
}
return 0;
}
```

H

H KISS

时间限制: 1000ms 内存限制: 131072kb

通过率: 1/3 (33.33%) 正确率: 1/42 (2.38%)

题目描述

Keep it Simple, Keep it Stupid.

KISS 原则是 *Linux* 的核心设计原则之一，在算法设计的很多场景，简单胜于复杂。

资源是有限的，有些时候寻找最优解的代价，远大于寻找近似解的代价。

但是 *Matrix53* 依旧需要你在本题中寻找最优解（光速逃跑

求第二优的二叉搜索树的搜索代价，若有两个最优则输出最优。

搜索关键字出现的频率为 $f_1, f_2 \dots f_n$ ，伪关键字出现的频率为 $p_0, p_1 \dots p_n$ 。

先以关键字的序号为键建树，再将伪关键字也作为结点加到搜索树上，结点深度 $depth(x) = depth(father(x)) + 1$, $depth(root) = 1$ 。

一颗搜索树关于上述频率的搜索代价为 $\sum(d_i \times v_i)$ ， v_i 为结点对应的(伪)关键字的出现频率， d_i 为结点深度。

```
// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn = 2e3 + 5;

ll dp[maxn][maxn], w[maxn][maxn];
int f[maxn], p[maxn], root[maxn][maxn];
```

```

11 dfs(int l, int r) {
12     res = LONG_LONG_MAX;
13     if (l + 1 == r) {
14         int k = l + r - root[l][r];
15         res = dp[l][k - 1] + dp[k + 1][r] + w[l][r];
16     } else {
17         if (2 <= root[l][r] - 1 - l + 1)
18             res = min(res, dfs(l, root[l][r] - 1) + dp[root[l][r] + 1][r] + w[l][r]);
19         if (2 <= r - root[l][r] - 1 + 1)
20             res = min(res, dfs(root[l][r] + 1, r) + dp[l][root[l][r] - 1] + w[l][r]);
21         for (int i = l; i <= r; ++i) {
22             if (i != root[l][r]) {
23                 res = min(res, dp[l][i - 1] + dp[i + 1][r] + w[l][r]);
24             }
25         }
26     }
27     return res;
28 }

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 1; i <= n; ++i) scanf("%lld", &f[i]);
    for (int i = 0; i <= n; ++i) scanf("%lld", &p[i]);
    for (int i = 1; i <= n + 1; ++i) {
        root[i][i - 1] = i;
        dp[i][i - 1] = p[i - 1];
        w[i][i - 1] = p[i - 1];
    }
    for (int len = 1; len <= n; ++len) {
        for (int l = 1; l <= n - len + 1; ++l) {
            int r = l + len - 1;
            dp[l][r] = LONG_LONG_MAX;
            w[l][r] = w[l][r - 1] + f[r] + p[r];
            for (int mid = root[l][r - 1]; mid <= root[l + 1][r]; ++mid) {
                ll sum = dp[l][mid - 1] + dp[mid + 1][r] + w[l][r];
                if (dp[l][r] > sum) {
                    dp[l][r] = sum;
                    root[l][r] = mid;
                }
            }
        }
    }
    printf("%lld", dfs(1, n));
    return 0;
}

```

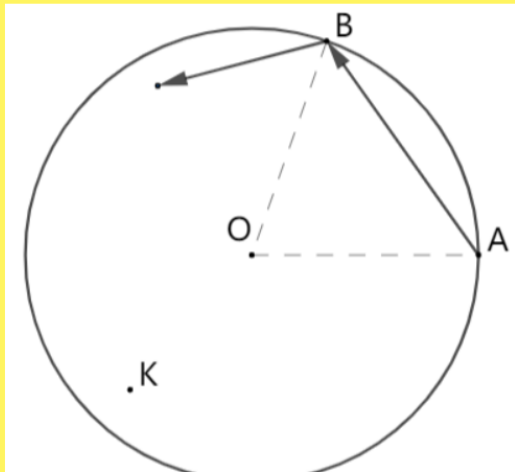
I 光与镜

时间限制: 1000ms 内存限制: 65536kb

通过率: 9/15 (60.00%) 正确率: 9/74 (12.16%)

题目描述

有一块圆形镜子，有一束不消失的光。



```
// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;

const double pi = acos(-1.0);
const double eps = 1e-6;

int sgn(double x) {
    if (fabs(x) < eps) return 0;
    return x > 0 ? 1 : -1;
}

struct Point {
    double x, y;
    Point() : x(0), y(0) {}
    Point(double a, double b) : x(a), y(b) {}
    Point operator-(Point a) { return Point(x - a.x, y - a.y); }
};

typedef Point Vector;
double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
double det(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
//向量A逆时针旋转rad弧度
Vector rotate(Vector a, double rad) {
    return Vector(a.x * cos(rad) - a.y * sin(rad),
                  a.x * sin(rad) + a.y * cos(rad));
}

struct Line {
    Point p1, p2;
    Line() {}
    Line(Point a, Point b) : p1(a), p2(b) {}
};

typedef Line Segment;
//点和线段的关系: 0为在线段延长线上, 1为线段端点, 2为在线段上, 3为左侧, 4为右侧
```

```

int relation(Point p, Segment s) {
    int sign = sgn(det(s.p2 - s.p1, p - s.p1));
    if (sign) return sign > 0 ? 3 : 4;
    sign = sgn(dot(s.p2 - p, s.p1 - p));
    if (sign) return sign > 0 ? 0 : 2;
    return 1;
}

int main() {
    // freopen("D:/workspace/algo/E6-H/data/4.in", "r", stdin);
    Point O, A, K, T1, T2;
    int alpha;
    scanf("%lf%lf", &O.x, &O.y);
    scanf("%lf%lf%d", &A.x, &A.y, &alpha);
    scanf("%lf%lf", &K.x, &K.y);
    K = K - O;
    A = A - O;
    for (int count = 0; count < 365; ++count) {
        T1 = rotate(A, ((alpha * 2 * count) % 360) * pi / 180);
        T2 = rotate(A, ((alpha * 2 * (count + 1)) % 360) * pi / 180);
        int res = relation(K, Segment(T1, T2));
        if (res == 1 || res == 2) {
            printf("%d", count);
            return 0;
        }
    }
    printf("-1");
    return 0;
}

```

J

J 游戏代练 - Hard

时间限制：1000ms 内存限制：65536kb

通过率：2/5 (40.00%) 正确率：2/32 (6.25%)

题目描述

简单版和困难版的唯一区别是每日任务的数量
简单版只有一个任务
困难版有两个

机缘巧合之下，Alex 进入了一个代练工作室

他需要帮助号主刷取游戏内的金币。

每天上线，Alex 都可以从游戏系统中获取 **两个** 每日任务，**且不会出现相同的任务** 根据任务的难度，会有不同数量的金币奖励。

任务共有 n 个，每个任务完成后可以收获 $coin_i$ 的奖励。

Alex 作为一个资深老玩家，完成每日任务自然不在话下。因此不论任务多难，他都能拿到满额奖励。

这个游戏还有一个机制，每个人每天都有 k 次机会，可以重新抽取每日任务。

每天每个任务出现的概率是固定的，重新抽取也是在所有任务中等概率抽取，**一次重抽会将两个任务都进行刷新，刷新后不会出现相同的任务。**

Alex 想知道，如果他充分利用这 k 次抽取的机会，它每天期望可以刷到多少金币？

```

#include <bits/stdc++.h>
#define maxn 2000466
using namespace std;
using i64 = long long;
i64 A[maxn * 3], B[maxn * 3], C[maxn * 3], limit, L;

```

```

const i64 MOD = 998244353, G = 3, Gi = 332748118;
int r[maxn * 3];
i64 Pow(i64 x, i64 p) {
    i64 ret = 1;
    for (x %= MOD, p %= (MOD - 1); p; p >>= 1, x = x * x % MOD) if (p & 1) ret =
ret * x % MOD;
    return ret;
}
void NTT(i64 *a, int f) {
    for (int i = 0; i < limit; ++i) if (i < r[i]) swap(a[i], a[r[i]]);
    for (int mid = 1; mid < limit; mid <= 1) {
        i64 wn = Pow(f == 1 ? G : Gi, (MOD - 1) / (mid << 1));
        for (int j = 0; j < limit; j += (mid << 1)) {
            i64 w = 1;
            for (int k = 0; k < mid; ++k, w = w * wn % MOD) {
                i64 x = a[j + k], y = w * a[j + k + mid] % MOD;
                a[j + k] = (x + y) % MOD;
                a[j + k + mid] = (x - y + MOD) % MOD;
            }
        }
    }
}
int aa[maxn];
double E[2333];
int main() {
    int n, k, Q;
    scanf("%d%d%d", &n, &k, &Q);
    for (int i = 1; i <= n; ++i) scanf("%d", aa + i), ++A[aa[i]];
    limit = 1, L = 0;
    while (limit <= 2000000) limit <= 1, ++L;
    for (int i = 0; i < limit; ++i) r[i] = (r[i >> 1] >> 1) | ((i & 1) << (L -
1));
    NTT(A, 1);
    for (int i = 0; i < limit; ++i) A[i] = A[i] * A[i] % MOD;
    NTT(A, -1);
    for (i64 i = 0, ii = Pow(limit, MOD - 2); i < limit; ++i) A[i] = A[i] * ii %
MOD;
    for (int i = 1; i <= n; ++i) --A[aa[i] * 2];
    i64 tot = (i64) n * (n - 1);
    i64 sum = 0, ss = 0;
    for (int i = 0; i < limit; ++i) sum += A[i] * i;
    E[0] = (double) sum / (double) tot;
    for (int i = 1, j = 0; i <= k; ++i) {
        while (j < E[i - 1] && j < limit) sum -= A[j] * j, ss += A[j], ++j;
        E[i] = ((double) sum + E[i - 1] * (double) ss) / (double) tot;
    }
    printf("%.4lf\n", E[k]);
    for (int i = 1; i <= Q; ++i) {
        int x, y, c;
        scanf("%d%d%d", &x, &y, &c);
        if (c == 0) puts("accept");
        else {
            double s1 = aa[x] + aa[y], s2 = E[c - 1];
            if (fabs(s1 - s2) < 1e-4) puts("both");
            else if (s1 < s2) puts("reselect");
            else puts("accept");
        }
    }
}

```

```
    return 0;  
}
```


E7

A

A 平行六面体

时间限制: 1000ms 内存限制: 65536kb

通过率: 236/237 (99.58%) 正确率: 236/461 (51.19%)

题目描述

给定表示平行六面体三条共点棱的向量，求该平行六面体体积。

P.S. \$Matrix53\$ 不想编题面了。

```
// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;

struct Point3 {
    double x, y, z;
    Point3() {}
    Point3(double a, double b, double c) : x(a), y(b), z(c) {}
};

typedef Point3 Vector3;
double dot(Vector3 a, Vector3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
Vector3 det(Vector3 a, Vector3 b) {
    return Point3(a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z,
                  a.x * b.y - a.y * b.x);
}

int main() {
    Point3 a, b, c;
    scanf("%lf%lf%lf%lf%lf%lf%lf%lf", &a.x, &a.y, &a.z, &b.x, &b.y, &b.z, &c.x,
          &c.y, &c.z);
    printf("%.1f", fabs(dot(det(a, b), c)));
    return 0;
}
```

B

题目描述

Arcaea 里的一些乐曲具有“天地交互”的片段。

给定表示乐谱的字符串 s , 表示“天地交互”的字符串 t 。

求“天地交互”出现的所有位置。

P.S. *Matrix53* 很喜欢这款音游。

```
// Author: Matrix53
#include <bits/stdc++.h>
using namespace std;

const int maxl = 1e6 + 5;

int fail[maxl];

void getFail(char *p, int plen) {
    fail[0] = 0;
    fail[1] = 0;
    for (int i = 1; i < plen; ++i) {
        int j = fail[i];
        while (j > 0 && p[i] != p[j]) j = fail[j];
        fail[i + 1] = (p[i] == p[j] ? j + 1 : 0);
    }
}

void kmp(char *p, char *s, int plen, int slen) {
    int last = -1;
    getFail(p, plen);
    int j = 0;
    for (int i = 0; i < slen; ++i) {
        while (j && s[i] != p[j]) j = fail[j];
        if (s[i] == p[j]) ++j;
        if (j == plen) {
            printf("%d ", i + 1 - plen);
        }
    }
}

char s[maxl], t[maxl];

int main() {
    scanf("%s%s", s, t);
    kmp(t, s, strlen(t), strlen(s));
    return 0;
}
```

题目描述

insere 助教有一个长度为 n 的只由 0 和 1 组成的序列 a ，他想要移除最多 $\frac{n}{2}$ 个元素，使得剩下的序列满足 $a_1 - a_2 + a_3 - a_4 + \dots = 0$ 。换言之剩下的序列中位于奇数位置的元素和等于位于偶数位置的元素和。

输入格式

第一行一个整数 $t(1 \leq t \leq 10^3)$ ，表示测试数据的组数。

每组测试数据的第一行包含了一个整数 $n(2 \leq n \leq 10^3)$ 且 n 是偶数，表示序列的长度。

第二行包含了 n 个整数 $a_1, a_2, \dots, a_n(0 \leq a_i \leq 1)$ 。

```
#include<bits/stdc++.h>
#define bug printf("!!!!!\n");
using namespace std;
typedef long long LL;
typedef pair<LL, LL> PII;
inline LL read() {
    LL x = 0, fl = 1; char c = getchar();
    while (c < '0' || c > '9') { if(c == '-') fl = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * fl;
}
const int N = 1010, M = 100010, mod = 1e9 + 7;
int n, m, k, top, t;
int a[N];
int main()
{
    // freopen("data5.in", "r", stdin);
    // freopen("data5.out", "w", stdout);
    t = read();
    while (t --) {
        n = read();
        int cnt0 = 0, cnt1 = 0;
        for (int i = 1; i <= n; i ++) {
            a[i] = read();
            if(a[i] == 0) cnt0 ++;
            else cnt1 ++;
        }
        if(cnt0 >= cnt1) {
            printf("%d\n", cnt0);
            for (int i = 0; i < cnt0; i ++) {
                if(i) printf(" ");
                printf("0");
            }
        }
        else {
            if(cnt1 & 1) {
                cnt1 --;
            }
            printf("%d\n", cnt1);
            for (int i = 0; i < cnt1; i ++) {
                if(i) printf(" ");
                printf("1");
            }
        }
    }
}
```

```
    puts("");
}

return 0;
}
```

D

D 做盲盒

时间限制：1000ms 内存限制：65536kb

通过率：222/228 (97.37%) 正确率：222/412 (53.88%)

题目描述

一玩具公司准备设计一款盲盒。他们在一个生产周期中生产出的玩具共有三种：红色，蓝色与无色。盲盒的制作需要满足如下条件：

- 盲盒里必须有恰好3件玩具
- 里面至少有1件红色玩具和1件蓝色玩具

现在已知了 T 个生产周期的玩具生产计划，请计算出每个周期可以制作出最多多少个盲盒。（各个周期之间相互独立）

```
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#define l(x) (x<<1)
#define r(x) ((x<<1)+1)
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int T,x,y,z,cnt,Ans;
int low,high,mid;

inline int Abs(int x){return (x<0)?-x:x;}
inline void Swap(int &a,int &b){a^=b^=a^=b;}
inline int Min(int a,int b){return (a<b)?a:b;}
inline int Max(int a,int b){return (a>b)?a:b;}

inline int read(){
    int p=0;    char    c=getchar();
    while (c<48||c>57)    c=getchar();
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p;
}

inline bool Check(int p){
    int sum=0;
    if (p>x||p>y)    return 0;
    sum=x-p+y-p;
    sum+=z;
    if (sum>=p)    return 1;
    else return 0;
}
```

```

int main(){
    T=read();
    while (T--){
        Ans=0;
        x=read();    y=read();    z=read();
        if (Min(x,y)==0)    {
            printf("0\n");
            continue;
        }
        low=0;    high=Max(Max(x,y),z);
        while (low<=high){
            mid=(low+high)>>1;
            if (Check(mid))    Ans=mid,low=mid+1;
            else high=mid-1;
        }
        printf("%d\n",Ans);
    }
    return 0;
}

```

E

E Phigros

时间限制: 1000ms 内存限制: 65536kb

通过率: 82/129 (63.57%) 正确率: 82/308 (26.62%)

题目描述

Phigros 中, 乐谱可以从左右两侧同时出现

当左右两侧出现相似片段时 (人话: 某个长为 L 前缀和长为 L 的后缀相同时), 节奏感会更强

给的表示乐谱的字符串 s , 求所有节奏感较强的片段长度

P.S. 我不知道 *Matrix53* 喜不喜欢这款音游。

```

// Author: Alex
#include <bits/stdc++.h>

using namespace std;

const int N = 1001000;

int n, m, ans[N], Next[N];
char str[N];

void get(char s[], int n) {
    Next[1] = 0;
    for(int i = 2, j = 0; i <= n; ++i) {
        while(j && s[j + 1] != s[i]) j = Next[j];
        if(s[j + 1] == s[i]) j++;
        Next[i] = j;
    }
    int now = n;
    while (now) {
        ans[++m] = now;
    }
}

```

```
        now = Next[now];
    }
}

int main() {
    scanf("%s", str + 1); n = strlen(str + 1); get(str, n);
    while(m--) printf("%d\n", ans[m + 1]);
    return 0;
}
```

F

F 带障碍的跳格子

时间限制：1000ms 内存限制：65536kb

通过率：214/224 (95.54%) 正确率：214/482 (44.40%)

题目描述

地上从左到右一共有 n 个格子，从最左边的格子跳到最右边，每次只能向右跳，跳跃长度随意，一共有多少种跳法，其中部分格子有障碍物，有障碍物的格子不能停留。

输入

第一行一个整数为格子数 $n(2 \leq n \leq 100000)$ 。

第二行一个长度为 n 的数列，用空格隔开，1表示有障碍，0表示无障碍。保证最左端和最右端为0。

```
#include<bits/stdc++.h>
using namespace std;
#define mod 1000000007

int main() {
    int n,k,ans=1;
    scanf("%d",&n);
    for(int i=0; i<n; i++) {
        scanf("%d",&k);
        if(!k&&i&&i!=n-1) ans*=2;
        ans%=mod;
    }
    printf("%d",ans);
    return 0;
}
```

G

G 完美的星图

时间限制：1000ms 内存限制：65536kb

通过率：44/53 (83.02%) 正确率：44/119 (36.97%)

题目描述

Marvolo最近迷上了天文学，他打算绘制一幅星图出来。通过观测，他得到了 n 颗恒星的数据，简便起见，可以认为恒星都位于一个二维平面上，第 i 颗恒星的坐标是 (i, a_i) 。两颗恒星 i, j 之间的距离定义为： $d(i, j) = |i - j| + |a_i - a_j|$ （也就是著名的曼哈顿距离）。

星图是由一些精心挑选出的恒星绘制而成的。对一个星图来说，如果它里面不存在**不同的三颗恒星** i, j, k ，满足 $d(i, k) = d(i, j) + d(j, k)$ 的话，那么这个星图就是一个完美的星图。现在，Marvolo打算从已有的 n 个恒星中，抽取出**编号连续的一段恒星序列**绘制星图，请问他一共能绘制出多少种不同的完美的星图呢？

特别的，如果一个星图只有1颗或者2颗恒星，也认为它是完美的星图。

```

#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define N 200010
#define INF 0x3f3f3f3f
using namespace std;

int T,n,i,j,Ans;
int a[N];

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&&c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

IL bool Check(int l,int r){
    reg int i=0,j=0,k=0;
    for (i=l;i<r;i++)
        for (j=i+1;j<r;j++)
            for (k=j+1;k<=r;k++){
                if (a[i]<=a[j] && a[j]<=a[k])    return 0;
                if (a[i]>=a[j] && a[j]>=a[k])    return 0;
            }
    return 1;
}

int main(){
    #ifdef __Marvolo
    freopen("5.in","r",stdin);
    freopen("5.out","w",stdout);
    #endif
    n=read();
    for (i=1;i<=n;i++)    a[i]=read();
    Ans=n+n-1;
    for (i=1;i<=n;i++)
        for (j=i+2;j<=i+3 && j<=n;j++){
            if (Check(i,j)) Ans++;
            else break;
        }
    printf("%d\n",Ans);
}

```

```
    return 0;
}
```

H

H Fill the tree

时间限制: 1000ms 内存限制: 65536kb

通过率: 39/58 (67.24%) 正确率: 39/133 (29.32%)

题目

无向完全图是每对顶点之间都恰连有一条无向边的简单图。

给定一个无向完全图的唯一的最小生成树上所有边的端点和权值, 其余的边权值未知, 问这个无向完全图所有边权值和的最小值。

输入

第一行一个整数 T 表示数据组数 ($1 \leq T \leq 10$)

每组数据第一行一个正整数 n , 表示完全图的点数 ($2 \leq n \leq 10^5$)

接下来 $n - 1$ 行, 每行三个整数 x, y, z , 表示 x, y 之间有一条权值为 z 的无向边 ($1 \leq x, y \leq n, 1 \leq z \leq 10000$)

```
#include<bits/stdc++.h>
#define rep(i, a, n) for(int i = a; i <= n; i++)
#define maxn 100010
using namespace std;
int t, n, u, v, w;
int a[maxn], f[maxn], cnt[maxn];
inline int read(){
    int k = 0, f = 1;
    char c = getchar();
    while(c < '0' || c > '9') { if(c == '-') f = -1; c = getchar(); }
    while(c >= '0' && c <= '9') { k = k * 10 + c - 48; c = getchar();}
    return k * f;
}
struct edge{
    int u, v, w;
}edges[maxn];

inline int find(int x){
    return f[x] == x ? x : f[x] = find(f[x]);
}

inline void init(){
    rep(i, 1, n){
        f[i] = i; cnt[i] = 1;
    }
}

bool cmp(edge a, edge b){
    return a.w < b.w;
}

int main(){
    t = read();
```



```

while(t--){
    n = read();
    init();
    rep(i, 0, n - 2){
        edges[i].u = read();
        edges[i].v = read();
        edges[i].w = read();
    }
    sort (edges, edges + n - 1, cmp);
    long long ans = 0;
    rep(i, 0, n - 2){
        int f1 = find(edges[i].u);
        int f2 = find(edges[i].v);
        ans += edges[i].w;
        if(f1 != f2){
            f[f1] = f[f2];
            ans += (edges[i].w + 111) * 111 * (cnt[f1] * 111 * cnt [f2] -
111);

            cnt[f2] += cnt[f1];
            cnt[f1] = 0;
        }
    }
    printf("%lld\n", ans);
}
}

```

I 刷题机器

时间限制：1000ms 内存限制：65536kb

通过率：225/229 (98.25%) 正确率：225/281 (80.07%)

题目描述

现在有 n 道题，每道题都有一个难度，小黄是个刷题机器，他在第 i 天能可以刷难度为 i 的题，而且他做题飞快，时间可以忽略不计，那么请问，从第1天到第 m 天，他每天最多可能已经刷过多少题。

```

// Author: Marvolo
#include<cstring>
#include<cstdio>
#include<iostream>
#include<algorithm>
#include<vector>
#include<cmath>
#include<queue>
#include<map>
#define l(x) (x<<1)
#define r(x) ((x<<1)|1)
#define IL inline
#define reg register
#define LL long long
#define INF 0x3f3f3f3f
using namespace std;

int n,m,i,x,y;

```

```

map <int,int> M;

IL int Abs(int x){return (x<0)?-x:x;}
IL void Swap(int &a,int &b){a^=b^=a^=b;}
IL int Min(int a,int b){return (a<b)?a:b;}
IL int Max(int a,int b){return (a>b)?a:b;}

IL int read(){
    int p=0,f=1;    char    c=getchar();
    while (c<48||c>57) {if (c=='-')    f=-1;    c=getchar();}
    while (c>=48&& c<=57)    p=(p<<1)+(p<<3)+c-48,c=getchar();
    return p*f;
}

int main(){
    #ifdef __Marvolo
    freopen("zht.in","r",stdin);
    freopen("zht.out","w",stdout);
    #endif
    n=read();    m=read();
    for (i=1;i<=n;i++)    M[read()]++;
    for (i=1;i<=m;i++){
        x += M[i];
        printf("%d ",x);
    }
    return 0;
}

```

J

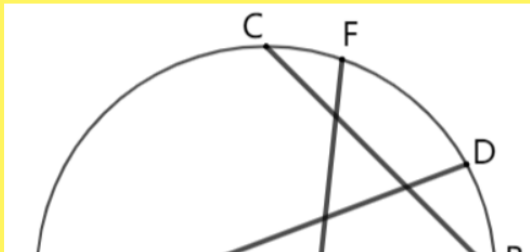
J 刀与镜

时间限制: 1000ms 内存限制: 65536kb

通过率: 5/30 (16.67%) 正确率: 5/102 (4.90%)

题目描述

有一块圆形镜子，有一把永不磨损的刀



```

// Author: Alex
#include <bits/stdc++.h>

using namespace std;

typedef pair<double, double> pdd;

const int N = 500100;

```

```

vector<pdd> A;
vector<double> B;
map<double, int> S;

struct Bit {
    #define Maxn 1000100
    long long val[Maxn];
    inline long long lowbit(int x) {return x & -x;}
    inline void add(int x, long long v) {while (x < Maxn) {val[x] += v; x +=
lowbit(x);}}
    inline long long ask(int x) {int res = 0; while (x) {res += val[x]; x -=
lowbit(x);} return res;}
    inline long long query(int l, int r) {return ask(r) - ask(l - 1);}
} bit;

int main() {
    int n; cin >> n;
    for (int i = 1; i <= n; ++i) {
        double x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
        double rho1 = atan2(y1, x1), rho2 = atan2(y2, x2);
        A.push_back(make_pair(min(rho1, rho2), max(rho1, rho2)));
        B.push_back(rho1); B.push_back(rho2);
    }
    sort(A.begin(), A.end());
    sort(B.begin(), B.end());
    for (int index = 0; index < B.size(); ++index)
        S[B[index]] = index + 1;
    long long ans = 0;
    for (auto &&[l, r] : A) {
        int L = S[l], R = S[r];
        ans += bit.query(L, R);
        bit.add(R, 1);
    }
    cout << ans << endl;
    return 0;
}

```